

In [269...

```
ClearAll["Global`*"];
$HistoryLength=0;

Print["====="];
Print["S124-T3 STANDALONE"];
Print["STREAMING TRANSFORMER PERCEPTION -> TCCT REASONING"];
Print["====="];
Print["WolframVersion=", $Version];
Print["Date=",DateString[]];

T3Stop[msg_] := (Print["FATAL: ", msg]; Abort[]);

r3mDepth=6;
r3mAnchors={1,2,3};
r3mHub=10;
r3mWLRounds=4;
r3mSupports={2,4,6};
r3mEffects={0.,0.05};
r3mTopMins={0.30,0.35};
r3mMinChildStates=2;
r3mMinGroupObs=80;
r3mMinValGain=0.001;
r3mPenalty=0.00025;
r3mMaxSplits=12;
r3mMaxBlocks=24;
r3mKappa=8.;
r3mZero=ConstantArray[0,{24,4}];

t3CoreSeed=1134001;
t3WorldSeed=7241101;
t3BankAB=15000;
t3BankC=6000;

t3PerceptionTrainPerClass=1200;
t3PerceptionValPerClass=300;

t3MicroLen=4;
t3SensorDim=12;
t3PosDim=8;
t3SensorScale=2.;
t3SensorNoise=0.30;
t3SensorDrift=0.12;
t3SensorGainJitter=0.12;

t3DModel=64;
t3Heads=4;
t3Layers=2;
t3FF=192;
t3Dropout=0.10;

t3LearningRate=0.0003;
t3BatchSize=64;
t3MaxRounds=20;
t3Patience=5;
```

```

t3TargetDevice="CPU";

t3PrototypeSeed=7242001;
t3PerceptionTrainSeed=7242002;
t3PerceptionValSeed=7242003;
t3PerceptionNetSeed=7242004;
t3SensorCSeed=7242005;
t3RandomControlSeed=7242006;

t3EventAccuracyGate=0.95;
t3ExactHistoryGate=0.80;
t3ValidHistoryGate=0.80;
t3QPreservationGate=0.90;
t3GainRetentionGate=0.85;

If[OddQ[t3PosDim],T3Stop["t3PosDim must be even"]];
If[Mod[t3DModel,t3Heads]!=0,T3Stop["dModel must be divisible by heads"]];

t3HeadDim=Quotient[t3DModel,t3Heads];

t3OutputDirectory=FileNameJoin[{Directory[],"S124_T3_Output"}];
If[!DirectoryQ[t3OutputDirectory],CreateDirectory[t3OutputDirectory]];

R3MFiniteQ[x_]:=NumericQ[x]&&FreeQ[x,Indeterminate|ComplexInfinity|_DirectedInfinity]
R3MUE[a_,b_]:=UndirectedEdge@@Sort[{a,b}];
R3MSeqKey[s_List]:=ToString[s,InputForm];
R3MCat[p_List]:=RandomChoice[N[p]->Range[Length[p]]];

R3MInitial[]:=<|
"Active"->Union[R3MUE[#,r3mHub]&/@r3mAnchors],
"Frozen"->{},
"Next"->100
|>;

R3MNeighbors[active_List,v_]:=
DeleteDuplicates@Join[
Cases[active,UndirectedEdge[a_,b_]/;a===v:>b],
Cases[active,UndirectedEdge[a_,b_]/;b===v:>a]
];

R3MCandidates[active_List,required_List]:=
Module[{verts,ns,cands={},x,y,z,e1,e2},
verts=DeleteDuplicates[Flatten[List@@@active]];
Do[
y=verts[[k]];
ns=Sort[R3MNeighbors[active,y]];
Do[
x=ns[[i]];
z=ns[[j]];
e1=R3MUE[x,y];
e2=R3MUE[y,z];
If[Intersection[{e1,e2},required]!={},AppendTo[cands,{x,y,z,e1,e2}]],
{i,1,Max[0,Length[ns]-1]},
{j,i+1,Length[ns]}
],
{k,1,Length[verts]}
];

```

```

];
cands
];

R3MInject[state_Association,event_Integer,seed_Integer,step_Integer]:=
Module[{active,frozen,next,anchor,fresh,w,inj,cands,c,x,y,z,e1,e2,newActive,newFroz
active=state["Active"];
frozen=state["Frozen"];
next=state["Next"];
anchor=r3mAnchors[[event]];
fresh=next;
w=next+1;
inj={R3MUE[anchor,fresh],R3MUE[fresh,r3mHub]};
active=Union[Join[active,inj]];
cands=R3MCandidates[active,inj];
If[
cands=={},
Return[
<|
"Active"->DeleteCases[active,e_/_;MemberQ[inj,e]],
"Frozen"->frozen,
"Next"->next+2
|>
]
];
c=BlockRandom[
SeedRandom[seed+100003*step+7919*event];
RandomChoice[cands]
];
{x,y,z,e1,e2}=c;
newActive=DeleteCases[active,e_/_;MemberQ[{e1,e2},e]];
newActive=Union[
Join[
newActive,
{R3MUE[x,z],R3MUE[x,w],R3MUE[w,z]}
]
];
newActive=DeleteCases[newActive,e_/_;MemberQ[inj,e]];
newFrozen=Union[
Join[
frozen,
{DirectedEdge[x,y],DirectedEdge[y,z]}
]
];
<|
"Active"->newActive,
"Frozen"->newFrozen,
"Next"->next+2
|>
];

R3MStateKey[state_Association]:=
Module[{active,frozen,verts,col,new,na,no,ni,others},
active=state["Active"];
frozen=state["Frozen"];
verts=Union[

```

```

r3mAnchors,
{r3mHub},
Flatten[List@@@active],
If[frozen=={},{},Flatten[List@@@frozen]]
];
col=Association@Table[
v->Which[
v===r3mAnchors[[1]], "A1",
v===r3mAnchors[[2]], "A2",
v===r3mAnchors[[3]], "A3",
v===r3mHub, "H",
True, "X"
],
{v,verts}
];
Do[
new=Association@Table[
na=Sort@Join[
Cases[active,UndirectedEdge[a_,b_]/;a===v:>col[b]],
Cases[active,UndirectedEdge[a_,b_]/;b===v:>col[a]]
];
no=Sort[Cases[frozen,DirectedEdge[a_,b_]/;a===v:>col[b]]];
ni=Sort[Cases[frozen,DirectedEdge[a_,b_]/;b===v:>col[a]]];
v->Hash[{col[v],na,no,ni},"SHA256","HexString"],
{v,verts}
];
col=new,
{r3mWLRounds}
];
others=Sort[
Lookup[
col,
Complement[verts,Join[r3mAnchors,{r3mHub}]]
]
];
Hash[
{
Lookup[col,r3mAnchors],
col[r3mHub],
others,
Length[active],
Length[frozen]
},
"SHA256",
"HexString"
]
];

R3MBuildCore[coreSeed_Integer]:=
Module[{rows,seq,state,seqKeys,keys},
rows={<|"Seq"->{},"State"->R3MInitial[]|>};
Do[
rows=Flatten[
Table[
Table[
seq=Append[row["Seq"],ev];

```

```

state=R3MInject[row["State"],ev,coreSeed,d];
<| "Seq"->seq, "State"->state|>,
{ev,1,3}
],
{row,rows}
],
1
];
Print["CoreDepth=",d," Histories=",Length[rows]],
{d,1,r3mDepth}
];
seqKeys=R3MSeqKey/@Lookup[rows,"Seq"];
keys=R3MStateKey/@Lookup[rows,"State"];
<|
"SeqToState"->AssociationThread[seqKeys,keys],
"SeqKeys"->seqKeys,
"KeyList"->keys,
"States"->DeleteDuplicates[keys],
"UniqueStates"->Length[DeleteDuplicates[keys]]
|>
];

r3oSeqs=Join[#{1,2}&/@DeleteDuplicates[Permutations[{1,2,3,3}]]];
r3oSeqKeys=R3MSeqKey/@r3oSeqs;

If[Length[r3oSeqs]!=12,T3Stop["expected exactly 12 S3Q histories"]];

If[
!And@@(
(
Sort[#]==Sort[{1,1,2,2,3,3}]&&
Take[#,-2]=={1,2}
)&/@r3oSeqs
),
T3Stop["S3Q order invariant failed"]
];

Print[""];
Print["====="];
Print["S3Q HISTORY PRECHECK"];
Print["====="];
Print["HistoryCount=",Length[r3oSeqs]];
Print["AllEventBags={2,2,2}=True"];
Print["AllRecent2={1,2}=True"];

r3mFullAll=R3MBuildCore[t3CoreSeed];

R3ORestrictData[data_Association]:=
Module[{vals},
vals=data["SeqToState"][#]&/@r3oSeqKeys;
<|
"SeqToState"->AssociationThread[r3oSeqKeys,vals],
"States"->DeleteDuplicates[vals],
"UniqueStates"->Length[DeleteDuplicates[vals]]
|>
];

```

```

t3FullData=R30RestrictData[r3mFullAll];

Print["RestrictedFullStructuralStates=",t3FullData["UniqueStates"]];

If[t3FullData["UniqueStates"]!=12,T3Stop["expected 12 Full TCCT structural states"]

t3BagStates=
Length[
DeleteDuplicates[
Hash[
{"BAG",Count[#,1],Count[#,2],Count[#,3]},
"SHA256",
"HexString"
]&/@r3oSeqs
]
];

t3RecentStates=
Length[
DeleteDuplicates[
Hash[
{"REC2",Take[#,-2]},
"SHA256",
"HexString"
]&/@r3oSeqs
]
];

If[t3BagStates!=1,T3Stop["EventBag shortcut precheck failed"]];
If[t3RecentStates!=1,T3Stop["Recent2 shortcut precheck failed"]];

Print["EventBagStates=",t3BagStates];
Print["Recent2States=",t3RecentStates];
Print["TCCTCoreChanged=False"];
Print["TCCT CORE PRECHECK=PASS"];

R30P[k_Integer]:=
0.08*ConstantArray[1.,4]+0.68*UnitVector[4,k];

R30World[seed_Integer]:=
BlockRandom[
SeedRandom[seed];
Module[{perm,regs,seqReg,off1,off2,p1,p2},
perm=RandomSample[r3oSeqKeys];
regs=Flatten[Table[ConstantArray[r,4],{r,1,3}]];
seqReg=AssociationThread[perm,regs];
off1=RandomInteger[{0,3}];
off2=RandomInteger[{0,3}];
p1=Table[
R30P[1+Mod[r+o+2*a+off1-2,4]],
{r,1,3},{o,1,4},{a,0,1}
];
p2=Table[
R30P[1+Mod[2*r+o+a6+2*a7+off2-2,4]],
{r,1,3},{o,1,4},{a6,0,1},{a7,0,1}
];

```

```

];
<| "SeqRegime" -> seqReg, "P1" -> p1, "P2" -> p2 |>
]
];

R3OSim[seed_Integer, world_Association] :=
BlockRandom[
SeedRandom[seed];
Module[{seq, key, reg, o, a6, a7, y1, y2},
seq = RandomChoice[r3oSeqs];
key = R3MSeqKey[seq];
reg = world["SeqRegime"][key];
o = RandomInteger[{1, 4}];
a6 = RandomInteger[{0, 1}];
a7 = RandomInteger[{0, 1}];
y1 = R3MCat[world["P1"]][reg, o, a6 + 1];
y2 = R3MCat[world["P2"]][reg, o, a6 + 1, a7 + 1];
<|
"SeqKey" -> key,
"O6" -> o,
"A6" -> a6,
"A7" -> a7,
"Y1" -> y1,
"Y2" -> y2
|>
]
];

R3MProbe1[o_Integer, a6_Integer] := 1 + 2*(o - 1) + a6;
R3MProbe2[o_Integer, a6_Integer, a7_Integer] := 9 + 4*(o - 1) + 2*a6 + a7;

R3MCountsRows[rows_List] :=
Module[{m = ConstantArray[0, {24, 4}], p1, p2},
Do[
p1 = R3MProbe1[row["O6"], row["A6"]];
p2 = R3MProbe2[row["O6"], row["A6"], row["A7"]];
m[[p1, row["Y1"]]]++;
m[[p2, row["Y2"]]]++;
{row, rows}
];
m
];

R3MStateCountsData[rows_List, data_Association] :=
Association@KeyValueMap[
Function[{k, rs}, k -> R3MCountsRows[rs]],
GroupBy[
rows,
data["SeqToState"][#["SeqKey"]]&
]
];

R3MGet[c_Association, s_] := Lookup[c, s, r3mZero];

R3MPooled[c_Association, states_List] :=
Total[R3MGet[c, #] &/@ states];

```

```

R3MObs[c_Association,states_List]:=
Total[Flatten[R3MPooled[c,states]]];

R3MAddCounts[a_Association,b_Association,states_List]:=
Association@Table[
s->(R3MGet[a,s]+R3MGet[b,s]),
{s,states}
];

R3MPrepare[rawA_List,rawB_List,data_Association]:=
Module[{a,b,states},
states=data["States"];
a=R3MStateCountsData[rawA,data];
b=R3MStateCountsData[rawB,data];
<|
"States"->states,
"A"->a,
"B"->b,
"AB"->R3MAddCounts[a,b,states]
|>
];

R3MLabel[v_,support_Integer,effect_?NumericQ,topMin_?NumericQ]:=
Module[{n,p,ord,top,runner},
If[!VectorQ[v,NumericQ]||Length[v]!=4,T3Stop["R3MLabel shape error"]];
n=Total[v];
If[n<support,Return[0]];
p=N[v/n];
ord=Reverse[Ordering[p]];
top=ord[[1]];
runner=ord[[2]];
If[
p[[top]]>=topMin&&
p[[top]]-p[[runner]]>=effect,
top,
0
]
];

R3MMakeGroups[
states_List,
cA_Association,
cB_Association,
probe_Integer,
support_Integer,
effect_?NumericQ,
topMin_?NumericQ
]:=
Module[{lab,ga,keys,pos,groups},
lab=Association@Table[
s->R3MLabel[R3MGet[cA,s][[probe]],support,effect,topMin],
{s,states}
];
ga=KeySort@GroupBy[states,lab[#]&];
keys=Keys[ga];

```



```

pos=Select[keys,#>0&];
If[Length[pos]<2,Return[$Failed]];
If[Min[Length/@Lookup[ga,pos]]<r3mMinChildStates,Return[$Failed]];
groups=Values[ga];
If[Min[R3MObs[cA,#]&/@groups]<r3mMinGroupObs,Return[$Failed]];
If[Min[R3MObs[cB,#]&/@groups]<r3mMinGroupObs,Return[$Failed]];
<|"Groups"->groups,"GroupLabels"->keys|>
];

R3MLossBase[train_,test_]:=
Module[{loss=0.,n=0,prob,den},
Do[
den=Total[train[[p]]]+4.;
prob=(N[train[[p]]+1.)/den;
loss-=N[test[[p]].Log[prob]];
n+=Total[test[[p]]],
{p,1,24}
];
{loss,n}
];

R3MLossShrink[train_,parent_,test_]:=
Module[{loss=0.,n=0,parentP,prob,den},
Do[
parentP=(N[parent[[p]]+1.)/(Total[parent[[p]]]+4.);
den=Total[train[[p]]+r3mKappa;
prob=(N[train[[p]]+r3mKappa*parentP)/den;
loss-=N[test[[p]].Log[prob]];
n+=Total[test[[p]]],
{p,1,24}
];
{loss,n}
];

R3MSplitEval[
states_List,
groups_List,
cA_Association,
cB_Association
]:=
Module[{pa,pb,parentLoss,splitLoss=0.,n=0,ca,cb,z,gain},
pa=R3MPooled[cA,states];
pb=R3MPooled[cB,states];
parentLoss=R3MLossBase[pa,pb];
Do[
ca=R3MPooled[cA,g];
cb=R3MPooled[cB,g];
z=R3MLossShrink[ca,pa,cb];
splitLoss+=z[[1]];
n+=z[[2]],
{g,groups}
];
If[n<=0||n!=parentLoss[[2]],Return[$Failed]];
gain=N[parentLoss[[1]]/n-splitLoss/n];
If[!R3MFiniteQ[gain],T3Stop["nonfinite split gain"]];
<|"ValGain"->gain|>

```

```

];

R3MBestSplit[
states_List,
cA_Association,
cB_Association
]:=
Module[{best=$Failed,bestScore=-Infinity,g,ev,score},
Do[
g=R3MMakeGroups[
states,
cA,
cB,
probe,
support,
effect,
topMin
];
If[
UnsameQ[g,$Failed],
ev=R3MSplitEval[states,g["Groups"],cA,cB];
If[
UnsameQ[ev,$Failed],
score=N[
ev["ValGain"]-
r3mPenalty*(Length[g["Groups"]]-1)
];
If[
score>bestScore,
bestScore=score;
best=Join[
<|
"Probe"->probe,
"Support"->support,
"Effect"->effect,
"TopMin"->topMin
|>,
g,
ev,
<| "Score"->score|>
]
]
],
{probe,1,24},
{support,r3mSupports},
{effect,r3mEffects},
{topMin,r3mTopMins}
];
best
];

R3MDiscover[
states_List,
cA_Association,
cB_Association

```

```

]:=
Module[{blocks={states},log={},step=0,cands,best,bi},
While[
step<r3mMaxSplits&&Length[blocks]<r3mMaxBlocks,
cands=Cases[
Table[
If[
Length[blocks[[i]]]<2*r3mMinChildStates,
Nothing,
With[
{x=R3MBestSplit[blocks[[i]],cA,cB]},
If[
UnsameQ[x,$Failed],
Join[
<|
"BlockIndex"->i,
"ParentStates"->Length[blocks[[i]]]
|>,
x
],
Nothing
]
],
{i,1,Length[blocks]}
],
_Association
];
If[cands=={},Break[]];
best=First[MaximalBy[cands,#["Score"]&]];
If[
best["ValGain"]<r3mMinValGain||
best["Score"]<=0.,
Break[]
];
bi=best["BlockIndex"];
step++;
Print[
"ACCEPT SPLIT=",step,
" STATES=",best["ParentStates"],
" PROBE=",best["Probe"],
" GAIN=",N[best["ValGain"]]
];
AppendTo[log,KeyDrop[best,{"Groups"}]];
blocks=Join[
Take[blocks,bi-1],
best["Groups"],
Drop[blocks,bi]
];
];
<|"Blocks"->blocks,"Log"->log,"Splits"->step|>
];

R3MBlockMap[blocks_List]:=
Association@Flatten[
MapIndexed[

```

```

Thread[#1->First[#2]]&,
blocks
]
];

R3MProspective[
blocks_List,
cTrain_Association,
cTest_Association,
states_List
]:=
Module[{map,blockTrain,globalTrain,baseLoss=0.,exactLoss=0.,qLoss=0.,n=0,test,z,bid},
map=R3MBlockMap[blocks];
globalTrain=R3MPooled[cTrain,states];
blockTrain=AssociationThread[
Range[Length[blocks]],
R3MPooled[cTrain,#]&/@blocks
];
Do[
test=R3MGet[cTest,s];
z=R3MLossBase[globalTrain,test];
baseLoss+=z[[1]];
z=R3MLossShrink[R3MGet[cTrain,s],globalTrain,test];
exactLoss+=z[[1]];
bid=map[s];
z=R3MLossShrink[blockTrain[bid],globalTrain,test];
qLoss+=z[[1]];
n+=Total[Flatten[test]],
{s,states}
];
If[n<=0,T3Stop["no prospective observations"]];
<|
"BaseNLL"->N[baseLoss/n],
"ExactStateNLL"->N[exactLoss/n],
"QuotientNLL"->N[qLoss/n],
"ExactGainVsBase"->N[(baseLoss-exactLoss)/n],
"QuotientGainVsBase"->N[(baseLoss-qLoss)/n],
"QuotientGainVsExact"->N[(exactLoss-qLoss)/n]
|>
];

R3MEvaluate[
model_Association,
rawC_List,
data_Association
]:=
Module[{cC,p},
cC=R3MStateCountsData[rawC,data];
p=R3MProspective[
model["Blocks"],
model["TrainCounts"],
cC,
model["States"]
];
Join[model,p]
];

```

```

Print[""];
Print["====="];
Print["PHASE 1: DISCOVERY A + VALIDATION B"];
Print["PROSPECTIVE C DOES NOT EXIST"];
Print["====="];

t3World=R30World[t3WorldSeed];

t3RawA=Table[
R30Sim[t3WorldSeed+71000000+i,t3World],
{i,1,t3BankAB}
];

t3RawB=Table[
R30Sim[t3WorldSeed+72000000+i,t3World],
{i,1,t3BankAB}
];

t3Prep=R3MPrepare[t3RawA,t3RawB,t3FullData];

Print["StructuralStates=",Length[t3Prep["States"]]];

If[
Length[t3Prep["States"]]!=12,
T3Stop["structural state count changed before quotient discovery"]
];

Print["Discovering predictive quotient..."];

t3Disc=R3MDiscover[
t3Prep["States"],
t3Prep["A"],
t3Prep["B"]
];

t3Blocks=t3Disc["Blocks"];
t3QuotientStates=Length[t3Blocks];

Print["QuotientStates=",t3QuotientStates];
Print["Splits=",t3Disc["Splits"]];

If[
t3QuotientStates!=3,
T3Stop["expected exactly 3 predictive quotient states"]
];

t3TCCTFreezeHash=Hash[
ToString[
{
"S124-T3",
t3WorldSeed,
t3CoreSeed,
t3Blocks,
t3Disc["Log"]
},

```

```

InputForm
],
"SHA256",
"HexString"
];

t3TCCTModel=<|
"Condition"->"Full",
"WorldSeed"->t3WorldSeed,
"CoreSeed"->t3CoreSeed,
"States"->t3Prep["States"],
"Blocks"->t3Blocks,
"TrainCounts"->t3Prep["AB"],
"StructuralStates"->Length[t3Prep["States"]],
"QuotientStates"->t3QuotientStates,
"Splits"->t3Disc["Splits"],
"FreezeHash"->t3TCCTFreezeHash
|>;

Print[""];
Print["TCCT FROZEN"];
Print["StructuralStates=", t3TCCTModel["StructuralStates"]];
Print["QuotientStates=", t3TCCTModel["QuotientStates"]];
Print["FreezeHash=", t3TCCTFreezeHash];

Put[
t3TCCTModel,
FileNameJoin[
{t3OutputDirectory, "S124_T3_TCCT_FROZEN_BEFORE_C.wl"}
]
];

BlockRandom[
SeedRandom[t3PrototypeSeed];
t3SensorBases=
Normalize/@RandomVariate[
NormalDistribution[0., 1.],
{3, t3SensorDim}
]
];

T3MicroPos[pos_Integer]:=
Flatten[
Table[
{
Sin[pos/(10000.^(2.0*k/t3PosDim))],
Cos[pos/(10000.^(2.0*k/t3PosDim))]}
],
{k, 0, t3PosDim/2-1}
]
];

T3EventSensory[event_Integer, seed_Integer]:=
BlockRandom[
SeedRandom[seed];
Module[{drift, gain},

```

```

drift=RandomReal[
{-t3SensorDrift,t3SensorDrift},
t3SensorDim
];
N[
Table[
gain=1.+RandomReal[
{-t3SensorGainJitter,t3SensorGainJitter}
];
Join[
t3SensorScale*
gain*
t3SensorBases[[event]]+
drift+
RandomVariate[
NormalDistribution[0.,t3SensorNoise],
t3SensorDim
],
T3MicroPos[m]
],
{m,1,t3MicroLen}
]
]
];

```

```

t3InputDim=t3SensorDim+t3PosDim;

```

```

T3MakePerceptionRaw[
nPerClass_Integer,
seed_Integer
]:=
BlockRandom[
SeedRandom[seed];
RandomSample[
Flatten[
Table[
Table[
<|
"Input"->T3EventSensory[
event,
seed+100000*event+i
],
"Label"->event
|>,
{i,1,nPerClass}
],
{event,1,3}
],
1
]
]
];

```

```

T3ToRules[data_List]:=
(#["Input"]->#["Label"]&)/@data;

```

```

T3Block[
modelDim_Integer,
heads_Integer,
ffDim_Integer,
drop_?NumericQ
]:=
Module[{hd},
hd=Quotient[modelDim,heads];
NetGraph[
<|
"LN1"->NormalizationLayer[2,"Same"],
"Q"->NetMapOperator[LinearLayer[{heads,hd}]],
"K"->NetMapOperator[LinearLayer[{heads,hd}]],
"V"->NetMapOperator[LinearLayer[{heads,hd}]],
"Attention"->AttentionLayer[
"Dot",
"MultiHead"->True,
"Mask"->"Causal",
"ScoreRescaling"->"DimensionSqrt",
"Dropout"->drop
],
"Merge"->NetMapOperator[
NetChain[
{
FlattenLayer[],
LinearLayer[modelDim]
}
]
],
"ADrop"->DropoutLayer[drop],
"Res1"->ThreadingLayer[Plus],
"LN2"->NormalizationLayer[2,"Same"],
"FF"->NetMapOperator[
NetChain[
{
LinearLayer[ffDim],
ElementwiseLayer[Ramp],
DropoutLayer[drop],
LinearLayer[modelDim]
}
]
],
"FDrop"->DropoutLayer[drop],
"Res2"->ThreadingLayer[Plus]
|>,
{
NetPort["Input"]->"LN1",
"LN1"->"Q",
"LN1"->"K",
"LN1"->"V",
"Q"->NetPort["Attention","Query"],
"K"->NetPort["Attention","Key"],
"V"->NetPort["Attention","Value"],
"Attention"->"Merge",
"Merge"->"ADrop",

```



```

{NetPort["Input"], "ADrop"}->"Res1",
"Res1"->"LN2",
"LN2"->"FF",
"FF"->"FDrop",
{"Res1", "FDrop"}->"Res2"
},
"Input"->{"Varying", modelDim}
]
];

T3Transformer[]:=
Module[{blocks},
blocks=Table[
T3Block[
t3DModel,
t3Heads,
t3FF,
t3Dropout
],
{t3Layers}
];
NetChain[
Join[
{
NetMapOperator[
LinearLayer[t3DModel]
]
},
blocks,
{
NormalizationLayer[2, "Same"],
SequenceLastLayer[],
LinearLayer[3],
SoftmaxLayer[]
}
],
"Input"->{"Varying", t3InputDim},
"Output"->NetDecoder[
{"Class", {1, 2, 3}}
]
]
];

Print[""];
Print["====="];
Print["PHASE 2: TRAIN STREAMING EVENT PERCEPTION"];
Print["TRANSFORMER OUTPUT CLASSES={1,2,3}"];
Print["TRANSFORMER NEVER SEES HISTORY LABEL"];
Print["TCCT ALREADY FROZEN"];
Print["PROSPECTIVE C STILL DOES NOT EXIST"];
Print["====="];

t3PerceptionTrain=
T3MakePerceptionRaw[
t3PerceptionTrainPerClass,
t3PerceptionTrainSeed

```

```

];

t3PerceptionVal=
T3MakePerceptionRaw[
t3PerceptionValPerClass,
t3PerceptionValSeed
];

t3TrainRules=T3ToRules[t3PerceptionTrain];
t3ValRules=T3ToRules[t3PerceptionVal];

SeedRandom[t3PerceptionNetSeed];

t3Net0=NetInitialize[
T3Transformer[]
];

Print["TransformerInitialized=True"];
Print["OutputClasses={1,2,3}"];
Print["MicroSequenceLength=",t3MicroLen];
Print["Layers=",t3Layers];
Print["dModel=",t3DModel];
Print["Heads=",t3Heads];
Print["InputDim=",t3InputDim];

t3TrainingResult=
NetTrain[
t3Net0,
t3TrainRules,
All,
ValidationSet->t3ValRules,
MaxTrainingRounds->t3MaxRounds,
BatchSize->t3BatchSize,
LearningRate->t3LearningRate,
Method->"ADAM",
TrainingProgressMeasurements->{
"Accuracy",
"ErrorRate"
},
TrainingStoppingCriterion-><|
"Criterion"->"Loss",
"Patience"->t3Patience
|>,
TrainingProgressReporting->"Print",
TargetDevice->t3TargetDevice,
RandomSeeding->t3PerceptionNetSeed
];

t3FrozenPerception=
t3TrainingResult["TrainedNet"];

t3PerceptionValAccuracy=
NetMeasurements[
t3FrozenPerception,
t3ValRules,
"Accuracy",

```

```

BatchSize->t3BatchSize
];

Print[""];
Print["STREAMING PERCEPTION FROZEN"];
Print["ValidationEventAccuracy=",t3PerceptionValAccuracy];

t3PerceptionModelFile=
FileNameJoin[
{
t3OutputDirectory,
"S124_T3_STREAMING_TRANSFORMER_FROZEN_BEFORE_C.wlnet"
}
];

Export[
t3PerceptionModelFile,
t3FrozenPerception
];

t3GlobalFreezeHash=
Hash[
ToString[
{
t3TCCTFreezeHash,
t3PerceptionNetSeed,
t3PerceptionValAccuracy,
t3SensorBases
},
InputForm
],
"SHA256",
"HexString"
];

Print[""];
Print["====="];
Print["GLOBAL FREEZE COMPLETE"];
Print["TCCTCoreChanged=False"];
Print["TCCTQFrozen=True"];
Print["StreamingTransformerFrozen=True"];
Print["TransformerHistoryClasses=False"];
Print["ProspectiveCExists=False"];
Print["GlobalFreezeHash=",t3GlobalFreezeHash];
Print["====="];

Clear[
t3RawA,
t3RawB,
t3PerceptionTrain,
t3PerceptionVal,
t3TrainRules,
t3ValRules,
t3Net0,
t3TrainingResult
];

```

```
T3SeqFromKey[key_String]:=ToExpression[key];
```

```
T3PredictBatched[  
net_,  
inputs_List,  
batch_Integer  
]:=  
Module[{out={},i,last,z},  
Do[  
last=Min[i+batch-1,Length[inputs]];  
z=net[Take[inputs,{i,last}]];  
If[!ListQ[z],z={z}];  
out=Join[out,z];  
Print["Perception=",last,"/",Length[inputs]],  
{i,1,Length[inputs],batch}  
];  
out  
];
```

```
T3BaseProb[parent_,probe_Integer]:=  
(N[parent[[probe]]+1.)/  
(Total[parent[[probe]]+4.));
```

```
T3BlockProb[  
local_,  
parent_,  
probe_Integer  
]:=  
Module[{parentP,den},  
parentP=T3BaseProb[parent,probe];  
den=Total[local[[probe]]+r3mKappa;  
(N[local[[probe]]+r3mKappa*parentP)/den  
];
```

```
T3HybridProspective[  
model_Association,  
rawC_List,  
predKeys_List,  
data_Association  
]:=  
Module[  
{  
blocks,  
states,  
cTrain,  
blockMap,  
blockTrain,  
globalTrain,  
loss=0.,  
n=0,  
valid=0,  
row,  
key,  
state,  
bid,
```

```

p1,
p2,
prob
},
blocks=model["Blocks"];
states=model["States"];
cTrain=model["TrainCounts"];
blockMap=R3MBlockMap[blocks];
globalTrain=R3MPooled[cTrain,states];
blockTrain=AssociationThread[
Range[Length[blocks]],
R3MPooled[cTrain,#]&/@blocks
];
Do[
row=rawC[[i]];
key=predKeys[[i]];
p1=R3MProbe1[row["06"],row["A6"]];
p2=R3MProbe2[row["06"],row["A6"],row["A7"]];
If[
KeyExistsQ[data["SeqToState"],key],
state=data["SeqToState"][key];
If[
KeyExistsQ[blockMap,state],
bid=blockMap[state];
valid++;
prob=T3BlockProb[
blockTrain[bid],
globalTrain,
p1
],
prob=T3BaseProb[
globalTrain,
p1
]
],
prob=T3BaseProb[
globalTrain,
p1
]
];
loss-=Log[
Max[10.^-15,prob[[row["Y1"]]]]
];
If[
KeyExistsQ[data["SeqToState"],key],
state=data["SeqToState"][key];
If[
KeyExistsQ[blockMap,state],
bid=blockMap[state];
prob=T3BlockProb[
blockTrain[bid],
globalTrain,
p2
],
prob=T3BaseProb[
globalTrain,

```

```

p2
]
],
prob=T3BaseProb[
globalTrain,
p2
]
];
loss-=Log[
Max[10.^-15,prob[[row["Y2"]]]]
];
n+=2,
{i,1,Length[rawC]}
];
<|
"QuotientNLL"->N[loss/n],
"ValidHistoryRate"->N[valid/Length[rawC]]
|>
];

Print[""];
Print["====="];
Print["PHASE 3: PROSPECTIVE C-FIRST OPENING"];
Print["NO MODEL CHANGES AFTER THIS LINE"];
Print["====="];

t3RawC=
Table[
R3OSim[
t3WorldSeed+73000000+i,
t3World
],
{i,1,t3BankC}
];

Print["ProspectiveHistories=",Length[t3RawC]];
Print["ProspectiveEvents=",6*Length[t3RawC]];

t3OracleResult=
R3MEvaluate[
t3TCCTModel,
t3RawC,
t3FullData
];

t3TrueKeys=Lookup[t3RawC,"SeqKey"];
t3TrueSeqs=T3SeqFromKey/@t3TrueKeys;

If[
!And@@((Length[#]==6&)/@t3TrueSeqs),
T3Stop["Prospective history length is not 6"]
];

t3FlatTrueEvents=Flatten[t3TrueSeqs];

Print["Generating independent sensory stream..."];

```

```

t3FlatInputs=
Flatten[
Table[
Table[
T3EventSensory[
t3TrueSeqs[[i,j]],
t3SensorCSeed+10000*i+j
],
{j,1,6}
],
{i,1,Length[t3TrueSeqs]}
],
1
];

If[
Length[t3FlatInputs]!=Length[t3FlatTrueEvents],
T3Stop["streaming sensory/event count mismatch"]
];

Print["Running streaming perception..."];

t3FlatPredEvents=
T3PredictBatched[
t3FrozenPerception,
t3FlatInputs,
256
];

If[
Length[t3FlatPredEvents]!=Length[t3FlatTrueEvents],
T3Stop["streaming prediction count mismatch"]
];

t3EventAccuracy=
N[
Mean[
MapThread[
Boole[SameQ[#1,#2]]&,
{
t3FlatPredEvents,
t3FlatTrueEvents
}
]
]
];

t3PredSeqs=
Partition[
t3FlatPredEvents,
6
];

t3ExactHistoryAccuracy=
N[

```

```

Mean[
  MapThread[
    Boole[SameQ[#1,#2]]&,
    {
      t3PredSeqs,
      t3TrueSeqs
    }
  ]
];

t3PredKeys=
R3MSeqKey/@t3PredSeqs;

t3AllowedHistoryRate=
N[
  Mean[
    Boole[
      KeyExistsQ[
        t3FullData["SeqToState"],
        #
      ]
    ]&/@t3PredKeys
  ]
];

t3StateMap=t3FullData["SeqToState"];
t3BlockMap=R3MBlockMap[t3Blocks];

T3QForKey[key_String]:=
Module[{state},
  If[
    !KeyExistsQ[t3StateMap,key],
    Return[Missing["UnknownHistory"]]
  ];
  state=t3StateMap[key];
  If[
    !KeyExistsQ[t3BlockMap,state],
    Return[Missing["UnknownState"]]
  ];
  t3BlockMap[state]
];

t3TrueQ=T3QForKey/@t3TrueKeys;
t3PredQ=T3QForKey/@t3PredKeys;

t3QPreservation=
N[
  Mean[
    MapThread[
      Boole[SameQ[#1,#2]]&,
      {
        t3PredQ,
        t3TrueQ
      }
    ]
  ]
];

```



```

]
];

Print[""];
Print["StreamingEventAccuracy=", t3EventAccuracy];
Print["ExactHistoryRecovery=", t3ExactHistoryAccuracy];
Print["AllowedHistoryRate=", t3AllowedHistoryRate];
Print["QStatePreservation=", t3QPreservation];

t3HybridResult=
T3HybridProspective[
t3TCCTModel,
t3RawC,
t3PredKeys,
t3FullData
];

BlockRandom[
SeedRandom[t3RandomControlSeed];
t3RandomPredSeqs=
Table[
RandomInteger[{1, 3}, 6],
{Length[t3RawC]}
]
];

t3RandomPredKeys=
R3MSeqKey/@t3RandomPredSeqs;

t3RandomResult=
T3HybridProspective[
t3TCCTModel,
t3RawC,
t3RandomPredKeys,
t3FullData
];

t3OracleBaseNLL=
t3OracleResult["BaseNLL"];

t3OracleQNLL=
t3OracleResult["QuotientNLL"];

t3HybridQNLL=
t3HybridResult["QuotientNLL"];

t3RandomQNLL=
t3RandomResult["QuotientNLL"];

t3OracleGain=
t3OracleBaseNLL-
t3OracleQNLL;

t3HybridGain=
t3OracleBaseNLL-
t3HybridQNLL;

```

```

t3RandomGain=
t3OracleBaseNLL-
t3RandomQNLL;

t3GainRetention=
If[
t3OracleGain>0.,
N[t3HybridGain/t3OracleGain],
Indeterminate
];

t3HybridPenalty=
t3HybridQNLL-
t3OracleQNLL;

t3HybridBeatsRandom=
t3HybridQNLL<t3RandomQNLL;

t3StrictPass=
And[
t3EventAccuracy>=t3EventAccuracyGate,
t3ExactHistoryAccuracy>=t3ExactHistoryGate,
t3AllowedHistoryRate>=t3ValidHistoryGate,
t3QPreservation>=t3QPreservationGate,
t3HybridGain>0.,
NumericQ[t3GainRetention],
t3GainRetention>=t3GainRetentionGate,
TrueQ[t3HybridBeatsRandom]
];

t3Diagnosis=
Which[
t3StrictPass,
"STREAMING_TRANSFORMER_PERCEPTION_SUCCESSFULLY_DRIVES_TCCT_REASONING_CORE",
t3EventAccuracy<t3EventAccuracyGate,
"EVENT_PERCEPTION_ACCURACY_TOO_LOW",
t3ExactHistoryAccuracy<t3ExactHistoryGate,
"EVENT_ERRORS_ACCUMULATE_INTO_TOO_MANY_HISTORY_ERRORS",
t3AllowedHistoryRate<t3ValidHistoryGate,
"TOO_MANY_STREAMING_PREDICTIONS_LEAVE_ALLOWED_HISTORY_SET",
t3QPreservation<t3QPreservationGate,
"STREAMING_ERRORS_CHANGE_TOO_MANY_TCCT_Q_STATES",
t3HybridGain<=0.,
"STREAMING_HYBRID_LOSES_TCCT_PREDICTIVE_VALUE",
NumericQ[t3GainRetention]&& t3GainRetention<t3GainRetentionGate,
"STREAMING_HYBRID_RETAINS_TOO_LITTLE_ORACLE_GAIN",
!TrueQ[t3HybridBeatsRandom],
"STREAMING_HYBRID_NOT_BETTER_THAN_RANDOM_PERCEPTION",
True,
"PARTIAL_STREAMING_INTERFACE_RESULT"
];

Print[""];
Print["====="];
Print["S124-T3 FINAL SUMMARY"];

```

```

Print[ "====="];
Print[ "TCCTCoreChanged=False"];
Print[ "TransformerHistoryClasses=False"];
Print[ "TransformerOutputClasses={1,2,3}"];
Print[ "StreamingPerception=True"];
Print[ "S3QHistories=", Length[r3oSeqs]];
Print[ "EventBagStates=", t3BagStates];
Print[ "Recent2States=", t3RecentStates];
Print[ "StructuralStates=", t3TCCTModel[ "StructuralStates"]];
Print[ "QuotientStates=", t3TCCTModel[ "QuotientStates"]];
Print[ "TCCTSplits=", t3TCCTModel[ "Splits"]];
Print[ "PerceptionValidationEventAccuracy=", t3PerceptionValAccuracy];
Print[ "ProspectiveEventAccuracy=", t3EventAccuracy];
Print[ "ExactHistoryRecovery=", t3ExactHistoryAccuracy];
Print[ "AllowedHistoryRate=", t3AllowedHistoryRate];
Print[ "QStatePreservation=", t3QPreservation];
Print[ "OracleBaseNLL=", t3OracleBaseNLL];
Print[ "OracleTCCT_QNLL=", t3OracleQNLL];
Print[ "StreamingHybrid_QNLL=", t3HybridQNLL];
Print[ "RandomStreaming_QNLL=", t3RandomQNLL];
Print[ "OracleTCCTGain=", t3OracleGain];
Print[ "StreamingHybridGain=", t3HybridGain];
Print[ "RandomControlGain=", t3RandomGain];
Print[ "HybridGainRetention=", t3GainRetention];
Print[ "HybridPenaltyVsOracle=", t3HybridPenalty];
Print[ "HybridValidHistoryRate=", t3HybridResult[ "ValidHistoryRate"]];
Print[ "RandomValidHistoryRate=", t3RandomResult[ "ValidHistoryRate"]];
Print[ "HybridBeatsRandom=", t3HybridBeatsRandom];
Print[ "STRICT STREAMING PASS=", t3StrictPass];
Print[ "DIAGNOSIS=", t3Diagnosis];
Print[ "====="];

```

```

t3Summary=<|
"Stage"->"S124-T3",
"Purpose"->"StreamingTransformerPerceptionToTCCTReasoningSeparation",
"TCCTCoreChanged"->False,
"TransformerHistoryClasses"->False,
"TransformerOutputClasses"->{1,2,3},
"StreamingPerception"->True,
"S3QHistories"->Length[r3oSeqs],
"EventBagStates"->t3BagStates,
"Recent2States"->t3RecentStates,
"WorldSeed"->t3WorldSeed,
"CoreSeed"->t3CoreSeed,
"StructuralStates"->t3TCCTModel[ "StructuralStates"],
"QuotientStates"->t3TCCTModel[ "QuotientStates"],
"TCCTSplits"->t3TCCTModel[ "Splits"],
"PerceptionValidationEventAccuracy"->t3PerceptionValAccuracy,
"ProspectiveEventAccuracy"->t3EventAccuracy,
"ExactHistoryRecovery"->t3ExactHistoryAccuracy,
"AllowedHistoryRate"->t3AllowedHistoryRate,
"QStatePreservation"->t3QPreservation,
"OracleBaseNLL"->t3OracleBaseNLL,
"OracleTCCTQNLL"->t3OracleQNLL,
"StreamingHybridQNLL"->t3HybridQNLL,
"RandomStreamingQNLL"->t3RandomQNLL,

```

```

"OracleGain"->t3OracleGain,
"StreamingHybridGain"->t3HybridGain,
"RandomGain"->t3RandomGain,
"HybridGainRetention"->t3GainRetention,
"HybridPenaltyVsOracle"->t3HybridPenalty,
"HybridValidHistoryRate"->t3HybridResult["ValidHistoryRate"],
"RandomValidHistoryRate"->t3RandomResult["ValidHistoryRate"],
"HybridBeatsRandom"->t3HybridBeatsRandom,
"StrictStreamingPass"->t3StrictPass,
"Diagnosis"->t3Diagnosis,
"TCCTFreezeHash"->t3TCCTFreezeHash,
"GlobalFreezeHash"->t3GlobalFreezeHash,
"ProspectiveGeneratedAfterBothFrozen"->True
|>;

t3SummaryFile=
FileNameJoin[
{
t3OutputDirectory,
"S124_T3_summary.wl"
}
];

Put[t3Summary,t3SummaryFile];

Print["SummaryFile=",t3SummaryFile];
Print[""];
Print["====="];
Print["S124-T3 COMPLETE"];
Print["====="];

```

=====

S124-T3 STANDALONE

STREAMING TRANSFORMER PERCEPTION -> TCCT REASONING

=====

WolframVersion=15.0.0 for Microsoft Windows (64-bit) (May 26, 2026)

Date=Wed 19 Aug 2026 21:44:02

=====

S3Q HISTORY PRECHECK

=====

HistoryCount=12

AllEventBags={2,2,2}=True

AllRecent2={1,2}=True

CoreDepth=1 Histories=3

CoreDepth=2 Histories=9

CoreDepth=3 Histories=27

CoreDepth=4 Histories=81

CoreDepth=5 Histories=243

CoreDepth=6 Histories=729

RestrictedFullStructuralStates=12

EventBagStates=1

Recent2States=1

TCCTCoreChanged=False

TCCT CORE PRECHECK=PASS

=====

PHASE 1: DISCOVERY A + VALIDATION B

PROSPECTIVE C DOES NOT EXIST

=====

StructuralStates=12

Discovering predictive quotient...

ACCEPT SPLIT=1 STATES=12 PROBE=1 GAIN=0.37538

QuotientStates=3

Splits=1

TCCT FROZEN

StructuralStates=12

QuotientStates=3

FreezeHash=edb1f87de6ce39b3d17ade237e5ebd1feda5d23bc427abe344ab797636daaca4

=====

PHASE 2: TRAIN STREAMING EVENT PERCEPTION

TRANSFORMER OUTPUT CLASSES={1,2,3}

TRANSFORMER NEVER SEES HISTORY LABEL

TCCT ALREADY FROZEN

PROSPECTIVE C STILL DOES NOT EXIST

=====

TransformerInitialized=True

OutputClasses={1,2,3}

MicroSequenceLength=4

Layers=2

dModel=64

Heads=4

InputDim=20

Starting training.

Optimization Method: ADAM

Beta1: 9.00×10^{-1}
 Beta2: 9.99×10^{-1}
 Epsilon: 1.00×10^{-5}
 Gradient Clipping: --
 L2 Regularization: --
 Learning Rate: 3.00×10^{-4}
 Learning Rate Schedule: --
 Weight Clipping: --

Device: CPU

Batch Size: 64

Batches Per Round: 57

Training Examples: 3600

%	round	batch	examples	inputs	learning	time	time	\
>	current	round	test	current	round	test	current\	
>	round	test						
	/20	/57	processed	/second	rate	elapsed	left	\
>	loss	loss	loss	acc.	acc.	acc.	error	\
>	error	error						
5	2	3	3840	1457	7.25×10^{-5}	2s	38s	\
>	1.37×10^{-1}	1.44×10^{-1}	2.01×10^{-3}	95.39%	95.15%	100%		\
>	4.61%	4.85%	0%					
10	3	5	7616	1602	1.01×10^{-4}	4s	35s	\
>	3.74×10^{-3}	3.88×10^{-3}	1.07×10^{-3}	100%	100%	100%		\
>	0%	0%	0%					
16	4	12	11712	1908	1.23×10^{-4}	6s	31s	\
>	2.32×10^{-3}	2.46×10^{-3}	6.98×10^{-4}	100%	100%	100%		\
>	0%	0%	0%					
22	5	28	16384	2258	1.43×10^{-4}	8s	28s	\
>	1.55×10^{-3}	1.72×10^{-3}	5.02×10^{-4}	100%	100%	100%		\
>	0%	0%	0%					
28	6	37	20608	2220	1.57×10^{-4}	10s	26s	\
>	1.11×10^{-3}	1.29×10^{-3}	3.87×10^{-4}	100%	100%	100%		\
>	0%	0%	0%					
34	7	47	24896	2268	1.70×10^{-4}	12s	23s	\
>	8.70×10^{-4}	1.01×10^{-3}	3.14×10^{-4}	100%	100%	100%		\
>	0%	0%	0%					
40	8	53	28928	2268	1.81×10^{-4}	14s	21s	\
>	7.15×10^{-4}	8.19×10^{-4}	2.63×10^{-4}	100%	100%	100%		\

>	0%	0%	0%					
45	9	57	32832	2250	1.90×10^{-4}	16s	20s	\
>	6.03×10^{-4}	6.00×10^{-4}	1.99×10^{-4}		100%	100%	100%	\
>	0%	0%	0%					
51	11	7	36928	1721	1.99×10^{-4}	18s	18s	\
>	5.25×10^{-4}	5.31×10^{-4}	1.78×10^{-4}		100%	100%	100%	\
>	0%	0%	0%					
56	12	16	41152	2024	2.07×10^{-4}	20s	16s	\
>	4.56×10^{-4}	4.64×10^{-4}	1.60×10^{-4}		100%	100%	100%	\
>	0%	0%	0%					
62	13	27	45504	2220	2.14×10^{-4}	22s	13s	\
>	4.02×10^{-4}	4.17×10^{-4}	1.46×10^{-4}		100%	100%	100%	\
>	0%	0%	0%					
68	14	35	49664	2230	2.20×10^{-4}	24s	11s	\
>	3.66×10^{-4}	3.80×10^{-4}	1.35×10^{-4}		100%	100%	100%	\
>	0%	0%	0%					
74	15	45	53952	2260	2.26×10^{-4}	26s	9s	\
>	3.26×10^{-4}	3.50×10^{-4}	1.25×10^{-4}		100%	100%	100%	\
>	0%	0%	0%					
80	16	52	58048	2336	2.32×10^{-4}	28s	7s	\
>	2.99×10^{-4}	3.19×10^{-4}	1.16×10^{-4}		100%	100%	100%	\
>	0%	0%	0%					
85	17	57	62016	2222	2.36×10^{-4}	30s	5s	\
>	2.77×10^{-4}	2.76×10^{-4}	1.02×10^{-4}		100%	100%	100%	\
>	0%	0%	0%					
91	19	9	66240	1745	2.41×10^{-4}	32s	3s	\
>	2.56×10^{-4}	2.58×10^{-4}	9.60×10^{-5}		100%	100%	100%	\
>	0%	0%	0%					
96	20	15	70272	1930	2.45×10^{-4}	34s	1s	\
>	2.41×10^{-4}	2.45×10^{-4}	9.07×10^{-5}		100%	100%	100%	\
>	0%	0%	0%					

STREAMING PERCEPTION FROZEN
ValidationEventAccuracy=1.

=====

```
GLOBAL FREEZE COMPLETE
TCCTCoreChanged=False
TCCTQFrozen=True
StreamingTransformerFrozen=True
TransformerHistoryClasses=False
ProspectiveCExists=False
GlobalFreezeHash=ed86ac4dedcb259b82d961fd4e50384ef06c03c0198522268fa7eeb23d4c\
```

```
> 1bfb
```

```
=====
PHASE 3: PROSPECTIVE C-FIRST OPENING
NO MODEL CHANGES AFTER THIS LINE
=====
```

```
ProspectiveHistories=6000
ProspectiveEvents=36000
Generating independent sensory stream...
Running streaming perception...
Perception=256/36000
Perception=512/36000
Perception=768/36000
Perception=1024/36000
Perception=1280/36000
Perception=1536/36000
Perception=1792/36000
Perception=2048/36000
Perception=2304/36000
Perception=2560/36000
Perception=2816/36000
Perception=3072/36000
Perception=3328/36000
Perception=3584/36000
Perception=3840/36000
Perception=4096/36000
Perception=4352/36000
Perception=4608/36000
Perception=4864/36000
Perception=5120/36000
Perception=5376/36000
Perception=5632/36000
Perception=5888/36000
Perception=6144/36000
Perception=6400/36000
Perception=6656/36000
Perception=6912/36000
Perception=7168/36000
Perception=7424/36000
Perception=7680/36000
Perception=7936/36000
Perception=8192/36000
Perception=8448/36000
Perception=8704/36000
Perception=8960/36000
Perception=9216/36000
Perception=9472/36000
```


Perception=9728/36000
Perception=9984/36000
Perception=10240/36000
Perception=10496/36000
Perception=10752/36000
Perception=11008/36000
Perception=11264/36000
Perception=11520/36000
Perception=11776/36000
Perception=12032/36000
Perception=12288/36000
Perception=12544/36000
Perception=12800/36000
Perception=13056/36000
Perception=13312/36000
Perception=13568/36000
Perception=13824/36000
Perception=14080/36000
Perception=14336/36000
Perception=14592/36000
Perception=14848/36000
Perception=15104/36000
Perception=15360/36000
Perception=15616/36000
Perception=15872/36000
Perception=16128/36000
Perception=16384/36000
Perception=16640/36000
Perception=16896/36000
Perception=17152/36000
Perception=17408/36000
Perception=17664/36000
Perception=17920/36000
Perception=18176/36000
Perception=18432/36000
Perception=18688/36000
Perception=18944/36000
Perception=19200/36000
Perception=19456/36000
Perception=19712/36000
Perception=19968/36000
Perception=20224/36000
Perception=20480/36000
Perception=20736/36000
Perception=20992/36000
Perception=21248/36000
Perception=21504/36000
Perception=21760/36000
Perception=22016/36000
Perception=22272/36000
Perception=22528/36000
Perception=22784/36000
Perception=23040/36000
Perception=23296/36000
Perception=23552/36000
Perception=23808/36000

Perception=24064/36000
Perception=24320/36000
Perception=24576/36000
Perception=24832/36000
Perception=25088/36000
Perception=25344/36000
Perception=25600/36000
Perception=25856/36000
Perception=26112/36000
Perception=26368/36000
Perception=26624/36000
Perception=26880/36000
Perception=27136/36000
Perception=27392/36000
Perception=27648/36000
Perception=27904/36000
Perception=28160/36000
Perception=28416/36000
Perception=28672/36000
Perception=28928/36000
Perception=29184/36000
Perception=29440/36000
Perception=29696/36000
Perception=29952/36000
Perception=30208/36000
Perception=30464/36000
Perception=30720/36000
Perception=30976/36000
Perception=31232/36000
Perception=31488/36000
Perception=31744/36000
Perception=32000/36000
Perception=32256/36000
Perception=32512/36000
Perception=32768/36000
Perception=33024/36000
Perception=33280/36000
Perception=33536/36000
Perception=33792/36000
Perception=34048/36000
Perception=34304/36000
Perception=34560/36000
Perception=34816/36000
Perception=35072/36000
Perception=35328/36000
Perception=35584/36000
Perception=35840/36000
Perception=36000/36000

StreamingEventAccuracy=1.
ExactHistoryRecovery=1.
AllowedHistoryRate=1.
QStatePreservation=1.

=====
S124-T3 FINAL SUMMARY

```

=====
TCCTCoreChanged=False
TransformerHistoryClasses=False
TransformerOutputClasses={1,2,3}
StreamingPerception=True
S3QHistories=12
EventBagStates=1
Recent2States=1
StructuralStates=12
QuotientStates=3
TCCTSplits=1
PerceptionValidationEventAccuracy=1.
ProspectiveEventAccuracy=1.
ExactHistoryRecovery=1.
AllowedHistoryRate=1.
QStatePreservation=1.
OracleBaseNLL=1.196
OracleTCCT_QNLL=0.824499
StreamingHybrid_QNLL=0.824499
RandomStreaming_QNLL=1.20231
OracleTCCTGain=0.371498
StreamingHybridGain=0.371498
RandomControlGain=-0.00631093
HybridGainRetention=1.

                                     -15
HybridPenaltyVsOracle=-1.9984 10
HybridValidHistoryRate=1.
RandomValidHistoryRate=0.015
HybridBeatsRandom=True
STRICT_STREAMING_PASS=True
DIAGNOSIS=STREAMING_TRANSFORMER_PERCEPTION_SUCCESSFULLY_DRIVES_TCCT_REASONING\

>   _CORE
=====
SummaryFile=E:\TCCT_CODEX_HANDOFF_2026-08-13\S97A_ReadoutBaseline_Development\

>   \S124_T3_Output\S124_T3_summary.wl

=====
S124-T3 COMPLETE
=====

```

In [565...

```

ClearAll["Global`*"];
$HistoryLength=0;

Print["====="];
Print["S124-T4 STANDALONE"];
Print["SHARED PERCEPTION REASONING ATTRIBUTION AUDIT"];
Print["TCCT-Q VS NEURAL REASONER"];
Print["====="];
Print["WolframVersion=", $Version];
Print["Date=",DateString[]];

T4Stop[msg_] := (Print["FATAL: ", msg]; Abort[]);

r3mDepth=6;

```

```
r3mAnchors={1,2,3};
r3mHub=10;
r3mWLRounds=4;
r3mSupports={2,4,6};
r3mEffects={0.,0.05};
r3mTopMins={0.30,0.35};
r3mMinChildStates=2;
r3mMinGroupObs=80;
r3mMinValGain=0.001;
r3mPenalty=0.00025;
r3mMaxSplits=12;
r3mMaxBlocks=24;
r3mKappa=8.;
r3mZero=ConstantArray[0,{24,4}];

t4CoreSeed=1134001;
t4WorldSeed=8242101;
t4BankA=15000;
t4BankB=15000;
t4BankC=6000;

t4PerceptionTrainPerClass=1200;
t4PerceptionValPerClass=300;

t4MicroLen=4;
t4SensorDim=12;
t4PosDim=8;
t4SensorScale=2.;
t4SensorNoise=0.30;
t4SensorDrift=0.12;
t4SensorGainJitter=0.12;

t4PerceptionDModel=64;
t4PerceptionHeads=4;
t4PerceptionLayers=2;
t4PerceptionFF=192;
t4PerceptionDropout=0.10;
t4PerceptionLR=0.0003;
t4PerceptionBatch=64;
t4PerceptionRounds=20;
t4PerceptionPatience=5;

t4ReasonHidden1=128;
t4ReasonHidden2=64;
t4ReasonDropout=0.10;
t4ReasonLR=0.0005;
t4ReasonBatch=128;
t4ReasonRounds=30;
t4ReasonPatience=5;

t4TargetDevice="CPU";

t4PrototypeSeed=8243001;
t4PerceptionTrainSeed=8243002;
t4PerceptionValSeed=8243003;
t4PerceptionNetSeed=8243004;
```

```

t4ReasonSelectionSeed=8243005;
t4ReasonFinalSeed=8243006;
t4SensorASeed=8243007;
t4SensorBSeed=8243008;
t4SensorCSeed=8243009;
t4RandomControlSeed=8243010;

t4WinnerMargin=0.02;
t4PerceptionGate=0.95;
t4ValidHistoryGate=0.98;
t4ReasonValidationGate=0.45;

If[OddQ[t4PosDim],T4Stop["t4PosDim must be even"]];
If[Mod[t4PerceptionDModel,t4PerceptionHeads]!=0,T4Stop["dModel must be divisible b

t4PerceptionHeadDim=Quotient[t4PerceptionDModel,t4PerceptionHeads];

t4OutputDirectory=FileNameJoin[{Directory[],"S124_T4_Output"}];
If[!DirectoryQ[t4OutputDirectory],CreateDirectory[t4OutputDirectory]];

R3MFiniteQ[x_]:=NumericQ[x]&&FreeQ[x,Indeterminate|ComplexInfinity|_DirectedInfinity]
R3MUE[a_,b_]:=UndirectedEdge@@Sort[{a,b}];
R3MSeqKey[s_List]:=ToString[s,InputForm];
R3MCat[p_List]:=RandomChoice[N[p]->Range[Length[p]]];

R3MInitial[]:=<|
"Active"->Union[R3MUE[#,r3mHub]&/@r3mAnchors],
"Frozen"->{},
"Next"->100
|>;

R3MNeighbors[active_List,v_]:=
DeleteDuplicates@Join[
Cases[active,UndirectedEdge[a_,b_]/;a==v:>b],
Cases[active,UndirectedEdge[a_,b_]/;b==v:>a]
];

R3MCandidates[active_List,required_List]:=
Module[{verts,ns,cands={},x,y,z,e1,e2},
verts=DeleteDuplicates[Flatten[List@@@active]];
Do[
y=verts[[k]];
ns=Sort[R3MNeighbors[active,y]];
Do[
x=ns[[i]];
z=ns[[j]];
e1=R3MUE[x,y];
e2=R3MUE[y,z];
If[Intersection[{e1,e2},required]!={},AppendTo[cands,{x,y,z,e1,e2}]],
{i,1,Max[0,Length[ns]-1]},
{j,i+1,Length[ns]}
],
{k,1,Length[verts]}
];
cands
];

```

```

R3MInject[state_Association,event_Integer,seed_Integer,step_Integer]:=
Module[{active,frozen,next,anchor,fresh,w,inj,cands,c,x,y,z,e1,e2,newActive,newFroz
active=state["Active"];
frozen=state["Frozen"];
next=state["Next"];
anchor=r3mAnchors[[event]];
fresh=next;
w=next+1;
inj={R3MUE[anchor,fresh],R3MUE[fresh,r3mHub]};
active=Union[Join[active,inj]];
cands=R3MCandidates[active,inj];
If[
cands==={},
Return[
<|
"Active"->DeleteCases[active,e_/_;MemberQ[inj,e]],
"Frozen"->frozen,
"Next"->next+2
|>
]
];
c=BlockRandom[
SeedRandom[seed+100003*step+7919*event];
RandomChoice[cands]
];
{x,y,z,e1,e2}=c;
newActive=DeleteCases[active,e_/_;MemberQ[{e1,e2},e]];
newActive=Union[
Join[
newActive,
{R3MUE[x,z],R3MUE[x,w],R3MUE[w,z]}
]
];
newActive=DeleteCases[newActive,e_/_;MemberQ[inj,e]];
newFrozen=Union[
Join[
frozen,
{DirectedEdge[x,y],DirectedEdge[y,z]}
]
];
<|
"Active"->newActive,
"Frozen"->newFrozen,
"Next"->next+2
|>
];

R3MStateKey[state_Association]:=
Module[{active,frozen,verts,col,new,na,no,ni,others},
active=state["Active"];
frozen=state["Frozen"];
verts=Union[
r3mAnchors,
{r3mHub},
Flatten[List@@@active],

```

```

If[frozen==={}, {}, Flatten[List@@@frozen]]
];
col=Association@Table[
v->Which[
v===r3mAnchors[[1]], "A1",
v===r3mAnchors[[2]], "A2",
v===r3mAnchors[[3]], "A3",
v===r3mHub, "H",
True, "X"
],
{v, verts}
];
Do[
new=Association@Table[
na=Sort@Join[
Cases[active, UndirectedEdge[a_, b_] /; a===v:>col[b]],
Cases[active, UndirectedEdge[a_, b_] /; b===v:>col[a]]
];
no=Sort[Cases[frozen, DirectedEdge[a_, b_] /; a===v:>col[b]]];
ni=Sort[Cases[frozen, DirectedEdge[a_, b_] /; b===v:>col[a]]];
v->Hash[{col[v], na, no, ni}, "SHA256", "HexString"],
{v, verts}
];
col=new,
{r3mWLRounds}
];
others=Sort[
Lookup[
col,
Complement[verts, Join[r3mAnchors, {r3mHub}]]
]
];
Hash[
{
Lookup[col, r3mAnchors],
col[r3mHub],
others,
Length[active],
Length[frozen]
},
"SHA256",
"HexString"
]
];

R3MBuildCore[coreSeed_Integer]:=
Module[{rows, seq, state, seqKeys, keys},
rows={<|"Seq"->{}, "State"->R3MInitial[]|>};
Do[
rows=Flatten[
Table[
Table[
seq=Append[row["Seq"], ev];
state=R3MInject[row["State"], ev, coreSeed, d];
<|"Seq"->seq, "State"->state|>,
{ev, 1, 3}

```

```

],
{row, rows}
],
1
];
Print["CoreDepth=", d, " Histories=", Length[rows]],
{d, 1, r3mDepth}
];
seqKeys=R3MSeqKey/@Lookup[rows, "Seq"];
keys=R3MStateKey/@Lookup[rows, "State"];
<|
"SeqToState"->AssociationThread[seqKeys, keys],
"SeqKeys"->seqKeys,
"KeyList"->keys,
"States"->DeleteDuplicates[keys],
"UniqueStates"->Length[DeleteDuplicates[keys]]
|>
];

r3oSeqs=Join[#, {1, 2}]&/@DeleteDuplicates[Permutations[{1, 2, 3, 3}]];
r3oSeqKeys=R3MSeqKey/@r3oSeqs;

If[Length[r3oSeqs]!=12, T4Stop["expected 12 S3Q histories"]];

If[
!And@@(
(
Sort[#]==Sort[{1, 1, 2, 2, 3, 3}]&&
Take[#, -2]=={1, 2}
)&/@r3oSeqs
),
T4Stop["S3Q history invariant failed"]
];

Print[""];
Print["====="];
Print["S3Q PRECHECK"];
Print["====="];
Print["HistoryCount=", Length[r3oSeqs]];
Print["AllEventBags={2, 2, 2}=True"];
Print["AllRecent2={1, 2}=True"];

r3mFullAll=R3MBuildCore[t4CoreSeed];

R3ORestrictData[data_Association]:=
Module[{vals},
vals=data["SeqToState"][#]&/@r3oSeqKeys;
<|
"SeqToState"->AssociationThread[r3oSeqKeys, vals],
"States"->DeleteDuplicates[vals],
"UniqueStates"->Length[DeleteDuplicates[vals]]
|>
];

t4FullData=R3ORestrictData[r3mFullAll];

```



```

t4BagStates=Length[
DeleteDuplicates[
Hash[
{"BAG",Count[#,1],Count[#,2],Count[#,3]},
"SHA256",
"HexString"
]&/@r3oSeqs
]
];

t4RecentStates=Length[
DeleteDuplicates[
Hash[{"REC2",Take[#,-2]}, "SHA256", "HexString"]&/@r3oSeqs
]
];

Print["StructuralStates=",t4FullData["UniqueStates"]];
Print["EventBagStates=",t4BagStates];
Print["Recent2States=",t4RecentStates];

If[t4FullData["UniqueStates"]!=12,T4Stop["expected 12 TCCT structural states"]];
If[t4BagStates!=1,T4Stop["EventBag shortcut failed"]];
If[t4RecentStates!=1,T4Stop["Recent2 shortcut failed"]];

Print["TCCTCoreChanged=False"];
Print["CORE PRECHECK=PASS"];

R30P[k_Integer]:=
0.08*ConstantArray[1.,4]+0.68*UnitVector[4,k];

R30World[seed_Integer]:=
BlockRandom[
SeedRandom[seed];
Module[{perm,regs,seqReg,off1,off2,p1,p2},
perm=RandomSample[r3oSeqKeys];
regs=Flatten[Table[ConstantArray[r,4],{r,1,3}]];
seqReg=AssociationThread[perm,regs];
off1=RandomInteger[{0,3}];
off2=RandomInteger[{0,3}];
p1=Table[
R30P[1+Mod[r+o+2*a+off1-2,4]],
{r,1,3},{o,1,4},{a,0,1}
];
p2=Table[
R30P[1+Mod[2*r+o+a6+2*a7+off2-2,4]],
{r,1,3},{o,1,4},{a6,0,1},{a7,0,1}
];
<|"SeqRegime"->seqReg,"P1"->p1,"P2"->p2|>
]
];

R30Sim[seed_Integer,world_Association]:=
BlockRandom[
SeedRandom[seed];
Module[{seq,key,reg,o,a6,a7,y1,y2},
seq=RandomChoice[r3oSeqs];

```

```

key=R3MSeqKey[seq];
reg=world["SeqRegime"][key];
o=RandomInteger[{1,4}];
a6=RandomInteger[{0,1}];
a7=RandomInteger[{0,1}];
y1=R3MCat[world["P1"][[reg,o,a6+1]]];
y2=R3MCat[world["P2"][[reg,o,a6+1,a7+1]]];
<|
"SeqKey" -> key,
"O6" -> o,
"A6" -> a6,
"A7" -> a7,
"Y1" -> y1,
"Y2" -> y2
|>
]
];

R3MProbe1[o_Integer,a6_Integer]:=1+2*(o-1)+a6;
R3MProbe2[o_Integer,a6_Integer,a7_Integer]:=9+4*(o-1)+2*a6+a7;

R3MCountsRows[rows_List]:=
Module[{m=ConstantArray[0,{24,4}],p1,p2},
Do[
p1=R3MProbe1[row["O6"],row["A6"]];
p2=R3MProbe2[row["O6"],row["A6"],row["A7"]];
m[[p1,row["Y1"]]]++;
m[[p2,row["Y2"]]]++;
{row,rows}
];
m
];

R3MStateCountsData[rows_List,data_Association]:=
Association@KeyValueMap[
Function[{k,rs},k->R3MCountsRows[rs]],
GroupBy[
rows,
data["SeqToState"][#["SeqKey"]]&
]
];

R3MGet[c_Association,s_]:=Lookup[c,s,r3mZero];

R3MPooled[c_Association,states_List]:=
Total[R3MGet[c,#]&/@states];

R3MObs[c_Association,states_List]:=
Total[Flatten[R3MPooled[c,states]]];

R3MAddCounts[a_Association,b_Association,states_List]:=
Association@Table[
s->(R3MGet[a,s]+R3MGet[b,s]),
{s,states}
];

```

```

R3MPrepare[rawA_List,rawB_List,data_Association]:=
Module[{a,b,states},
states=data["States"];
a=R3MStateCountsData[rawA,data];
b=R3MStateCountsData[rawB,data];
<|
"States"->states,
"A"->a,
"B"->b,
"AB"->R3MAddCounts[a,b,states]
|>
];

R3MLabel[v_,support_Integer,effect_?NumericQ,topMin_?NumericQ]:=
Module[{n,p,ord,top,runner},
If[!VectorQ[v,NumericQ]||Length[v]!=4,T4Stop["R3MLabel shape error"]];
n=Total[v];
If[n<support,Return[0]];
p=N[v/n];
ord=Reverse[Ordering[p]];
top=ord[[1]];
runner=ord[[2]];
If[
p[[top]]>=topMin&&
p[[top]]-p[[runner]]>=effect,
top,
0
]
];

R3MMakeGroups[
states_List,
cA_Association,
cB_Association,
probe_Integer,
support_Integer,
effect_?NumericQ,
topMin_?NumericQ
]:=
Module[{lab,ga,keys,pos,groups},
lab=Association@Table[
s->R3MLabel[R3MGet[cA,s][[probe]],support,effect,topMin],
{s,states}
];
ga=KeySort@GroupBy[states,lab[#]&];
keys=Keys[ga];
pos=Select[keys,#>0&];
If[Length[pos]<2,Return[$Failed]];
If[Min[Length/@Lookup[ga,pos]]<r3mMinChildStates,Return[$Failed]];
groups=Values[ga];
If[Min[R3MObs[cA,#]&/@groups]<r3mMinGroupObs,Return[$Failed]];
If[Min[R3MObs[cB,#]&/@groups]<r3mMinGroupObs,Return[$Failed]];
<|"Groups"->groups,"GroupLabels"->keys|>
];

R3MLossBase[train_,test_]:=

```

```

Module[{loss=0.,n=0,prob,den},
Do[
den=Total[train[[p]]]+4.;
prob=(N[train[[p]]]+1.)/den;
loss-=N[test[[p]].Log[prob]];
n+=Total[test[[p]]],
{p,1,24}
];
{loss,n}
];

R3MLossShrink[train_,parent_,test_]:=
Module[{loss=0.,n=0,parentP,prob,den},
Do[
parentP=(N[parent[[p]]]+1.)/(Total[parent[[p]]]+4.);
den=Total[train[[p]]]+r3mKappa;
prob=(N[train[[p]]]+r3mKappa*parentP)/den;
loss-=N[test[[p]].Log[prob]];
n+=Total[test[[p]]],
{p,1,24}
];
{loss,n}
];

R3MSplitEval[
states_List,
groups_List,
cA_Association,
cB_Association
]:=
Module[{pa,pb,parentLoss,splitLoss=0.,n=0,ca,cb,z,gain},
pa=R3MPooled[cA,states];
pb=R3MPooled[cB,states];
parentLoss=R3MLossBase[pa,pb];
Do[
ca=R3MPooled[cA,g];
cb=R3MPooled[cB,g];
z=R3MLossShrink[ca,pa,cb];
splitLoss+=z[[1]];
n+=z[[2]],
{g,groups}
];
If[n<=0||n!=parentLoss[[2]],Return[$Failed]];
gain=N[parentLoss[[1]]/n-splitLoss/n];
If[!R3MFiniteQ[gain],T4Stop["nonfinite split gain"]];
<|"ValGain"->gain|>
];

R3MBestSplit[
states_List,
cA_Association,
cB_Association
]:=
Module[{best=$Failed,bestScore=-Infinity,g,ev,score},
Do[
g=R3MMakeGroups[

```

```

states,
cA,
cB,
probe,
support,
effect,
topMin
];
If[
UnsameQ[g,$Failed],
ev=R3MSplitEval[states,g["Groups"],cA,cB];
If[
UnsameQ[ev,$Failed],
score=N[ev["ValGain"]-r3mPenalty*(Length[g["Groups"]]-1)];
If[
score>bestScore,
bestScore=score;
best=Join[
<|
"Probe"->probe,
"Support"->support,
"Effect"->effect,
"TopMin"->topMin
|>,
g,
ev,
<|"Score"->score|>
]
]
],
{probe,1,24},
{support,r3mSupports},
{effect,r3mEffects},
{topMin,r3mTopMins}
];
best
];

R3MDiscover[
states_List,
cA_Association,
cB_Association
]:=
Module[{blocks={states},log={},step=0,cands,best,bi},
While[
step<r3mMaxSplits&&Length[blocks]<r3mMaxBlocks,
cands=Cases[
Table[
If[
Length[blocks[[i]]]<2*r3mMinChildStates,
Nothing,
With[
{x=R3MBestSplit[blocks[[i]],cA,cB}],
If[
UnsameQ[x,$Failed],

```

```

Join[
  <|
    "BlockIndex"->i,
    "ParentStates"->Length[blocks[[i]]]
  |>,
  x
],
Nothing
]
],
{
  {i, 1, Length[blocks]}
},
_Association
];
If[cands=={}, Break[]];
best=First[MaximalBy[cands, #["Score"]&]];
If[
  best["ValGain"]<r3mMinValGain||
  best["Score"]<=0.,
  Break[]
];
bi=best["BlockIndex"];
step++;
Print[
  "ACCEPT SPLIT=", step,
  " STATES=", best["ParentStates"],
  " PROBE=", best["Probe"],
  " GAIN=", N[best["ValGain"]]
];
AppendTo[log, KeyDrop[best, {"Groups"}]];
blocks=Join[
  Take[blocks, bi-1],
  best["Groups"],
  Drop[blocks, bi]
];
];
<| "Blocks"->blocks, "Log"->log, "Splits"->step |>
];

R3MBlockMap[blocks_List]:=
Association@Flatten[
  MapIndexed[
    Thread[#1->First[#2]]&,
    blocks
  ]
];

Print[""];
Print["====="];
Print["PHASE 1: TRAIN EVENT PERCEPTION"];
Print["WORLD A/B/C DO NOT EXIST"];
Print["====="];

BlockRandom[
  SeedRandom[t4PrototypeSeed];

```

```

t4SensorBases=
Normalize/@RandomVariate[
NormalDistribution[0.,1.],
{3,t4SensorDim}
]
];

T4MicroPos[pos_Integer]:=
Flatten[
Table[
{
Sin[pos/(10000.^(2.0*k/t4PosDim))],
Cos[pos/(10000.^(2.0*k/t4PosDim))]}
],
{k,0,t4PosDim/2-1}
]
];

T4EventSensory[event_Integer,seed_Integer]:=
BlockRandom[
SeedRandom[seed];
Module[{drift,gain},
drift=RandomReal[
{-t4SensorDrift,t4SensorDrift},
t4SensorDim
];
N[
Table[
gain=1.+RandomReal[
{-t4SensorGainJitter,t4SensorGainJitter}
];
Join[
t4SensorScale*
gain*
t4SensorBases[[event]]+
drift+
RandomVariate[
NormalDistribution[0.,t4SensorNoise],
t4SensorDim
],
T4MicroPos[m]
],
{m,1,t4MicroLen}
]
]
];

t4PerceptionInputDim=t4SensorDim+t4PosDim;

T4MakePerceptionRaw[nPerClass_Integer,seed_Integer]:=
BlockRandom[
SeedRandom[seed];
RandomSample[
Flatten[
Table[

```

```

Table[
<|
"Input"->T4EventSensory[event,seed+100000*event+i],
"Label"->event
|>,
{i,1,nPerClass}
],
{event,1,3}
],
1
]
];

T4PerceptionBlock[modelDim_Integer,heads_Integer,ffDim_Integer,drop_?NumericQ]:=
Module[{hd},
hd=Quotient[modelDim,heads];
NetGraph[
<|
"LN1"->NormalizationLayer[2,"Same"],
"Q"->NetMapOperator[LinearLayer[{heads,hd}]],
"K"->NetMapOperator[LinearLayer[{heads,hd}]],
"V"->NetMapOperator[LinearLayer[{heads,hd}]],
"Attention"->AttentionLayer[
"Dot",
"MultiHead"->True,
"Mask"->"Causal",
"ScoreRescaling"->"DimensionSqrt",
"Dropout"->drop
],
"Merge"->NetMapOperator[
NetChain[{FlattenLayer[],LinearLayer[modelDim]}]
],
"ADrop"->DropoutLayer[drop],
"Res1"->ThreadingLayer[Plus],
"LN2"->NormalizationLayer[2,"Same"],
"FF"->NetMapOperator[
NetChain[
{
LinearLayer[ffDim],
ElementwiseLayer[Ramp],
DropoutLayer[drop],
LinearLayer[modelDim]
}
]
],
"FDrop"->DropoutLayer[drop],
"Res2"->ThreadingLayer[Plus]
|>,
{
NetPort["Input"]->"LN1",
"LN1"->"Q",
"LN1"->"K",
"LN1"->"V",
"Q"->NetPort["Attention","Query"],
"K"->NetPort["Attention","Key"],

```



```

"V"->NetPort["Attention","Value"],
"Attention"->"Merge",
"Merge"->"ADrop",
{NetPort["Input"],"ADrop"}->"Res1",
"Res1"->"LN2",
"LN2"->"FF",
"FF"->"FDrop",
{"Res1","FDrop"}->"Res2"
},
"Input"->{"Varying",modelDim}
]
];

T4PerceptionNet[]:=
Module[{blocks},
blocks=Table[
T4PerceptionBlock[
t4PerceptionDModel,
t4PerceptionHeads,
t4PerceptionFF,
t4PerceptionDropout
],
{t4PerceptionLayers}
];
NetChain[
Join[
{NetMapOperator[LinearLayer[t4PerceptionDModel]]},
blocks,
{
NormalizationLayer[2,"Same"],
SequenceLastLayer[],
LinearLayer[3],
SoftmaxLayer[]
}
],
"Input"->{"Varying",t4PerceptionInputDim},
"Output"->NetDecoder[{"Class",{1,2,3}}]
]
];

t4PerceptionTrain=T4MakePerceptionRaw[
t4PerceptionTrainPerClass,
t4PerceptionTrainSeed
];

t4PerceptionVal=T4MakePerceptionRaw[
t4PerceptionValPerClass,
t4PerceptionValSeed
];

t4PerceptionTrainRules=
(#["Input"]->#["Label"]&)/@t4PerceptionTrain;

t4PerceptionValRules=
(#["Input"]->#["Label"]&)/@t4PerceptionVal;

```

```

SeedRandom[t4PerceptionNetSeed];

t4PerceptionNet0=NetInitialize[T4PerceptionNet[]];

t4PerceptionTraining=
NetTrain[
t4PerceptionNet0,
t4PerceptionTrainRules,
All,
ValidationSet->t4PerceptionValRules,
MaxTrainingRounds->t4PerceptionRounds,
BatchSize->t4PerceptionBatch,
LearningRate->t4PerceptionLR,
Method->"ADAM",
TrainingProgressMeasurements->{"Accuracy", "ErrorRate"},
TrainingStoppingCriterion-><|
"Criterion"->"Loss",
"Patience"->t4PerceptionPatience
|>,
TrainingProgressReporting->"Print",
TargetDevice->t4TargetDevice,
RandomSeeding->t4PerceptionNetSeed
];

t4FrozenPerception=t4PerceptionTraining["TrainedNet"];

t4PerceptionValidationAccuracy=
NetMeasurements[
t4FrozenPerception,
t4PerceptionValRules,
"Accuracy",
BatchSize->t4PerceptionBatch
];

Print[""];
Print["PERCEPTION FROZEN"];
Print["ValidationEventAccuracy=", t4PerceptionValidationAccuracy];

If[
t4PerceptionValidationAccuracy<t4PerceptionGate,
T4Stop["perception validation gate failed"]
];

Export[
FileNameJoin[
{t4OutputDirectory, "S124_T4_PERCEPTION_FROZEN_BEFORE_WORLD.wlnet"}
],
t4FrozenPerception
];

Clear[
t4PerceptionTrain,
t4PerceptionVal,
t4PerceptionTrainRules,
t4PerceptionValRules,
t4PerceptionNet0,

```

```

t4PerceptionTraining
];

T4SeqFromKey[key_String]:=ToExpression[key];

T4PerceiveRows[net_,rows_List,baseSeed_Integer,batchRows_Integer:128]:=
Module[
{predSeqs={},trueSeqs,flatInputs,flatPred,chunk,last,i,j,eventAcc,historyAcc,validR
trueSeqs=T4SeqFromKey/@Lookup[rows,"SeqKey"];
Do[
last=Min[i+batchRows-1,Length[rows]];
chunk=Take[trueSeqs,{i,last}];
flatInputs=Flatten[
Table[
Table[
T4EventSensory[
chunk[[j,k]],
baseSeed+100000*(i+j-1)+k
],
{k,1,6}
],
{j,1,Length[chunk]}
],
1
];
flatPred=net[flatInputs];
predSeqs=Join[predSeqs,Partition[flatPred,6]];
Print["PerceivedRows=",last,"/",Length[rows]],
{i,1,Length[rows],batchRows}
];
eventAcc=N[
Mean[
Flatten[
MapThread[
Boole[#1==#2]&,
{Flatten[predSeqs],Flatten[trueSeqs]}
]
]
];
historyAcc=N[
Mean[
MapThread[
Boole[SameQ[#1,#2]]&,
{predSeqs,trueSeqs}
]
]
];
validRate=N[
Mean[
Boole[
MemberQ[r3oSeqKeys,R3MSeqKey[#]]
]&/@predSeqs
]
];
<|

```

```

"PredSeqs" -> predSeqs,
"PredKeys" -> (R3MSeqKey/@predSeqs),
"TrueSeqs" -> trueSeqs,
"TrueKeys" -> Lookup[rows, "SeqKey"],
"EventAccuracy" -> eventAcc,
"ExactHistoryAccuracy" -> historyAcc,
"ValidHistoryRate" -> validRate
|>
];

T4ReplaceKeys[rows_List, keys_List] :=
MapThread[
Join[
KeyDrop[#1, {"SeqKey"}],
<| "SeqKey" -> #2 |>
]&,
{rows, keys}
];

Print[""];
Print["====="];
Print["PHASE 2: GENERATE DISCOVERY A + VALIDATION B"];
Print["PROSPECTIVE C DOES NOT EXIST"];
Print["====="];

t4World = R30World[t4WorldSeed];

t4RawA = Table[
R30Sim[t4WorldSeed + 81000000 + i, t4World],
{i, 1, t4BankA}
];

t4RawB = Table[
R30Sim[t4WorldSeed + 82000000 + i, t4World],
{i, 1, t4BankB}
];

Print["Perceiving Discovery A..."];
t4PerceivedA = T4PerceiveRows[
t4FrozenPerception,
t4RawA,
t4SensorASeed
];

Print["Perceiving Validation B..."];
t4PerceivedB = T4PerceiveRows[
t4FrozenPerception,
t4RawB,
t4SensorBSeed
];

Print[""];
Print["A EventAccuracy=", t4PerceivedA["EventAccuracy"]];
Print["A ExactHistoryAccuracy=", t4PerceivedA["ExactHistoryAccuracy"]];
Print["A ValidHistoryRate=", t4PerceivedA["ValidHistoryRate"]];
Print["B EventAccuracy=", t4PerceivedB["EventAccuracy"]];

```

```

Print["B ExactHistoryAccuracy=",t4PerceivedB["ExactHistoryAccuracy"]];
Print["B ValidHistoryRate=",t4PerceivedB["ValidHistoryRate"]];

If[
t4PerceivedA["ValidHistoryRate"]<t4ValidHistoryGate||
t4PerceivedB["ValidHistoryRate"]<t4ValidHistoryGate,
T4Stop["A/B perception produces too many invalid histories"]
];

If[
t4PerceivedA["ValidHistoryRate"]<1.||
t4PerceivedB["ValidHistoryRate"]<1.,
T4Stop["T4 requires all A/B perceived histories to remain in the controlled 12-hist
];

t4RawAPerceived=
T4ReplaceKeys[t4RawA,t4PerceivedA["PredKeys"]];

t4RawBPerceived=
T4ReplaceKeys[t4RawB,t4PerceivedB["PredKeys"]];

Print[""];
Print["====="];
Print["PHASE 3: FREEZE TCCT REASONING CORE"];
Print["PROSPECTIVE C STILL DOES NOT EXIST"];
Print["====="];

t4Prep=
R3MPrepare[
t4RawAPerceived,
t4RawBPerceived,
t4FullData
];

Print["StructuralStates=",Length[t4Prep["States"]]];

t4Disc=
R3MDiscover[
t4Prep["States"],
t4Prep["A"],
t4Prep["B"]
];

t4Blocks=t4Disc["Blocks"];
t4QStates=Length[t4Blocks];

Print["QuotientStates=",t4QStates];
Print["TCCTSplits=",t4Disc["Splits"]];

If[t4QStates!=3,T4Stop["expected exactly 3 predictive quotient states"]];

t4TCCTModel=<|
"States"->t4Prep["States"],
"Blocks"->t4Blocks,
"TrainCounts"->t4Prep["AB"],
"StructuralStates"->Length[t4Prep["States"]],

```

```

"QuotientStates"->t4QStates,
"Splits"->t4Disc["Splits"]
|>;

t4TCCTFreezeHash=
Hash[
ToString[
{
"S124-T4",
t4WorldSeed,
t4CoreSeed,
t4Blocks,
t4Disc["Log"]
},
InputForm
],
"SHA256",
"HexString"
];

Print["TCCTFreezeHash=", t4TCCTFreezeHash];

T4A7Vector[a7_Integer, probe_Integer] :=
If[
probe == 1,
UnitVector[3, 1],
UnitVector[3, a7 + 2]
];

T4ReasonInput[
seq_List,
o_Integer,
a6_Integer,
a7_Integer,
probe_Integer
] :=
N@Join[
Flatten[UnitVector[3, #] & /@ seq],
UnitVector[4, o],
UnitVector[2, a6 + 1],
T4A7Vector[a7, probe],
UnitVector[2, probe]
];

t4ReasonInputDim = 6 * 3 + 4 + 2 + 3 + 2;

T4ReasonRules[rows_List, seqs_List] :=
Flatten[
MapThread[
{
T4ReasonInput[
#1,
#2["O6"],
#2["A6"],
0,
1

```

```

]->#2["Y1"],
T4ReasonInput[
#1,
#2["06"],
#2["A6"],
#2["A7"],
2
]->#2["Y2"]
}&,
{seqs,rows}
],
1
];

T4ReasonNet[]:=
NetChain[
{
LinearLayer[t4ReasonHidden1],
ElementwiseLayer[Ramp],
DropoutLayer[t4ReasonDropout],
LinearLayer[t4ReasonHidden2],
ElementwiseLayer[Ramp],
LinearLayer[4],
SoftmaxLayer[]
},
"Input"->{t4ReasonInputDim},
"Output"->NetDecoder[{"Class",{1,2,3,4}}]
];

Print[""];
Print["====="];
Print["PHASE 4: TRAIN MATCHED NEURAL REASONER"];
Print["SAME PERCEIVED A/B AS TCCT"];
Print["PROSPECTIVE C STILL DOES NOT EXIST"];
Print["====="];

t4ReasonARules=
T4ReasonRules[
t4RawA,
t4PerceivedA["PredSeqs"]
];

t4ReasonBRules=
T4ReasonRules[
t4RawB,
t4PerceivedB["PredSeqs"]
];

SeedRandom[t4ReasonSelectionSeed];

t4ReasonSelectionNet=
NetInitialize[T4ReasonNet[]];

t4ReasonSelectionTraining=
NetTrain[
t4ReasonSelectionNet,

```

```

t4ReasonARules,
All,
ValidationSet->t4ReasonBRules,
MaxTrainingRounds->t4ReasonRounds,
BatchSize->t4ReasonBatch,
LearningRate->t4ReasonLR,
Method->"ADAM",
TrainingProgressMeasurements->{"Accuracy", "ErrorRate"},
TrainingStoppingCriterion-><|
"Criterion"->"Loss",
"Patience"->t4ReasonPatience
|>,
TrainingProgressReporting->"Print",
TargetDevice->t4TargetDevice,
RandomSeeding->t4ReasonSelectionSeed
];

t4ReasonBestRound=
t4ReasonSelectionTraining["BestValidationRound"];

t4ReasonValidationAccuracy=
NetMeasurements[
t4ReasonSelectionTraining["TrainedNet"],
t4ReasonBRules,
"Accuracy",
BatchSize->t4ReasonBatch
];

Print[""];
Print["ReasonSelectionBestRound=", t4ReasonBestRound];
Print["ReasonValidationAccuracy=", t4ReasonValidationAccuracy];

If[
t4ReasonValidationAccuracy<t4ReasonValidationGate,
T4Stop["neural reasoner failed validation competence gate"]
];

t4ReasonABRules=
Join[
t4ReasonARules,
t4ReasonBRules
];

SeedRandom[t4ReasonFinalSeed];

t4ReasonFinalNet0=
NetInitialize[T4ReasonNet[]];

t4ReasonFinalTraining=
NetTrain[
t4ReasonFinalNet0,
t4ReasonABRules,
All,
MaxTrainingRounds->Max[1, t4ReasonBestRound],
BatchSize->t4ReasonBatch,
LearningRate->t4ReasonLR,

```



```

Method->"ADAM",
TrainingProgressMeasurements->{"Accuracy", "ErrorRate"},
TrainingProgressReporting->"Print",
TargetDevice->t4TargetDevice,
RandomSeeding->t4ReasonFinalSeed
];

t4FrozenReasoner=
t4ReasonFinalTraining["TrainedNet"];

Export[
FileNameJoin[
{t4OutputDirectory, "S124_T4_NEURAL_REASONER_FROZEN_BEFORE_C.wlnet"}
],
t4FrozenReasoner
];

Print[""];
Print["NEURAL REASONER FROZEN"];

t4GlobalFreezeHash=
Hash[
ToString[
{
t4TCCTFreezeHash,
t4ReasonBestRound,
t4ReasonFinalSeed,
t4PerceptionNetSeed
},
InputForm
],
"SHA256",
"HexString"
];

Print[""];
Print["====="];
Print["GLOBAL FREEZE COMPLETE"];
Print["SharedPerception=True"];
Print["TCCTFrozen=True"];
Print["NeuralReasonerFrozen=True"];
Print["ProspectiveCExists=False"];
Print["GlobalFreezeHash=", t4GlobalFreezeHash];
Print["====="];

T4BaseProb[parent_, probe_Integer] :=
(N[parent[[probe]]+1.)/
(Total[parent[[probe]]+4.]);

T4BlockProb[local_, parent_, probe_Integer] :=
Module[{parentP, den},
parentP=T4BaseProb[parent, probe];
den=Total[local[[probe]]+r3mKappa;
(N[local[[probe]]+r3mKappa*parentP]/den
];

```

```

T4TCCTEvaluateKeys[
model_Association,
rows_List,
keys_List,
data_Association
]:=
Module[
{
blocks,states,cTrain,blockMap,blockTrain,globalTrain,
loss=0.,correct=0,n=0,valid=0,row,key,state,bid,p1,p2,prob,pred
},
blocks=model["Blocks"];
states=model["States"];
cTrain=model["TrainCounts"];
blockMap=R3MBlockMap[blocks];
globalTrain=R3MPooled[cTrain,states];
blockTrain=AssociationThread[
Range[Length[blocks]],
R3MPooled[cTrain,#]&/@blocks
];
Do[
row=rows[[i]];
key=keys[[i]];
p1=R3MProbe1[row["06"],row["A6"]];
p2=R3MProbe2[row["06"],row["A6"],row["A7"]];
If[
KeyExistsQ[data["SeqToState"],key],
state=data["SeqToState"][key];
If[
KeyExistsQ[blockMap,state],
bid=blockMap[state];
valid++;
prob=T4BlockProb[blockTrain[bid],globalTrain,p1],
prob=T4BaseProb[globalTrain,p1]
],
prob=T4BaseProb[globalTrain,p1]
];
loss-=Log[Max[10.^-15,prob[[row["Y1"]]]]];
pred=First[Ordering[prob,-1]];
correct+=Boole[pred==row["Y1"]];
If[
KeyExistsQ[data["SeqToState"],key],
state=data["SeqToState"][key];
If[
KeyExistsQ[blockMap,state],
bid=blockMap[state];
prob=T4BlockProb[blockTrain[bid],globalTrain,p2],
prob=T4BaseProb[globalTrain,p2]
],
prob=T4BaseProb[globalTrain,p2]
];
loss-=Log[Max[10.^-15,prob[[row["Y2"]]]]];
pred=First[Ordering[prob,-1]];
correct+=Boole[pred==row["Y2"]];
n+=2,
{i,1,Length[rows]}

```

```

];
<|
"NLL"->N[loss/n],
"Accuracy"->N[correct/n],
"ValidHistoryRate"->N[valid/Length[rows]]
|>
];

Print[""];
Print["====="];
Print["PHASE 5: PROSPECTIVE C-FIRST OPENING"];
Print["NO MODEL CHANGES AFTER THIS LINE"];
Print["====="];

t4RawC=Table[
R30Sim[t4WorldSeed+83000000+i,t4World],
{i,1,t4BankC}
];

Print["ProspectiveRows=",Length[t4RawC]];

Print["Perceiving Prospective C..."];

t4PerceivedC=
T4PerceiveRows[
t4FrozenPerception,
t4RawC,
t4SensorCSeed
];

Print[""];
Print["C EventAccuracy=",t4PerceivedC["EventAccuracy"]];
Print["C ExactHistoryAccuracy=",t4PerceivedC["ExactHistoryAccuracy"]];
Print["C ValidHistoryRate=",t4PerceivedC["ValidHistoryRate"]];

t4OracleTCCT=
T4TCCTEvaluateKeys[
t4TCCTModel,
t4RawC,
Lookup[t4RawC,"SeqKey"],
t4FullData
];

t4SharedTCCT=
T4TCCTEvaluateKeys[
t4TCCTModel,
t4RawC,
t4PerceivedC["PredKeys"],
t4FullData
];

BlockRandom[
SeedRandom[t4RandomControlSeed];
t4MatchedRandomKeys=
RandomChoice[
r30SeqKeys,

```

```

Length[t4RawC]
]
];

t4MatchedRandomTCCT=
T4TCCTEvaluateKeys[
t4TCCTModel,
t4RawC,
t4MatchedRandomKeys,
t4FullData
];

t4ReasonCRules=
T4ReasonRules[
t4RawC,
t4PerceivedC["PredSeqs"]
];

t4NeuralProspectiveAccuracy=
NetMeasurements[
t4FrozenReasoner,
t4ReasonCRules,
"Accuracy",
BatchSize->t4ReasonBatch
];

t4TCCTAccuracy=
t4SharedTCCT["Accuracy"];

t4AccuracyDelta=
N[
t4TCCTAccuracy-
t4NeuralProspectiveAccuracy
];

t4OraclePenalty=
N[
t4SharedTCCT["NLL"]-
t4OracleTCCT["NLL"]
];

t4ProtocolPass=
And[
t4PerceptionValidationAccuracy>=t4PerceptionGate,
t4PerceivedC["EventAccuracy"]>=t4PerceptionGate,
t4PerceivedC["ValidHistoryRate"]>=t4ValidHistoryGate,
t4QStates===3,
t4MatchedRandomTCCT["ValidHistoryRate"]===1.,
t4ReasonValidationAccuracy>=t4ReasonValidationGate
];

t4ReasoningDiagnosis=
Which[
!TrueQ[t4ProtocolPass],
"PROTOCOL_GATE_FAILED",
t4AccuracyDelta>=t4WinnerMargin,

```

```

"TCCT_REASONING_ADVANTAGE_UNDER_SHARED_PERCEPTION",
t4AccuracyDelta<=-t4WinnerMargin,
"NEURAL_REASONING_ADVANTAGE_UNDER_SHARED_PERCEPTION",
True,
"REASONING_PARITY_WITHIN_PREDECLARED_2_PERCENT_MARGIN"
];

Print[""];
Print["====="];
Print["S124-T4 FINAL SUMMARY"];
Print["====="];
Print["TCCTCoreChanged=False"];
Print["SharedPerception=True"];
Print["TransformerHistoryClasses=False"];
Print["PerceptionOutputClasses={1,2,3}"];
Print["S3QHistories=",Length[r3oSeqs]];
Print["EventBagStates=",t4BagStates];
Print["Recent2States=",t4RecentStates];
Print["StructuralStates=",t4TCCTModel["StructuralStates"]];
Print["QuotientStates=",t4TCCTModel["QuotientStates"]];
Print["TCCTSplits=",t4TCCTModel["Splits"]];
Print["PerceptionValidationAccuracy=",t4PerceptionValidationAccuracy];
Print["AEventAccuracy=",t4PerceivedA["EventAccuracy"]];
Print["BEventAccuracy=",t4PerceivedB["EventAccuracy"]];
Print["CEventAccuracy=",t4PerceivedC["EventAccuracy"]];
Print["CExactHistoryAccuracy=",t4PerceivedC["ExactHistoryAccuracy"]];
Print["CValidHistoryRate=",t4PerceivedC["ValidHistoryRate"]];
Print["ReasonValidationAccuracy=",t4ReasonValidationAccuracy];
Print["ReasonBestRound=",t4ReasonBestRound];
Print["OracleTCCT_NLL=",t4OracleTCCT["NLL"]];
Print["OracleTCCT_Accuracy=",t4OracleTCCT["Accuracy"]];
Print["SharedPerceptionTCCT_NLL=",t4SharedTCCT["NLL"]];
Print["SharedPerceptionTCCT_Accuracy=",t4SharedTCCT["Accuracy"]];
Print["NeuralReasoner_Accuracy=",t4NeuralProspectiveAccuracy];
Print["MatchedRandomTCCT_NLL=",t4MatchedRandomTCCT["NLL"]];
Print["MatchedRandomTCCT_Accuracy=",t4MatchedRandomTCCT["Accuracy"]];
Print["MatchedRandomValidHistoryRate=",t4MatchedRandomTCCT["ValidHistoryRate"]];
Print["TCCTMinusNeuralAccuracy=",t4AccuracyDelta];
Print["TCCTPenaltyVsOracle=",t4OraclePenalty];
Print["WinnerMargin=",t4WinnerMargin];
Print["STRICT_PROTOCOL_PASS=",t4ProtocolPass];
Print["REASONING_DIAGNOSIS=",t4ReasoningDiagnosis];
Print["====="];

t4Summary=<|
"Stage"->"S124-T4",
"Purpose"->"SharedPerceptionReasoningAttributionAudit",
"TCCTCoreChanged"->False,
"SharedPerception"->True,
"TransformerHistoryClasses"->False,
"PerceptionOutputClasses"->{1,2,3},
"S3QHistories"->Length[r3oSeqs],
"EventBagStates"->t4BagStates,
"Recent2States"->t4RecentStates,
"StructuralStates"->t4TCCTModel["StructuralStates"],
"QuotientStates"->t4TCCTModel["QuotientStates"],

```

```

"TCCTSplits"->t4TCCTModel["Splits"],
"PerceptionValidationAccuracy"->t4PerceptionValidationAccuracy,
"AEventAccuracy"->t4PerceivedA["EventAccuracy"],
"BEventAccuracy"->t4PerceivedB["EventAccuracy"],
"CEventAccuracy"->t4PerceivedC["EventAccuracy"],
"CExactHistoryAccuracy"->t4PerceivedC["ExactHistoryAccuracy"],
"CValidHistoryRate"->t4PerceivedC["ValidHistoryRate"],
"ReasonValidationAccuracy"->t4ReasonValidationAccuracy,
"ReasonBestRound"->t4ReasonBestRound,
"OracleTCCTNLL"->t4OracleTCCT["NLL"],
"OracleTCCTAccuracy"->t4OracleTCCT["Accuracy"],
"SharedTCCTNLL"->t4SharedTCCT["NLL"],
"SharedTCCTAccuracy"->t4SharedTCCT["Accuracy"],
"NeuralReasonerAccuracy"->t4NeuralProspectiveAccuracy,
"MatchedRandomTCCTNLL"->t4MatchedRandomTCCT["NLL"],
"MatchedRandomTCCTAccuracy"->t4MatchedRandomTCCT["Accuracy"],
"MatchedRandomValidHistoryRate"->t4MatchedRandomTCCT["ValidHistoryRate"],
"TCCTMinusNeuralAccuracy"->t4AccuracyDelta,
"TCCTPenaltyVsOracle"->t4OraclePenalty,
"WinnerMargin"->t4WinnerMargin,
"StrictProtocolPass"->t4ProtocolPass,
"ReasoningDiagnosis"->t4ReasoningDiagnosis,
"TCCTFreezeHash"->t4TCCTFreezeHash,
"GlobalFreezeHash"->t4GlobalFreezeHash,
"ProspectiveGeneratedAfterAllModelsFrozen"->True
|>;

t4SummaryFile=
FileNameJoin[
{t4OutputDirectory,"S124_T4_summary.wl"}
];

Put[t4Summary,t4SummaryFile];

Print["SummaryFile=",t4SummaryFile];
Print[""];
Print["====="];
Print["S124-T4 COMPLETE"];
Print["====="];

```

```

=====
S124-T4 STANDALONE
SHARED PERCEPTION REASONING ATTRIBUTION AUDIT
TCCT-Q VS NEURAL REASONER
=====
WolframVersion=15.0.0 for Microsoft Windows (64-bit) (May 26, 2026)
Date=Wed 19 Aug 2026 21:55:04

```

```

=====
S3Q PRECHECK
=====
HistoryCount=12
AllEventBags={2,2,2}=True
AllRecent2={1,2}=True
CoreDepth=1 Histories=3
CoreDepth=2 Histories=9
CoreDepth=3 Histories=27
CoreDepth=4 Histories=81
CoreDepth=5 Histories=243
CoreDepth=6 Histories=729
StructuralStates=12
EventBagStates=1
Recent2States=1
TCCTCoreChanged=False
CORE PRECHECK=PASS

```

```

=====
PHASE 1: TRAIN EVENT PERCEPTION
WORLD A/B/C DO NOT EXIST
=====

```

```

Starting training.
Optimization Method: ADAM
    Beta1: 9.00*^-1
    Beta2: 9.99*^-1
    Epsilon: 1.00*^-5
    Gradient Clipping: --
    L2 Regularization: --
    Learning Rate: 3.00*^-4
    Learning Rate Schedule: --
    Weight Clipping: --

```

```

Device: CPU
Batch Size: 64
Batches Per Round: 57
Training Examples: 3600

```

%	round	batch	examples	inputs	learning	time	time	\
>	current	round	test	current	round	test	current\	
>	round	test						
	/20	/57	processed	/second	rate	elapsed	left	\
>	loss	loss	loss	acc.	acc.	acc.	error	\
>	error	error						
	6	2	7	4096	1906	7.48*^-5	2s	34s \

>	1.13 ⁻¹	1.27 ⁻¹	2.50 ⁻³	96.24%	95.78%	100%	\
>	3.76%	4.22%	0%				
11	3	14	8192	1923	1.04 ⁻⁴	4s	32s \
>	3.95 ⁻³	4.34 ⁻³	1.32 ⁻³	100%	100%	100%	\
>	0%	0%	0%				
17	4	21	12288	2100	1.25 ⁻⁴	6s	30s \
>	2.37 ⁻³	2.69 ⁻³	8.46 ⁻⁴	100%	100%	100%	\
>	0%	0%	0%				
22	5	28	16384	2171	1.43 ⁻⁴	8s	28s \
>	1.68 ⁻³	1.87 ⁻³	6.00 ⁻⁴	100%	100%	100%	\
>	0%	0%	0%				
28	6	34	20416	2206	1.57 ⁻⁴	10s	26s \
>	1.20 ⁻³	1.41 ⁻³	4.55 ⁻⁴	100%	100%	100%	\
>	0%	0%	0%				
34	7	46	24832	2250	1.70 ⁻⁴	12s	23s \
>	9.38 ⁻⁴	1.08 ⁻³	3.65 ⁻⁴	100%	100%	100%	\
>	0%	0%	0%				
40	8	57	29184	2242	1.82 ⁻⁴	14s	21s \
>	7.62 ⁻⁴	7.46 ⁻⁴	2.57 ⁻⁴	100%	100%	100%	\
>	0%	0%	0%				
46	10	7	33280	1676	1.91 ⁻⁴	16s	19s \
>	6.38 ⁻⁴	6.46 ⁻⁴	2.24 ⁻⁴	100%	100%	100%	\
>	0%	0%	0%				
51	11	14	37376	1967	2.00 ⁻⁴	18s	17s \
>	5.59 ⁻⁴	5.69 ⁻⁴	1.98 ⁻⁴	100%	100%	100%	\
>	0%	0%	0%				
57	12	22	41536	2098	2.07 ⁻⁴	20s	15s \
>	4.88 ⁻⁴	5.05 ⁻⁴	1.77 ⁻⁴	100%	100%	100%	\
>	0%	0%	0%				
62	13	28	45568	2197	2.14 ⁻⁴	22s	13s \
>	4.36 ⁻⁴	4.55 ⁻⁴	1.60 ⁻⁴	100%	100%	100%	\
>	0%	0%	0%				
68	14	35	49664	2194	2.20 ⁻⁴	24s	11s \
>	3.86 ⁻⁴	4.11 ⁻⁴	1.46 ⁻⁴	100%	100%	100%	\

>	0%	0%	0%					
	74	15	42	53760	2257	2.26×10^{-4}	26s	9s \
>	3.59×10^{-4}	3.70×10^{-4}	1.35×10^{-4}			100%	100%	100% \
>	0%	0%	0%					
	79	16	49	57856	2256	2.31×10^{-4}	28s	7s \
>	3.18×10^{-4}	3.49×10^{-4}	1.25×10^{-4}			100%	100%	100% \
>	0%	0%	0%					
	85	17	56	61952	2254	2.36×10^{-4}	30s	5s \
>	2.91×10^{-4}	3.14×10^{-4}	1.16×10^{-4}			100%	100%	100% \
>	0%	0%	0%					
	90	18	57	65664	2234	2.40×10^{-4}	32s	4s \
>	2.74×10^{-4}	2.74×10^{-4}	1.02×10^{-4}			100%	100%	100% \
>	0%	0%	0%					
	96	20	8	69824	1727	2.45×10^{-4}	34s	2s \
>	2.55×10^{-4}	2.56×10^{-4}	9.65×10^{-5}			100%	100%	100% \
>	0%	0%	0%					

PERCEPTION FROZEN

ValidationEventAccuracy=1.

=====

PHASE 2: GENERATE DISCOVERY A + VALIDATION B

PROSPECTIVE C DOES NOT EXIST

=====

Perceiving Discovery A...

PerceivedRows=128/15000

PerceivedRows=256/15000

PerceivedRows=384/15000

PerceivedRows=512/15000

PerceivedRows=640/15000

PerceivedRows=768/15000

PerceivedRows=896/15000

PerceivedRows=1024/15000

PerceivedRows=1152/15000

PerceivedRows=1280/15000

PerceivedRows=1408/15000

PerceivedRows=1536/15000

PerceivedRows=1664/15000

PerceivedRows=1792/15000

PerceivedRows=1920/15000

PerceivedRows=2048/15000

PerceivedRows=2176/15000

PerceivedRows=2304/15000

PerceivedRows=2432/15000

PerceivedRows=2560/15000

PerceivedRows=2688/15000
PerceivedRows=2816/15000
PerceivedRows=2944/15000
PerceivedRows=3072/15000
PerceivedRows=3200/15000
PerceivedRows=3328/15000
PerceivedRows=3456/15000
PerceivedRows=3584/15000
PerceivedRows=3712/15000
PerceivedRows=3840/15000
PerceivedRows=3968/15000
PerceivedRows=4096/15000
PerceivedRows=4224/15000
PerceivedRows=4352/15000
PerceivedRows=4480/15000
PerceivedRows=4608/15000
PerceivedRows=4736/15000
PerceivedRows=4864/15000
PerceivedRows=4992/15000
PerceivedRows=5120/15000
PerceivedRows=5248/15000
PerceivedRows=5376/15000
PerceivedRows=5504/15000
PerceivedRows=5632/15000
PerceivedRows=5760/15000
PerceivedRows=5888/15000
PerceivedRows=6016/15000
PerceivedRows=6144/15000
PerceivedRows=6272/15000
PerceivedRows=6400/15000
PerceivedRows=6528/15000
PerceivedRows=6656/15000
PerceivedRows=6784/15000
PerceivedRows=6912/15000
PerceivedRows=7040/15000
PerceivedRows=7168/15000
PerceivedRows=7296/15000
PerceivedRows=7424/15000
PerceivedRows=7552/15000
PerceivedRows=7680/15000
PerceivedRows=7808/15000
PerceivedRows=7936/15000
PerceivedRows=8064/15000
PerceivedRows=8192/15000
PerceivedRows=8320/15000
PerceivedRows=8448/15000
PerceivedRows=8576/15000
PerceivedRows=8704/15000
PerceivedRows=8832/15000
PerceivedRows=8960/15000
PerceivedRows=9088/15000
PerceivedRows=9216/15000
PerceivedRows=9344/15000
PerceivedRows=9472/15000
PerceivedRows=9600/15000
PerceivedRows=9728/15000

PerceivedRows=9856/15000
PerceivedRows=9984/15000
PerceivedRows=10112/15000
PerceivedRows=10240/15000
PerceivedRows=10368/15000
PerceivedRows=10496/15000
PerceivedRows=10624/15000
PerceivedRows=10752/15000
PerceivedRows=10880/15000
PerceivedRows=11008/15000
PerceivedRows=11136/15000
PerceivedRows=11264/15000
PerceivedRows=11392/15000
PerceivedRows=11520/15000
PerceivedRows=11648/15000
PerceivedRows=11776/15000
PerceivedRows=11904/15000
PerceivedRows=12032/15000
PerceivedRows=12160/15000
PerceivedRows=12288/15000
PerceivedRows=12416/15000
PerceivedRows=12544/15000
PerceivedRows=12672/15000
PerceivedRows=12800/15000
PerceivedRows=12928/15000
PerceivedRows=13056/15000
PerceivedRows=13184/15000
PerceivedRows=13312/15000
PerceivedRows=13440/15000
PerceivedRows=13568/15000
PerceivedRows=13696/15000
PerceivedRows=13824/15000
PerceivedRows=13952/15000
PerceivedRows=14080/15000
PerceivedRows=14208/15000
PerceivedRows=14336/15000
PerceivedRows=14464/15000
PerceivedRows=14592/15000
PerceivedRows=14720/15000
PerceivedRows=14848/15000
PerceivedRows=14976/15000
PerceivedRows=15000/15000
Perceiving Validation B...
PerceivedRows=128/15000
PerceivedRows=256/15000
PerceivedRows=384/15000
PerceivedRows=512/15000
PerceivedRows=640/15000
PerceivedRows=768/15000
PerceivedRows=896/15000
PerceivedRows=1024/15000
PerceivedRows=1152/15000
PerceivedRows=1280/15000
PerceivedRows=1408/15000
PerceivedRows=1536/15000
PerceivedRows=1664/15000

PerceivedRows=1792/15000
PerceivedRows=1920/15000
PerceivedRows=2048/15000
PerceivedRows=2176/15000
PerceivedRows=2304/15000
PerceivedRows=2432/15000
PerceivedRows=2560/15000
PerceivedRows=2688/15000
PerceivedRows=2816/15000
PerceivedRows=2944/15000
PerceivedRows=3072/15000
PerceivedRows=3200/15000
PerceivedRows=3328/15000
PerceivedRows=3456/15000
PerceivedRows=3584/15000
PerceivedRows=3712/15000
PerceivedRows=3840/15000
PerceivedRows=3968/15000
PerceivedRows=4096/15000
PerceivedRows=4224/15000
PerceivedRows=4352/15000
PerceivedRows=4480/15000
PerceivedRows=4608/15000
PerceivedRows=4736/15000
PerceivedRows=4864/15000
PerceivedRows=4992/15000
PerceivedRows=5120/15000
PerceivedRows=5248/15000
PerceivedRows=5376/15000
PerceivedRows=5504/15000
PerceivedRows=5632/15000
PerceivedRows=5760/15000
PerceivedRows=5888/15000
PerceivedRows=6016/15000
PerceivedRows=6144/15000
PerceivedRows=6272/15000
PerceivedRows=6400/15000
PerceivedRows=6528/15000
PerceivedRows=6656/15000
PerceivedRows=6784/15000
PerceivedRows=6912/15000
PerceivedRows=7040/15000
PerceivedRows=7168/15000
PerceivedRows=7296/15000
PerceivedRows=7424/15000
PerceivedRows=7552/15000
PerceivedRows=7680/15000
PerceivedRows=7808/15000
PerceivedRows=7936/15000
PerceivedRows=8064/15000
PerceivedRows=8192/15000
PerceivedRows=8320/15000
PerceivedRows=8448/15000
PerceivedRows=8576/15000
PerceivedRows=8704/15000
PerceivedRows=8832/15000

PerceivedRows=8960/15000
PerceivedRows=9088/15000
PerceivedRows=9216/15000
PerceivedRows=9344/15000
PerceivedRows=9472/15000
PerceivedRows=9600/15000
PerceivedRows=9728/15000
PerceivedRows=9856/15000
PerceivedRows=9984/15000
PerceivedRows=10112/15000
PerceivedRows=10240/15000
PerceivedRows=10368/15000
PerceivedRows=10496/15000
PerceivedRows=10624/15000
PerceivedRows=10752/15000
PerceivedRows=10880/15000
PerceivedRows=11008/15000
PerceivedRows=11136/15000
PerceivedRows=11264/15000
PerceivedRows=11392/15000
PerceivedRows=11520/15000
PerceivedRows=11648/15000
PerceivedRows=11776/15000
PerceivedRows=11904/15000
PerceivedRows=12032/15000
PerceivedRows=12160/15000
PerceivedRows=12288/15000
PerceivedRows=12416/15000
PerceivedRows=12544/15000
PerceivedRows=12672/15000
PerceivedRows=12800/15000
PerceivedRows=12928/15000
PerceivedRows=13056/15000
PerceivedRows=13184/15000
PerceivedRows=13312/15000
PerceivedRows=13440/15000
PerceivedRows=13568/15000
PerceivedRows=13696/15000
PerceivedRows=13824/15000
PerceivedRows=13952/15000
PerceivedRows=14080/15000
PerceivedRows=14208/15000
PerceivedRows=14336/15000
PerceivedRows=14464/15000
PerceivedRows=14592/15000
PerceivedRows=14720/15000
PerceivedRows=14848/15000
PerceivedRows=14976/15000
PerceivedRows=15000/15000

A EventAccuracy=1.
A ExactHistoryAccuracy=1.
A ValidHistoryRate=1.
B EventAccuracy=1.
B ExactHistoryAccuracy=1.
B ValidHistoryRate=1.

```

=====
PHASE 3: FREEZE TCCT REASONING CORE
PROSPECTIVE C STILL DOES NOT EXIST
=====
StructuralStates=12
ACCEPT SPLIT=1 STATES=12 PROBE=1 GAIN=0.383994
QuotientStates=3
TCCTSplits=1
TCCTFreezeHash=06ac89bb527745181f0521a53b85f8c438610f0b874b367cbe8aa0f2c675f9\

```

> 71

```

=====
PHASE 4: TRAIN MATCHED NEURAL REASONER
SAME PERCEIVED A/B AS TCCT
PROSPECTIVE C STILL DOES NOT EXIST
=====

```

Starting training.

Optimization Method: ADAM

```

Beta1: 9.00*^-1
Beta2: 9.99*^-1
Epsilon: 1.00*^-5
Gradient Clipping: --
L2 Regularization: --
Learning Rate: 5.00*^-4
Learning Rate Schedule: --
Weight Clipping: --

```

Device: CPU

Batch Size: 128

Batches Per Round: 235

Training Examples: 30000

%	round	batch	examples	inputs	learning	time	time	\
>	current	round	test	current	round	test	current\	
>	round	test						
	/30	/235	processed	/second	rate	elapsed	left	\
>	loss	loss	loss	acc.	acc.	acc.	error	\
>	error	error						
8	3	113	74624	61432	3.32*^-4	2s	23s	\
>	1.28*^+0	1.25*^+0	1.16*^+0	41.2%	44.7%	58.7%		\
>	58.8%	55.3%	41.3%					
17	6	28	153984	36394	4.18*^-4	4s	19s	\
>	1.06*^+0	1.02*^+0	9.63*^-1	63.0%	66.1%	71.0%		\
>	37.0%	33.9%	29.1%					
27	8	235	240640	63508	4.60*^-4	6s	17s	\
>	9.57*^-1	9.34*^-1	8.85*^-1	70.9%	72.2%	75.4%		\

>	29.1%	27.8%	24.6%					
37	11	235	330880	60184	4.81×10^{-4}	8s	14s	\
>	9.06×10^{-1}	8.95×10^{-1}	8.53×10^{-1}	73.9%	74.4%	76.2%		\
>	26.1%	25.6%	23.8%					
47	15	4	421632	5692	4.91×10^{-4}	10s	12s	\
>	8.80×10^{-1}	8.76×10^{-1}	8.42×10^{-1}	74.9%	75.0%	76.2%		\
>	25.1%	25.0%	23.8%					
55	17	120	496640	43140	4.95×10^{-4}	12s	10s	\
>	8.70×10^{-1}	8.66×10^{-1}	8.37×10^{-1}	75.1%	75.2%	76.2%		\
>	24.9%	24.8%	23.8%					
64	20	45	577280	62609	4.97×10^{-4}	14s	8s	\
>	8.60×10^{-1}	8.61×10^{-1}	8.30×10^{-1}	75.5%	75.4%	76.2%		\
>	24.5%	24.6%	23.8%					
74	23	30	665600	59797	4.99×10^{-4}	16s	6s	\
>	8.55×10^{-1}	8.52×10^{-1}	8.29×10^{-1}	75.4%	75.5%	76.2%		\
>	24.6%	24.5%	23.8%					
84	26	23	754944	27798	4.99×10^{-4}	18s	4s	\
>	8.49×10^{-1}	8.48×10^{-1}	8.30×10^{-1}	75.6%	75.6%	76.2%		\
>	24.4%	24.4%	23.8%					
93	28	235	842240	42792	5.00×10^{-4}	20s	1s	\
>	8.48×10^{-1}	8.46×10^{-1}	8.28×10^{-1}	75.6%	75.6%	76.2%		\
>	24.4%	24.4%	23.8%					

ReasonSelectionBestRound=30

ReasonValidationAccuracy=0.762

Starting training.

Optimization Method: ADAM

Beta1: 9.00×10^{-1}

Beta2: 9.99×10^{-1}

Epsilon: 1.00×10^{-5}

Gradient Clipping: --

L2 Regularization: --

Learning Rate: 5.00×10^{-4}

Learning Rate Schedule: --

Weight Clipping: --

Device: CPU

Batch Size: 128

Batches Per Round: 469

Training Examples: 60000

	% round	batch	examples	inputs	learning	time	time	\
>	current		round	current	round	current	round	

	/30	/469	processed	/second	rate	elapsed	left	\
> loss	loss	acc.	acc.	error	error			
8	3	122	135680	47674	4.04* ⁻⁴	2s	26s	\
> 1.19* ⁺⁰	1.10* ⁺⁰	51.2%	60.0%	48.8%	40.0%			
15	5	264	273920	113588	4.70* ⁻⁴	4s	22s	\
> 9.47* ⁻¹	9.40* ⁻¹	71.4%	71.8%	28.6%	28.2%			
23	7	450	417792	49135	4.90* ⁻⁴	6s	20s	\
> 8.86* ⁻¹	8.89* ⁻¹	74.8%	74.8%	25.2%	25.2%			
31	10	201	566016	47847	4.97* ⁻⁴	8s	18s	\
> 8.63* ⁻¹	8.59* ⁻¹	75.5%	75.6%	24.5%	24.4%			
39	12	374	708224	65256	4.99* ⁻⁴	10s	15s	\
> 8.51* ⁻¹	8.52* ⁻¹	75.8%	75.7%	24.2%	24.3%			
46	14	395	830976	52563	5.00* ⁻⁴	12s	14s	\
> 8.45* ⁻¹	8.45* ⁻¹	75.8%	75.8%	24.2%	24.2%			
53	16	397	951296	65662	5.00* ⁻⁴	14s	13s	\
> 8.42* ⁻¹	8.42* ⁻¹	75.8%	75.9%	24.2%	24.1%			
60	18	415	1073664	69513	5.00* ⁻⁴	16s	11s	\
> 8.39* ⁻¹	8.40* ⁻¹	76.0%	75.9%	24.0%	24.1%			
67	21	7	1201536	75751	5.00* ⁻⁴	18s	9s	\
> 8.37* ⁻¹	8.36* ⁻¹	75.9%	75.9%	24.1%	24.1%			
73	22	448	1318016	63281	5.00* ⁻⁴	20s	7s	\
> 8.35* ⁻¹	8.36* ⁻¹	75.9%	75.9%	24.1%	24.1%			
80	24	426	1435264	52247	5.00* ⁻⁴	22s	6s	\
> 8.36* ⁻¹	8.35* ⁻¹	75.9%	75.9%	24.1%	24.1%			
87	26	458	1559424	50638	5.00* ⁻⁴	24s	4s	\
> 8.33* ⁻¹	8.33* ⁻¹	75.9%	75.9%	24.1%	24.1%			
93	29	14	1682688	67141	5.00* ⁻⁴	26s	2s	\
> 8.33* ⁻¹	8.32* ⁻¹	75.9%	75.9%	24.1%	24.1%			

NEURAL REASONER FROZEN

```

=====
GLOBAL FREEZE COMPLETE
SharedPerception=True
TCCTFrozen=True
NeuralReasonerFrozen=True
ProspectiveCExists=False
GlobalFreezeHash=c97190ccf79a605734e12fff20af750040b0acfb7d0aee8c092af4f0cf7a\

> 9a12
=====

```


=====

PHASE 5: PROSPECTIVE C-FIRST OPENING

NO MODEL CHANGES AFTER THIS LINE

=====

ProspectiveRows=6000
Perceiving Prospective C...
PerceivedRows=128/6000
PerceivedRows=256/6000
PerceivedRows=384/6000
PerceivedRows=512/6000
PerceivedRows=640/6000
PerceivedRows=768/6000
PerceivedRows=896/6000
PerceivedRows=1024/6000
PerceivedRows=1152/6000
PerceivedRows=1280/6000
PerceivedRows=1408/6000
PerceivedRows=1536/6000
PerceivedRows=1664/6000
PerceivedRows=1792/6000
PerceivedRows=1920/6000
PerceivedRows=2048/6000
PerceivedRows=2176/6000
PerceivedRows=2304/6000
PerceivedRows=2432/6000
PerceivedRows=2560/6000
PerceivedRows=2688/6000
PerceivedRows=2816/6000
PerceivedRows=2944/6000
PerceivedRows=3072/6000
PerceivedRows=3200/6000
PerceivedRows=3328/6000
PerceivedRows=3456/6000
PerceivedRows=3584/6000
PerceivedRows=3712/6000
PerceivedRows=3840/6000
PerceivedRows=3968/6000
PerceivedRows=4096/6000
PerceivedRows=4224/6000
PerceivedRows=4352/6000
PerceivedRows=4480/6000
PerceivedRows=4608/6000
PerceivedRows=4736/6000
PerceivedRows=4864/6000
PerceivedRows=4992/6000
PerceivedRows=5120/6000
PerceivedRows=5248/6000
PerceivedRows=5376/6000
PerceivedRows=5504/6000
PerceivedRows=5632/6000
PerceivedRows=5760/6000
PerceivedRows=5888/6000
PerceivedRows=6000/6000

C EventAccuracy=1.
C ExactHistoryAccuracy=1.

C ValidHistoryRate=1.

=====

S124-T4 FINAL SUMMARY

=====

TCCTCoreChanged=False
SharedPerception=True
TransformerHistoryClasses=False
PerceptionOutputClasses={1,2,3}
S3QHistories=12
EventBagStates=1
Recent2States=1
StructuralStates=12
QuotientStates=3
TCCTSplits=1
PerceptionValidationAccuracy=1.
AEventAccuracy=1.
BEventAccuracy=1.
CEventAccuracy=1.
CExactHistoryAccuracy=1.
CValidHistoryRate=1.
ReasonValidationAccuracy=0.762
ReasonBestRound=30
OracleTCCT_NLL=0.818524
OracleTCCT_Accuracy=0.759167
SharedPerceptionTCCT_NLL=0.818524
SharedPerceptionTCCT_Accuracy=0.759167
NeuralReasoner_Accuracy=0.759167
MatchedRandomTCCT_NLL=1.658
MatchedRandomTCCT_Accuracy=0.383833
MatchedRandomValidHistoryRate=1.
TCCTMinusNeuralAccuracy=0.
TCCTPenaltyVsOracle=0.
WinnerMargin=0.02
STRICT PROTOCOL PASS=True
REASONING DIAGNOSIS=REASONING_PARITY_WITHIN_PREDECLARED_2_PERCENT_MARGIN
=====

SummaryFile=E:\TCCT_CODEX_HANDOFF_2026-08-13\S97A_ReadoutBaseline_Development\

> \S124_T4_Output\S124_T4_summary.wl

=====

S124-T4 COMPLETE

=====

In [890...

```
ClearAll["Global`*"];
$HistoryLength=0;
Print["====="];
Print["S124-T5 STANDALONE"];
Print["S119B NEURALIZED HIGH-ORDER GENERALIZATION ATTRIBUTION"];
Print["SHARED TRANSFORMER PERCEPTION"];
Print["PAIRWISE TRAINING -> UNSEEN >=3-FACTOR COMBINATIONS"];
Print["====="];
Print["WolframVersion=", $Version];
Print["Date=",DateString[]];
T5Stop[msg_]:= (Print["FATAL: ", msg]; Abort[]);
```

```

publicActionsS119B=Range[0,7];
publicProbesS119B=Range[0,13];
supportScanDepthS119B=8;
localBFSSafetyS119B=128;
preparationBFSSafetyS119B=256;
hiddenBenchmarkSeedS119B=1194701;
t5PerceptionSensorDim=12;
t5PerceptionPosDim=8;
t5PerceptionMicroLen=4;
t5PerceptionScale=2.;
t5PerceptionNoise=0.30;
t5PerceptionDrift=0.12;
t5PerceptionGainJitter=0.12;
t5PerceptionDModel=64;
t5PerceptionHeads=4;
t5PerceptionLayers=2;
t5PerceptionFF=192;
t5PerceptionDropout=0.10;
t5PerceptionLR=0.0003;
t5PerceptionBatch=64;
t5PerceptionRounds=20;
t5PerceptionPatience=5;
t5PerceptionTrainPerClass=1200;
t5PerceptionValPerClass=300;
t5PerceptionPrototypeSeed=9250001;
t5PerceptionTrainSeed=9250002;
t5PerceptionValSeed=9250003;
t5PerceptionNetSeed=9250004;
t5PerceptionQuerySeed=9250005;
t5ReasonDModel=64;
t5ReasonHeads=4;
t5ReasonLayers=2;
t5ReasonFF=192;
t5ReasonDropout=0.10;
t5ReasonPosDim=8;
t5ReasonLR=0.0003;
t5ReasonBatch=64;
t5ReasonRounds=35;
t5ReasonPatience=6;
t5ReasonSplitSeed=9251001;
t5ReasonSelectSeed=9251002;
t5ReasonFinalSeed=9251003;
t5TargetDevice="CPU";
t5PerceptionGate=0.98;
t5TrainingPerceptionGate=0.98;
t5WinnerMargin=0.02;
If[OddQ[t5PerceptionPosDim],T5Stop["perception positional dimension must be even"]];
If[OddQ[t5ReasonPosDim],T5Stop["reason positional dimension must be even"]];
If[Mod[t5PerceptionDModel,t5PerceptionHeads]!=0,T5Stop["perception dModel/head mis"];
If[Mod[t5ReasonDModel,t5ReasonHeads]!=0,T5Stop["reason dModel/head mismatch"]];
t5OutputDirectory=FileNameJoin[{Directory[],"S124_T5_Output"}];
If[!DirectoryQ[t5OutputDirectory],CreateDirectory[t5OutputDirectory]];
Print["PublicActions=",publicActionsS119B];
Print["PublicProbes=",publicProbesS119B];
Print["MaximumTrainingInteractionOrder=2"];
discoveryPolicySpecS119B=<|

```

```

"Actions"->publicActionsS119B,
"Probes"->publicProbesS119B,
"SupportScanDepth"->supportScanDepthS119B,
"FactorRule"->"EqualObservedProbeSupport",
"LocalLearning"->"BFSAtBaselineContext",
"InteractionDiscovery"->"OneCandidateContextFactorAtATime",
"TransitionRepresentation"->"SparseConditionalLocalTable",
"MaximumTrainingInteractionOrder"->2
|>;
frozenDiscoveryPolicyHashS119B=Hash[discoveryPolicySpecS119B,"SHA256","HexString"];
Print["PolicyHash=",frozenDiscoveryPolicyHashS119B];
Print["HIDDEN WORLD DOES NOT EXIST YET."];
BlockRandom[
SeedRandom[t5PerceptionPrototypeSeed];
t5PerceptionBases=Normalize/@RandomVariate[NormalDistribution[0.,1.],{2,t5Perceptio
}];
T5PerceptionPos[pos_Integer]:=
Flatten[
Table[
{
Sin[pos/(10000.^(2.0*k/t5PerceptionPosDim))],
Cos[pos/(10000.^(2.0*k/t5PerceptionPosDim))]}],
{k,0,t5PerceptionPosDim/2-1}
]
];
T5BinarySensory[y_Integer,seed_Integer]:=
BlockRandom[
SeedRandom[seed];
Module[{drift,gain},
drift=RandomReal[{-t5PerceptionDrift,t5PerceptionDrift},t5PerceptionSensorDim];
N[
Table[
gain=1.+RandomReal[{-t5PerceptionGainJitter,t5PerceptionGainJitter}];
Join[
t5PerceptionScale*gain*t5PerceptionBases[[y+1]]+
drift+
RandomVariate[NormalDistribution[0.,t5PerceptionNoise],t5PerceptionSensorDim],
T5PerceptionPos[m]
],
{m,1,t5PerceptionMicroLen}
]
]
];
t5PerceptionInputDim=t5PerceptionSensorDim+t5PerceptionPosDim;
T5MakePerceptionData[n_Integer,seed_Integer]:=
BlockRandom[
SeedRandom[seed];
RandomSample[
Flatten[
Table[
Table[
<|
"Input"->T5BinarySensory[y,seed+100000*y+i],
"Label"->y

```

```

|>,
{i,1,n}
],
{y,0,1}
],
1
]
]
];
T5PerceptionRules[data_List]:=(#["Input"]->#["Label"]&)/@data;
T5PerceptionBlock[modelDim_Integer,heads_Integer,ffDim_Integer,drop_?NumericQ]:=
Module[{hd},
hd=Quotient[modelDim,heads];
NetGraph[
<|
"LN1"->NormalizationLayer[2,"Same"],
"Q"->NetMapOperator[LinearLayer[{heads,hd}]],
"K"->NetMapOperator[LinearLayer[{heads,hd}]],
"V"->NetMapOperator[LinearLayer[{heads,hd}]],
"Attention"->AttentionLayer["Dot","MultiHead"->True,"Mask"->"Causal","ScoreRescaling"],
"Merge"->NetMapOperator[NetChain[{FlattenLayer[],LinearLayer[modelDim]}]],
"ADrop"->DropoutLayer[drop],
"Res1"->ThreadingLayer[Plus],
"LN2"->NormalizationLayer[2,"Same"],
"FF"->NetMapOperator[NetChain[{LinearLayer[ffDim],ElementwiseLayer[Ramp],DropoutLayer[drop]}]],
"FDrop"->DropoutLayer[drop],
"Res2"->ThreadingLayer[Plus]
|>,
{
NetPort["Input"]->"LN1",
"LN1"->"Q",
"LN1"->"K",
"LN1"->"V",
"Q"->NetPort["Attention","Query"],
"K"->NetPort["Attention","Key"],
"V"->NetPort["Attention","Value"],
"Attention"->"Merge",
"Merge"->"ADrop",
{NetPort["Input"],"ADrop"}->"Res1",
"Res1"->"LN2",
"LN2"->"FF",
"FF"->"FDrop",
{"Res1","FDrop"}->"Res2"
},
"Input"->{"Varying",modelDim}
]
];
T5PerceptionNet[]:=
Module[{blocks},
blocks=Table[
T5PerceptionBlock[t5PerceptionDModel,t5PerceptionHeads,t5PerceptionFF,t5PerceptionDrop]
{t5PerceptionLayers}
];
NetChain[
Join[
{NetMapOperator[LinearLayer[t5PerceptionDModel]]},

```

```

blocks,
{
NormalizationLayer[2, "Same"],
SequenceLastLayer[],
LinearLayer[2],
SoftmaxLayer[]
}
],
"Input"->{"Varying", t5PerceptionInputDim},
"Output"->NetDecoder[{"Class", {0, 1}}]
]
];
Print["====="];
Print["PHASE 1: TRAIN SHARED BINARY PERCEPTION"];
Print["HIDDEN WORLD STILL DOES NOT EXIST"];
Print["====="];
t5PerceptionTrain=T5MakePerceptionData[t5PerceptionTrainPerClass, t5PerceptionTrainSeed];
t5PerceptionVal=T5MakePerceptionData[t5PerceptionValPerClass, t5PerceptionValSeed];
t5PerceptionTrainRules=T5PerceptionRules[t5PerceptionTrain];
t5PerceptionValRules=T5PerceptionRules[t5PerceptionVal];
SeedRandom[t5PerceptionNetSeed];
t5PerceptionNet0=NetInitialize[t5PerceptionNet[]];
t5PerceptionTraining=
NetTrain[
t5PerceptionNet0,
t5PerceptionTrainRules,
All,
ValidationSet->t5PerceptionValRules,
MaxTrainingRounds->t5PerceptionRounds,
BatchSize->t5PerceptionBatch,
LearningRate->t5PerceptionLR,
Method->"ADAM",
TrainingProgressMeasurements->{"Accuracy", "ErrorRate"},
TrainingStoppingCriterion-><|"Criterion"->"Loss", "Patience"->t5PerceptionPatience|>
TrainingProgressReporting->"Print",
TargetDevice->t5TargetDevice,
RandomSeeding->t5PerceptionNetSeed
];
t5FrozenPerception=t5PerceptionTraining["TrainedNet"];
t5PerceptionValidationAccuracy=
NetMeasurements[t5FrozenPerception, t5PerceptionValRules, "Accuracy", BatchSize->t5PerceptionBatch];
Print["PerceptionValidationAccuracy=", t5PerceptionValidationAccuracy];
If[t5PerceptionValidationAccuracy<t5PerceptionGate, T5Stop["shared perception failed"],
Export[
FileNameJoin[{t5OutputDirectory, "S124_T5_PERCEPTION_FROZEN_BEFORE_WORLD.wlnet"}],
t5FrozenPerception
];
Clear[t5PerceptionTrain, t5PerceptionVal, t5PerceptionTrainRules, t5PerceptionValRules];
Print["SHARED PERCEPTION FROZEN"];
Print["====="];
Print["PHASE 2: GENERATE ORIGINAL S119B HIDDEN WORLD"];
Print["====="];
hiddenFactorSizesS119B=
BlockRandom[
SeedRandom[hiddenBenchmarkSeedS119B];
RandomSample[{2, 3, 4, 5}]
];

```

```

];
hiddenTrueFactorCountS119B=Length[hiddenFactorSizesS119B];
hiddenSourceFactorsS119B=Flatten[Position[hiddenFactorSizesS119B,2|3]];
hiddenTargetFactorsS119B=Flatten[Position[hiddenFactorSizesS119B,4|5]];
hiddenSourceFactorsS119B=
BlockRandom[
SeedRandom[hiddenBenchmarkSeedS119B+1];
RandomSample[hiddenSourceFactorsS119B]
];
hiddenTargetFactorsS119B=
BlockRandom[
SeedRandom[hiddenBenchmarkSeedS119B+2];
RandomSample[hiddenTargetFactorsS119B]
];
hiddenCouplingPairsS119B=Thread[hiddenSourceFactorsS119B->hiddenTargetFactorsS119B]
hiddenActionRolePoolS119B=
Flatten[
Table[
Module[{parentForPlus},
parentForPlus=Lookup[Association[Reverse/@hiddenCouplingPairsS119B],factorIndex,0];
{{factorIndex,1,parentForPlus},{factorIndex,-1,0}}
],
{factorIndex,1,hiddenTrueFactorCountS119B}
],
1
];
hiddenActionRolePermutationS119B=
BlockRandom[
SeedRandom[hiddenBenchmarkSeedS119B+3];
RandomSample[hiddenActionRolePoolS119B]
];
hiddenActionRolesS119B=AssociationThread[publicActionsS119B->hiddenActionRolePermut
hiddenProbeRolePoolS119B=
Flatten[
Table[
Table[
{factorIndex,value},
{value,0,hiddenFactorSizesS119B[[factorIndex]]-1}
],
{factorIndex,1,hiddenTrueFactorCountS119B}
],
1
];
hiddenProbeRolePermutationS119B=
BlockRandom[
SeedRandom[hiddenBenchmarkSeedS119B+4];
RandomSample[hiddenProbeRolePoolS119B]
];
hiddenProbeRolesS119B=AssociationThread[publicProbesS119B->hiddenProbeRolePermutati
hiddenStartStateS119B=ConstantArray[0,hiddenTrueFactorCountS119B];
S119BEnvStep[state_List,action_Integer]:=
Module[{role,targetFactor,direction,parentFactor,targetSize,delta,nextState},
role=Lookup[hiddenActionRolesS119B,action,Missing["UnknownAction"]];
If[
!ListQ[role],
Missing["UnknownAction"],

```

```

targetFactor=role[[1]];
direction=role[[2]];
parentFactor=role[[3]];
targetSize=hiddenFactorSizesS119B[[targetFactor]];
delta=Which[
parentFactor==0,direction,
direction==1&&state[[parentFactor]]==0,1,
direction==1&&state[[parentFactor]]!=0,2,
True,direction
];
nextState=state;
nextState[[targetFactor]]=Mod[nextState[[targetFactor]]+delta,targetSize];
nextState
];
];
S119BEnvStateAfter[seq_List]:=Fold[S119BEnvStep,hiddenStartStateS119B,seq];
S119BEnvRawOutput[seq_List,probe_Integer]:=
Module[{state,role,factorIndex,value},
state=S119BEnvStateAfter[seq];
role=Lookup[hiddenProbeRolesS119B,probe,Missing["UnknownProbe"]];
If[
!ListQ[state]||!ListQ[role],
Missing["EnvironmentFailure"],
factorIndex=role[[1]];
value=role[[2]];
Boole[state[[factorIndex]]==value]
];
];
hiddenIndependentMinusActionByFactorS119B=
Association[
Table[
factorIndex->
SelectFirst[
publicActionsS119B,
Function[action,Lookup[hiddenActionRolesS119B,action]=={factorIndex,-1,0}]
],
{factorIndex,1,hiddenTrueFactorCountS119B}
];
];
S119BEvaluatorCanonicalSequence[tuple_List]:=
Flatten[
Table[
ConstantArray[
Lookup[hiddenIndependentMinusActionByFactorS119B,factorIndex],
Mod[-tuple[[factorIndex]],hiddenFactorSizesS119B[[factorIndex]]]
],
{factorIndex,1,hiddenTrueFactorCountS119B}
],
1
];
allTrueJointTuplesS119B=
Tuples[Map[Range[0,#-1]&,hiddenFactorSizesS119B]];
trueJointStateCountS119B=Length[allTrueJointTuplesS119B];
prospectiveHighOrderTuplesS119B=
Select[allTrueJointTuplesS119B,Count[#,Except[0]]>=3&];
prospectiveHighOrderHoldoutS119B=

```



```

Table[
<|"TupleEvaluatorOnly"->tuple,"Sequence"->S119BEvaluatorCanonicalSequence[tuple]|>,
{tuple,prospectiveHighOrderTuplesS119B}
];
Print["NEW HIDDEN SPARSELY COUPLED WORLD GENERATED"];
Print["TrueJointStates=",trueJointStateCountS119B];
Print["ProspectiveHighOrderStates=",Length[prospectiveHighOrderHoldoutS119B]];
Print["HIGH-ORDER OUTPUTS SEALED"];
S119BSeqKey[seq_List]:=ToString[InputForm[seq]];
S119BQueryKey[seq_List,probe_Integer]:=ToString[InputForm[{seq,probe}]];
T5QuerySensorSeed[seq_List,probe_Integer]:=
1+Mod[Hash[{seq,probe,t5PerceptionQuerySeed},"CRC32"],2000000000];
membershipCacheS119B=<||>;
queriedSequenceRegistryS119B=<||>;
membershipUniqueCountS119B=0;
t5MembershipRows={};
S119BMembershipY[seq_List,probe_Integer,source_String:"SupportScan"]:=
Module[{key,seqKey,rawY,perceivedY},
key=S119BQueryKey[seq,probe];
seqKey=S119BSeqKey[seq];
If[
KeyExistsQ[membershipCacheS119B,key],
Lookup[membershipCacheS119B,key],
rawY=S119BEnvRawOutput[seq,probe];
If[!MemberQ[{0,1},rawY],T5Stop["environment returned non-binary output"]];
perceivedY=t5FrozenPerception[
T5BinarySensory[rawY,T5QuerySensorSeed[seq,probe]]
];
If[!MemberQ[{0,1},perceivedY],T5Stop["perception returned invalid class"]];
AssociateTo[membershipCacheS119B,key->perceivedY];
AssociateTo[queriedSequenceRegistryS119B,seqKey->seq];
membershipUniqueCountS119B++;
AppendTo[
t5MembershipRows,
<|
"Seq"->seq,
"Probe"->probe,
"Y"->perceivedY,
"RawYEvaluatorOnly"->rawY,
"Source"->source
|>
];
perceivedY
];
];
S119BObserveSignature[seq_List,probes_List,source_String]:=
Table[S119BMembershipY[seq,probe,source],{probe,probes}];
S119BSignatureKey[sig_List]:=ToString[InputForm[sig]];
Print["====="];
Print["PHASE 3: ORIGINAL S119B FACTOR SUPPORT DISCOVERY"];
Print["HIGH-ORDER OUTPUTS STILL SEALED"];
Print["====="];
baselineSignatureS119B=S119BObserveSignature[{},publicProbesS119B,"SupportScan"];
actionSupportRowsS119B=
Table[
Module[{scanSignatures,changedPositions,support},

```

```

scanSignatures=
Table[
S119BObserveSignature[ConstantArray[action,repetition],publicProbesS119B,"SupportSc
{repetition,1,supportScanDepthS119B}
];
changedPositions=
Select[
Range[Length[publicProbesS119B]],
Function[position,
AnyTrue[
scanSignatures,
Function[signature,signature[[position]]!=baselineSignatureS119B[[position]]]
]
]
];
support=publicProbesS119B[[changedPositions]];
<|"Action"->action,"ObservedProbeSupport"->Sort[support],"SupportSize"->Length[supp
],
{action,publicActionsS119B}
];
If[AnyTrue[actionSupportRowsS119B,Length[#["ObservedProbeSupport"]]==0&],T5Stop["em
supportGroupsS119B=
Values[
GroupBy[
actionSupportRowsS119B,
ToString[InputForm[#["ObservedProbeSupport"]]]&
]
];
factorDescriptorsS119B=
Table[
<|
"Probes"->Sort[supportGroupsS119B[[g,1]]["ObservedProbeSupport"]],
"Actions"->Sort[(#["Action"]&)/@supportGroupsS119B[[g]]]
|>,
{g,1,Length[supportGroupsS119B]}
];
factorDescriptorsS119B=SortBy[factorDescriptorsS119B,First[#["Probes"]]&];
factorDescriptorsS119B=
Table[
Join[factorDescriptorsS119B[[i]],<|"LearnedFactor"->i|>],
{i,1,Length[factorDescriptorsS119B]}
];
inferredFactorCountS119B=Length[factorDescriptorsS119B];
learnedProbeGroupsS119B=(#["Probes"]&)/@factorDescriptorsS119B;
learnedActionGroupsS119B=(#["Actions"]&)/@factorDescriptorsS119B;
Print["InferredFactors=",inferredFactorCountS119B];
Print["ProbeGroupSizes=",Length/@learnedProbeGroupsS119B];
Print["ActionGroupSizes=",Length/@learnedActionGroupsS119B];
If[inferredFactorCountS119B!=4,T5Stop["S119B failed to recover four factors"]];
S119BLocalTransitionKey[state_Integer,action_Integer]:=
ToString[InputForm[{state,action}]];
S119BLearnLocalFactor[descriptor_Association]:=
Module[
{probes,actions,startSignature,signatureToState,stateSignatures,representatives,tra
probes=descriptor["Probes"];
actions=descriptor["Actions"];

```

```

startSignature=S119BObserveSignature[{},probes,"LocalBFS"];
signatureToState=<|S119BSignatureKey[startSignature]->1|>;
stateSignatures=<|1->startSignature|>;
representatives=<|1->{}|>;
transitions=<||>;
queue={1};
guard=0;
While[
Length[queue]>0,
guard++;
If[guard>localBFSSafetyS119B,T5Stop["local BFS safety exceeded"]];
currentState=First[queue];
queue=Rest[queue];
currentRep=Lookup[representatives,currentState];
Do[
nextSeq=Append[currentRep,action];
nextSignature=S119BObserveSignature[nextSeq,probes,"LocalBFS"];
nextKey=S119BSignatureKey[nextSignature];
If[
KeyExistsQ[signatureToState,nextKey],
nextState=Lookup[signatureToState,nextKey],
newState=Length[stateSignatures]+1;
nextState=newState;
AssociateTo[signatureToState,nextKey->newState];
AssociateTo[stateSignatures,newState->nextSignature];
AssociateTo[representatives,newState->nextSeq];
AppendTo[queue,newState]
];
AssociateTo[
transitions,
S119BLocalTransitionKey[currentState,action]->nextState
],
{action,actions}
];
<|
"LearnedFactor"->descriptor["LearnedFactor"],
"Probes"->probes,
"Actions"->actions,
"StateCount"->Length[stateSignatures],
"StateSignatures"->stateSignatures,
"RepresentativeByState"->representatives,
"TransitionsAtBaseline"->transitions,
"StartState"->1,
"ObservedOneHot"->AllTrue[Values[stateSignatures],Total[#]==1&]
|>
];
learnedFactorsS119B=S119BLearnLocalFactor/@factorDescriptorsS119B;
learnedLocalStateCountsS119B=(#[["StateCount"]&])/@learnedFactorsS119B;
Print["LocalStateCounts=",learnedLocalStateCountsS119B];
Print["StoredLocalStates=",Total[learnedLocalStateCountsS119B]];
Print["ProductCapacity=",Times@@learnedLocalStateCountsS119B];
actionToLearnedFactorS119B=
Association[
Flatten[
Table[

```

```

Table[action->factorIndex,{action,learnedFactorsS119B[[factorIndex]]["Actions"]}],
{factorIndex,1,Length[learnedFactorsS119B]}
]
];
probeToLearnedFactorS119B=
Association[
Flatten[
Table[
Table[probe->factorIndex,{probe,learnedFactorsS119B[[factorIndex]]["Probes"]}],
{factorIndex,1,Length[learnedFactorsS119B]}
]
]
];
S119BLocalStateFromSignature[factorIndex_Integer,signature_List]:=
SelectFirst[
Range[learnedFactorsS119B[[factorIndex]]["StateCount"]],
Function[localState,
Lookup[learnedFactorsS119B[[factorIndex]]["StateSignatures"],localState]==signature
],
Missing["UnknownLocalSignature"]
];
S119BPrepareFactorState[baseSequence_List,factorIndex_Integer,desiredLocalState_Int
Module[{result,probes,actions,desiredSignature},
probes=learnedFactorsS119B[[factorIndex]]["Probes"];
actions=learnedFactorsS119B[[factorIndex]]["Actions"];
desiredSignature=Lookup[learnedFactorsS119B[[factorIndex]]["StateSignatures"],desiredLocalState];
result=
Catch[
Module[{startSignature,queue,visited,current,currentSequence,currentSignature,nextSignature},
startSignature=S119BObserveSignature[baseSequence,probes,source];
If[startSignature==desiredSignature,Throw[baseSequence,"S119B_PREPARED"]];
queue={{baseSequence,startSignature}};
visited=<|S119BSignatureKey[startSignature]->True|>;
guard=0;
While[
Length[queue]>0,
guard++;
If[guard>preparationBFSSafetyS119B,Throw[$Failed,"S119B_PREPARED"]];
current=First[queue];
queue=Rest[queue];
currentSequence=current[[1]];
currentSignature=current[[2]];
Do[
nextSequence=Append[currentSequence,action];
nextSignature=S119BObserveSignature[nextSequence,probes,source];
If[nextSignature==desiredSignature,Throw[nextSequence,"S119B_PREPARED"]];
nextKey=S119BSignatureKey[nextSignature];
If[
!KeyExistsQ[visited,nextKey],
AssociateTo[visited,nextKey->True];
AppendTo[queue,{nextSequence,nextSignature}]
],
{action,actions}
]
];
];

```

```

$Failed
],
"S119B_PREPARED"
];
If[result===$Failed,T5Stop["could not prepare requested local factor state"],result
];
S119BPrepareContext[assignments_List,source_String]:=
Module[{seq={}},
Do[
seq=S119BPrepareFactorState[seq,assignment[[1]],assignment[[2]],source],
{assignment,assignments}
];
seq
];
Print["====="];
Print["PHASE 4: ACTIVE PAIRWISE INTERACTION DISCOVERY"];
Print["HIGH-ORDER OUTPUTS STILL SEALED"];
Print["====="];
interactionScanRowsS119B=
Flatten[
Table[
Module[{targetFactor,seq,nextSeq,targetSignature,destinationState},
targetFactor=Lookup[actionToLearnedFactorS119B,action];
seq=
S119BPrepareContext[
{{targetFactor,targetState},{candidateParent,parentState}},
"InteractionScan"
];
nextSeq=Append[seq,action];
targetSignature=
S119BObserveSignature[
nextSeq,
learnedFactorsS119B[[targetFactor]]["Probes"],
"InteractionScan"
];
destinationState=
S119BLocalStateFromSignature[targetFactor,targetSignature];
If[!IntegerQ[destinationState],T5Stop["interaction scan reached unknown state"]];
<|
"Action"->action,
"TargetFactor"->targetFactor,
"CandidateParent"->candidateParent,
"TargetState"->targetState,
"ParentState"->parentState,
"DestinationState"->destinationState
|>
],
{action,publicActionsS119B},
{candidateParent,DeleteCases[Range[inferredFactorCountS119B],Lookup[actionToLearned
{targetState,Range[learnedFactorsS119B[[Lookup[actionToLearnedFactorS119B,action]]]
{parentState,Range[learnedFactorsS119B[[candidateParent]]["StateCount"]]]}
],
3
];
S119BDependencyDetectedQ[action_Integer,candidateParent_Integer]:=
Module[{rows,targetFactor,targetStateCount},

```

```

rows=
Select[
interactionScanRowsS119B,
#[ "Action" ]===action&&#[ "CandidateParent" ]===candidateParent&
];
targetFactor=Lookup[actionToLearnedFactorS119B,action];
targetStateCount=learnedFactorsS119B[[targetFactor]][ "StateCount" ];
AnyTrue[
Range[targetStateCount],
Function[targetState,
Length[
DeleteDuplicates[
(#[ "DestinationState" ]&)/@
Select[rows,#[ "TargetState" ]===targetState&]
]
]>1
]
];
learnedParentsByActionS119B=
Association[
Table[
action->
Sort[
Select[
DeleteCases[
Range[inferredFactorCountS119B],
Lookup[actionToLearnedFactorS119B,action]
],
S119BDependencyDetectedQ[action,#]&
],
],
{action,publicActionsS119B}
]
];
learnedInteractionEdgesS119B=
DeleteDuplicates[
Flatten[
Table[
Module[{targetFactor},
targetFactor=Lookup[actionToLearnedFactorS119B,action];
Table[parent->targetFactor,{parent,Lookup[learnedParentsByActionS119B,action]}]
],
{action,publicActionsS119B}
],
1
]
];
Print["LearnedParentsByAction=",learnedParentsByActionS119B];
Print["LearnedInteractionEdges=",learnedInteractionEdgesS119B];
S119BConditionalTransitionKey[action_Integer,targetState_Integer,parentStates_List]
ToString[InputForm[{action,targetState,parentStates}]]];
conditionalTransitionsS119B=<| |>;
conditionalTransitionRowsS119B={};
Do[
Module[{targetFactor,parentFactors,parentStateTuples,destination,seq,nextSeq,target

```

```

targetFactor=Lookup[actionToLearnedFactorS119B,action];
parentFactors=Lookup[learnedParentsByActionS119B,action];
If[
Length[parentFactors]==0,
Do[
destination=
Lookup[
learnedFactorsS119B[[targetFactor]]["TransitionsAtBaseline"],
S119BLocalTransitionKey[targetState,action]
];
AssociateTo[
conditionalTransitionsS119B,
S119BConditionalTransitionKey[action,targetState,{}]->destination
];
AppendTo[
conditionalTransitionRowsS119B,
<|
"Action"->action,
"TargetFactor"->targetFactor,
"ParentFactors"->{},
"TargetState"->targetState,
"ParentStates"->{},
"DestinationState"->destination
|>
],
{targetState,1,learnedFactorsS119B[[targetFactor]]["StateCount"]
},
parentStateTuples=
Tuples[
Table[
Range[learnedFactorsS119B[[parentFactor]]["StateCount"]
],
{parentFactor,parentFactors}
];
Do[
Do[
assignments=
Join[
{{targetFactor,targetState}},
MapThread[List,{parentFactors,parentStates}]
];
seq=S119BPrepareContext[assignments,"ConditionalTable"];
nextSeq=Append[seq,action];
targetSignature=
S119BObserveSignature[
nextSeq,
learnedFactorsS119B[[targetFactor]]["Probes"],
"ConditionalTable"
];
destination=
S119BLocalStateFromSignature[targetFactor,targetSignature];
If[!IntegerQ[destination],T5Stop["conditional table reached unknown state"]];
AssociateTo[
conditionalTransitionsS119B,
S119BConditionalTransitionKey[action,targetState,parentStates]->destination
];

```

```

AppendTo[
conditionalTransitionRowsS119B,
<|
"Action"->action,
"TargetFactor"->targetFactor,
"ParentFactors"->parentFactors,
"TargetState"->targetState,
"ParentStates"->parentStates,
"DestinationState"->destination
|>
],
{parentStates,parentStateTuples}
],
{targetState,1,learnedFactorsS119B[[targetFactor]][["StateCount"]]}
]
],
{action,publicActionsS119B}
];
Print["ConditionalTransitionCells=",Length[conditionalTransitionsS119B]];
factorizedStartStateS119B=(#["StartState"]&)/@learnedFactorsS119B;
S119BFactorizedStep[jointState_List,action_Integer]:=
Module[{targetFactor,parentFactors,targetState,parentStates,destination,nextState},
targetFactor=Lookup[actionToLearnedFactorS119B,action,Missing["UnknownAction"]];
If[
!IntegerQ[targetFactor],
Missing["UnknownAction"],
parentFactors=Lookup[learnedParentsByActionS119B,action,{}];
targetState=jointState[[targetFactor]];
parentStates=If[Length[parentFactors]==0,{},jointState[[parentFactors]]];
destination=
Lookup[
conditionalTransitionsS119B,
S119BConditionalTransitionKey[action,targetState,parentStates],
Missing["UnknownConditionalTransition"]
];
If[
!IntegerQ[destination],
Missing["UnknownConditionalTransition"],
nextState=jointState;
nextState[[targetFactor]]=destination;
nextState
]
]
];
S119BFactorizedStateAfter[seq_List]:=
Fold[
Function[{state,action},
If[ListQ[state],S119BFactorizedStep[state,action],state]
],
factorizedStartStateS119B,
seq
];
S119BFactorizedOutput[seq_List,probe_Integer]:=
Module[{state,factorIndex,localState,localSignature,probePosition},
state=S119BFactorizedStateAfter[seq];

```



```

factorIndex=Lookup[probeToLearnFactorS119B,probe,Missing["UnknownProbe"]];
If[
!ListQ[state]||!IntegerQ[factorIndex],
Missing["ModelFailure"],
localState=state[[factorIndex]];
localSignature=
Lookup[
learnedFactorsS119B[[factorIndex]]["StateSignatures"],
localState,
Missing["UnknownState"]
];
probePosition=
FirstPosition[
learnedFactorsS119B[[factorIndex]]["Probes"],
probe,
Missing["UnknownProbe"]
];
If[
!ListQ[localSignature]||MissingQ[probePosition],
Missing["ModelFailure"],
localSignature[[probePosition[[1]]]]
]
];
highOrderTouchedBeforeFreezeS119B=
Count[
prospectiveHighOrderHoldoutS119B,
row_Association/;
KeyExistsQ[
queriedSequenceRegistryS119B,
S119BSeqKey[row["Sequence"]]
]
];
membershipQueriesBeforeFreezeS119B=membershipUniqueCountS119B;
policyHashUnchangedS119B=
Hash[discoveryPolicySpecS119B,"SHA256","HexString"]===frozenDiscoveryPolicyHashS119B
If[!TrueQ[policyHashUnchangedS119B],T5Stop["frozen S119B policy changed"]];
frozenModelHashS119B=
Hash[
{
factorDescriptorsS119B,
learnedFactorsS119B,
actionToLearnFactorS119B,
learnedParentsByActionS119B,
conditionalTransitionsS119B,
factorizedStartStateS119B
},
"SHA256",
"HexString"
];
Print["====="];
Print["S119B TCCT MODEL FROZEN"];
Print["Factors=",inferredFactorCountS119B];
Print["LocalStateCounts=",learnedLocalStateCountsS119B];
Print["LearnedInteractionEdges=",learnedInteractionEdgesS119B];
Print["ConditionalTransitionCells=",Length[conditionalTransitionsS119B]];

```

```

Print["MembershipQueries=",membershipQueriesBeforeFreezeS119B];
Print["HighOrderHoldoutTouched=",highOrderTouchedBeforeFreezeS119B,"/",Length[prosp
Print["TCCTFreezeHash=",frozenModelHashS119B];
Print["====="];
If[highOrderTouchedBeforeFreezeS119B!=0,T5Stop["high-order holdout sequence leakag
t5TrainingPerceptionAccuracy=
N[
Mean[
Boole[#["Y"]==#["RawEvaluatorOnly"]]&/@t5MembershipRows
]
];
t5TrainingUniqueSequences=
DeleteDuplicatesBy[
t5MembershipRows,
S119BSeqKey[#["Seq"]]&
];
t5MaximumTrainingInteractionOrder=
Max[
Count[S119BEnvStateAfter[#["Seq"]],Except[0]]&/@t5TrainingUniqueSequences
];
Print["TrainingPerceptionAccuracy=",t5TrainingPerceptionAccuracy];
Print["MaximumObservedTrainingInteractionOrder=",t5MaximumTrainingInteractionOrder]
If[t5TrainingPerceptionAccuracy<t5TrainingPerceptionGate,T5Stop["shared perception
If[t5MaximumTrainingInteractionOrder>2,T5Stop["training exceeded pairwise interacti
Put[
<|
"Factors"->factorDescriptorsS119B,
"LearnedFactors"->learnedFactorsS119B,
"Parents"->learnedParentsByActionS119B,
"ConditionalTransitions"->conditionalTransitionsS119B,
"StartState"->factorizedStartStateS119B,
"Hash"->frozenModelHashS119B
|>,
FileNameJoin[{t5OutputDirectory,"S124_T5_TCCT_FROZEN_BEFORE_HIGHORDER.wl"}]
];
Print["====="];
Print["PHASE 5: TRAIN MATCHED NEURAL TRANSFORMER REASONER"];
Print["USES EXACT SAME MEMBERSHIP DATA ACQUIRED BY S119B"];
Print["HIGH-ORDER OUTPUTS STILL SEALED"];
Print["====="];
T5ReasonPos[pos_Integer]:=
Flatten[
Table[
{
Sin[pos/(10000.^(2.0*k/t5ReasonPosDim))],
Cos[pos/(10000.^(2.0*k/t5ReasonPosDim))],
},
{k,0,t5ReasonPosDim/2-1}
]
];
t5ReasonMaxSeqLen=
Max[
Length/@Join[
Lookup[t5MembershipRows,"Seq"],
Lookup[prospectiveHighOrderHoldoutS119B,"Sequence"]
]

```

```

];
t5ReasonInputDim=8+14+3+t5ReasonPosDim;
T5ReasonActionToken[action_Integer,pos_Integer]:=
Join[
UnitVector[8,action+1],
ConstantArray[0.,14],
{1.,0.,0.},
T5ReasonPos[pos]
];
T5ReasonPadToken[pos_Integer]:=
Join[
ConstantArray[0.,8],
ConstantArray[0.,14],
{0.,1.,0.},
T5ReasonPos[pos]
];
T5ReasonProbeToken[probe_Integer,pos_Integer]:=
Join[
ConstantArray[0.,8],
UnitVector[14,probe+1],
{0.,0.,1.},
T5ReasonPos[pos]
];
T5ReasonInput[seq_List,probe_Integer]:=
Module[{padded},
padded=PadRight[seq,t5ReasonMaxSeqLen,-1];
N@Join[
MapIndexed[
If[
#1== -1,
{T5ReasonPadToken[First[#2]]},
{T5ReasonActionToken[#1,First[#2]]}
]&,
padded
],
{
T5ReasonProbeToken[probe,t5ReasonMaxSeqLen+1]
}
]
];
T5ReasonRules[rows_List]:=
(T5ReasonInput[#[ "Seq"],#[ "Probe"]]->#[ "Y"]&)/@rows;
T5ReasonBlock[modelDim_Integer,heads_Integer,ffDim_Integer,drop_?NumericQ]:=
Module[{hd},
hd=Quotient[modelDim,heads];
NetGraph[
<|
"LN1"->NormalizationLayer[2,"Same"],
"Q"->NetMapOperator[LinearLayer[{heads,hd}]],
"K"->NetMapOperator[LinearLayer[{heads,hd}]],
"V"->NetMapOperator[LinearLayer[{heads,hd}]],
"Attention"->AttentionLayer["Dot","MultiHead"->True,"Mask"->"Causal","ScoreRescaling"],
"Merge"->NetMapOperator[NetChain[{FlattenLayer[],LinearLayer[modelDim]}]],
"ADrop"->DropoutLayer[drop],
"Res1"->ThreadingLayer[Plus],
"LN2"->NormalizationLayer[2,"Same"],

```

```

"FF"->NetMapOperator[NetChain[{LinearLayer[ffDim],ElementwiseLayer[Ramp],DropoutLayer[drop],
"FDrop"->DropoutLayer[drop],
"Res2"->ThreadingLayer[Plus]
|>,
{
NetPort["Input"]->"LN1",
"LN1"->"Q",
"LN1"->"K",
"LN1"->"V",
"Q"->NetPort["Attention","Query"],
"K"->NetPort["Attention","Key"],
"V"->NetPort["Attention","Value"],
"Attention"->"Merge",
"Merge"->"ADrop",
{NetPort["Input"],"ADrop"}->"Res1",
"Res1"->"LN2",
"LN2"->"FF",
"FF"->"FDrop",
{"Res1","FDrop"}->"Res2"
}],
"Input"->{"Varying",modelDim}
]
];
T5ReasonNet[]:=
Module[{blocks},
blocks=
Table[
T5ReasonBlock[t5ReasonDModel,t5ReasonHeads,t5ReasonFF,t5ReasonDropout],
{t5ReasonLayers}
];
NetChain[
Join[
{NetMapOperator[LinearLayer[t5ReasonDModel]]},
blocks,
{
NormalizationLayer[2,"Same"],
SequenceLastLayer[],
LinearLayer[2],
SoftmaxLayer[]
}
],
"Input"->{"Varying",t5ReasonInputDim},
"Output"->NetDecoder[{"Class",{0,1}}]
]
];
t5RowsBySequence=
GroupBy[t5MembershipRows,S119BSeqKey[#[["Seq"]]&]];
t5SequenceKeys=Keys[t5RowsBySequence];
t5ShuffledSequenceKeys=
BlockRandom[
SeedRandom[t5ReasonSplitSeed];
RandomSample[t5SequenceKeys]
];
t5ValSequenceCount=Max[1,Round[0.20*Length[t5ShuffledSequenceKeys]]];
t5ValSequenceKeys=Take[t5ShuffledSequenceKeys,t5ValSequenceCount];
t5TrainSequenceKeys=Drop[t5ShuffledSequenceKeys,t5ValSequenceCount];

```

```

t5ReasonTrainRows=Flatten[Lookup[t5RowsBySequence,t5TrainSequenceKeys],1];
t5ReasonValRows=Flatten[Lookup[t5RowsBySequence,t5ValSequenceKeys],1];
t5ReasonTrainRules=T5ReasonRules[t5ReasonTrainRows];
t5ReasonValRules=T5ReasonRules[t5ReasonValRows];
Print["ReasonTrainingRows=",Length[t5ReasonTrainRows]];
Print["ReasonValidationRows=",Length[t5ReasonValRows]];
Print["ReasonMaxSequenceLength=",t5ReasonMaxSeqLen];
SeedRandom[t5ReasonSelectSeed];
t5ReasonSelectNet0=NetInitialize[T5ReasonNet[]];
t5ReasonSelection=
NetTrain[
t5ReasonSelectNet0,
t5ReasonTrainRules,
All,
ValidationSet->t5ReasonValRules,
MaxTrainingRounds->t5ReasonRounds,
BatchSize->t5ReasonBatch,
LearningRate->t5ReasonLR,
Method->"ADAM",
TrainingProgressMeasurements->{"Accuracy","ErrorRate"},
TrainingStoppingCriterion-><|"Criterion"->"Loss","Patience"->t5ReasonPatience|>,
TrainingProgressReporting->"Print",
TargetDevice->t5TargetDevice,
RandomSeeding->t5ReasonSelectSeed
];
t5ReasonBestRound=t5ReasonSelection["BestValidationRound"];
t5ReasonValidationAccuracy=
NetMeasurements[
t5ReasonSelection["TrainedNet"],
t5ReasonValRules,
"Accuracy",
BatchSize->t5ReasonBatch
];
Print["ReasonBestRound=",t5ReasonBestRound];
Print["ReasonValidationAccuracy=",t5ReasonValidationAccuracy];
If[!IntegerQ[t5ReasonBestRound]||t5ReasonBestRound<1,t5ReasonBestRound=t5ReasonRounds];
t5AllReasonRules=T5ReasonRules[t5MembershipRows];
SeedRandom[t5ReasonFinalSeed];
t5ReasonFinalNet0=NetInitialize[T5ReasonNet[]];
t5ReasonFinalTraining=
NetTrain[
t5ReasonFinalNet0,
t5AllReasonRules,
All,
MaxTrainingRounds->t5ReasonBestRound,
BatchSize->t5ReasonBatch,
LearningRate->t5ReasonLR,
Method->"ADAM",
TrainingProgressMeasurements->{"Accuracy","ErrorRate"},
TrainingProgressReporting->"Print",
TargetDevice->t5TargetDevice,
RandomSeeding->t5ReasonFinalSeed
];
t5FrozenReasoner=t5ReasonFinalTraining["TrainedNet"];
Export[
FileNameJoin[{t5OutputDirectory,"S124_T5_NEURAL_REASONER_FROZEN_BEFORE_HIGHERORDER.wl

```

```

t5FrozenReasoner
];
t5GlobalFreezeHash=
Hash[
{
frozenModelHashS119B,
t5PerceptionValidationAccuracy,
t5ReasonBestRound,
t5ReasonValidationAccuracy,
t5ReasonFinalSeed
},
"SHA256",
"HexString"
];
Print["====="];
Print["GLOBAL FREEZE COMPLETE"];
Print["SharedPerceptionFrozen=True"];
Print["TCCTFrozen=True"];
Print["NeuralReasonerFrozen=True"];
Print["HighOrderOutputsOpened=False"];
Print["GlobalFreezeHash=",t5GlobalFreezeHash];
Print["====="];
Clear[
t5ReasonTrainRules,
t5ReasonValRules,
t5AllReasonRules,
t5ReasonSelectNet0,
t5ReasonSelection,
t5ReasonFinalNet0,
t5ReasonFinalTraining
];
T5NeuralSignature[seq_List]:=
Table[
t5FrozenReasoner[T5ReasonInput[seq,probe]],
{probe,publicProbesS119B}
];
Print["====="];
Print["PHASE 6: OPEN SEALED >=3-FACTOR HIGH-ORDER HOLDOUT"];
Print["NO MODEL CHANGES AFTER THIS LINE"];
Print["====="];
t5HighOrderRows=
Table[
Module[{seq,trueSignature,tcctSignature,neuralSignature},
seq=row["Sequence"];
trueSignature=
Table[
S119BEnvRawOutput[seq,probe],
{probe,publicProbesS119B}
];
tcctSignature=
Table[
S119BFactorizedOutput[seq,probe],
{probe,publicProbesS119B}
];
neuralSignature=T5NeuralSignature[seq];
<|

```

```

"TupleEvaluatorOnly"->row["TupleEvaluatorOnly"],
"Sequence"->seq,
"TrueSignature"->trueSignature,
"TCCTSignature"->tcctSignature,
"NeuralSignature"->neuralSignature,
"TCCTExact"->SameQ[tcctSignature,trueSignature],
"NeuralExact"->SameQ[neuralSignature,trueSignature]
|>
],
{row,prospectiveHighOrderHoldoutS119B}
];
t5TCCTExactCount=Count[Lookup[t5HighOrderRows,"TCCTExact"],True];
t5NeuralExactCount=Count[Lookup[t5HighOrderRows,"NeuralExact"],True];
t5TCCTExactAccuracy=N[t5TCCTExactCount/Length[t5HighOrderRows]];
t5NeuralExactAccuracy=N[t5NeuralExactCount/Length[t5HighOrderRows]];
t5TrueProbeFlat=Flatten[Lookup[t5HighOrderRows,"TrueSignature"]];
t5TCCTProbeFlat=Flatten[Lookup[t5HighOrderRows,"TCCTSignature"]];
t5NeuralProbeFlat=Flatten[Lookup[t5HighOrderRows,"NeuralSignature"]];
t5TCCTProbeAccuracy=
N[
Mean[
MapThread[Boole[SameQ[#1,#2]]&,{t5TCCTProbeFlat,t5TrueProbeFlat}]
]
];
t5NeuralProbeAccuracy=
N[
Mean[
MapThread[Boole[SameQ[#1,#2]]&,{t5NeuralProbeFlat,t5TrueProbeFlat}]
]
];
t5PositivePositions=Flatten[Position[t5TrueProbeFlat,1]];
t5NegativePositions=Flatten[Position[t5TrueProbeFlat,0]];
t5NeuralPositiveRecall=
N[
Mean[
Boole[#==1]&/@t5NeuralProbeFlat[[t5PositivePositions]]
]
];
t5NeuralNegativeAccuracy=
N[
Mean[
Boole[#==0]&/@t5NeuralProbeFlat[[t5NegativePositions]]
]
];
t5NeuralBalancedAccuracy=
N[(t5NeuralPositiveRecall+t5NeuralNegativeAccuracy)/2.];
t5TCCTMinusNeuralExact=
N[t5TCCTExactAccuracy-t5NeuralExactAccuracy];
t5TCCTMinusNeuralProbe=
N[t5TCCTProbeAccuracy-t5NeuralProbeAccuracy];
t5ProtocolPass=
And[
t5PerceptionValidationAccuracy>=t5PerceptionGate,
t5TrainingPerceptionAccuracy>=t5TrainingPerceptionGate,
t5MaximumTrainingInteractionOrder<=2,
highOrderTouchedBeforeFreezeS119B==0,

```

```

policyHashUnchangedS119B,
inferredFactorCountS119B==4,
Length[t5HighOrderRows]>0
];
t5GeneralizationDiagnosis=
Which[
!TrueQ[t5ProtocolPass],
"PROTOCOL_GATE_FAILED",
t5TCCTMinusNeuralExact>=t5WinnerMargin,
"TCCT_HIGH_ORDER_COMPOSITIONAL_GENERALIZATION_ADVANTAGE",
t5TCCTMinusNeuralExact<=-t5WinnerMargin,
"NEURAL_HIGH_ORDER_COMPOSITIONAL_GENERALIZATION_ADVANTAGE",
True,
"HIGH_ORDER_GENERALIZATION_PARITY_WITHIN_PREDECLARED_MARGIN"
];
Print[""];
Print["====="];
Print["S124-T5 FINAL SUMMARY"];
Print["====="];
Print["BenchmarkSource=S119B"];
Print["SharedPerception=True"];
Print["PerceptionValidationAccuracy=",t5PerceptionValidationAccuracy];
Print["TrainingMembershipPerceptionAccuracy=",t5TrainingPerceptionAccuracy];
Print["MaximumTrainingInteractionOrder=",t5MaximumTrainingInteractionOrder];
Print["HighOrderDefinition=>=3 nonzero factors"];
Print["TrueJointStates=",trueJointStateCountS119B];
Print["HighOrderHoldoutStates=",Length[t5HighOrderRows]];
Print["HighOrderTouchedBeforeFreeze=",highOrderTouchedBeforeFreezeS119B];
Print["InferredFactors=",inferredFactorCountS119B];
Print["LocalStateCounts=",learnedLocalStateCountsS119B];
Print["LearnedInteractionEdges=",learnedInteractionEdgesS119B];
Print["ConditionalTransitionCells=",Length[conditionalTransitionsS119B]];
Print["MembershipQueriesBeforeFreeze=",membershipQueriesBeforeFreezeS119B];
Print["ReasonValidationAccuracy=",t5ReasonValidationAccuracy];
Print["ReasonBestRound=",t5ReasonBestRound];
Print["ZeroOnlyProbeBaseline=",N[10/14]];
Print["TCCTHighOrderExact=",t5TCCTExactCount,"/",Length[t5HighOrderRows]];
Print["TCCTHighOrderExactAccuracy=",t5TCCTExactAccuracy];
Print["NeuralHighOrderExact=",t5NeuralExactCount,"/",Length[t5HighOrderRows]];
Print["NeuralHighOrderExactAccuracy=",t5NeuralExactAccuracy];
Print["TCCTProbeAccuracy=",t5TCCTProbeAccuracy];
Print["NeuralProbeAccuracy=",t5NeuralProbeAccuracy];
Print["NeuralPositiveRecall=",t5NeuralPositiveRecall];
Print["NeuralNegativeAccuracy=",t5NeuralNegativeAccuracy];
Print["NeuralBalancedAccuracy=",t5NeuralBalancedAccuracy];
Print["TCCTMinusNeuralExact=",t5TCCTMinusNeuralExact];
Print["TCCTMinusNeuralProbe=",t5TCCTMinusNeuralProbe];
Print["WinnerMargin=",t5WinnerMargin];
Print["STRICT PROTOCOL PASS=",t5ProtocolPass];
Print["GENERALIZATION DIAGNOSIS=",t5GeneralizationDiagnosis];
Print["====="];
t5Summary=<|
"Stage"->"S124-T5",
"BenchmarkSource"->"S119B",
"Purpose"->"NeuralizedS119BHighOrderGeneralizationAttribution",
"SharedPerception"->True,

```



```

"PerceptionValidationAccuracy" -> t5PerceptionValidationAccuracy,
"TrainingMembershipPerceptionAccuracy" -> t5TrainingPerceptionAccuracy,
"MaximumTrainingInteractionOrder" -> t5MaximumTrainingInteractionOrder,
"HighOrderDefinition" -> ">=3NonzeroFactors",
"TrueJointStates" -> trueJointStateCountS119B,
"HighOrderHoldoutStates" -> Length[t5HighOrderRows],
"HighOrderTouchedBeforeFreeze" -> highOrderTouchedBeforeFreezeS119B,
"InferredFactors" -> inferredFactorCountS119B,
"LocalStateCounts" -> learnedLocalStateCountsS119B,
"LearnedInteractionEdges" -> learnedInteractionEdgesS119B,
"ConditionalTransitionCells" -> Length[conditionalTransitionsS119B],
"MembershipQueriesBeforeFreeze" -> membershipQueriesBeforeFreezeS119B,
"ReasonValidationAccuracy" -> t5ReasonValidationAccuracy,
"ReasonBestRound" -> t5ReasonBestRound,
"TCCTHighOrderExactCount" -> t5TCCTExactCount,
"TCCTHighOrderExactAccuracy" -> t5TCCTExactAccuracy,
"NeuralHighOrderExactCount" -> t5NeuralExactCount,
"NeuralHighOrderExactAccuracy" -> t5NeuralExactAccuracy,
"TCCTProbeAccuracy" -> t5TCCTProbeAccuracy,
"NeuralProbeAccuracy" -> t5NeuralProbeAccuracy,
"NeuralPositiveRecall" -> t5NeuralPositiveRecall,
"NeuralNegativeAccuracy" -> t5NeuralNegativeAccuracy,
"NeuralBalancedAccuracy" -> t5NeuralBalancedAccuracy,
"TCCTMinusNeuralExact" -> t5TCCTMinusNeuralExact,
"TCCTMinusNeuralProbe" -> t5TCCTMinusNeuralProbe,
"WinnerMargin" -> t5WinnerMargin,
"StrictProtocolPass" -> t5ProtocolPass,
"GeneralizationDiagnosis" -> t5GeneralizationDiagnosis,
"TCCTFreezeHash" -> frozenModelHashS119B,
"GlobalFreezeHash" -> t5GlobalFreezeHash,
"HighOrderOutputsOpenedAfterAllModelsFrozen" -> True
|>;
t5SummaryFile=
FileNameJoin[{t5OutputDirectory, "S124_T5_summary.wl"}];
Put[t5Summary, t5SummaryFile];
Print["SummaryFile=", t5SummaryFile];
Print["====="];
Print["S124-T5 COMPLETE"];
Print["====="];

```

```

=====
S124-T5 STANDALONE
S119B NEURALIZED HIGH-ORDER GENERALIZATION ATTRIBUTION
SHARED TRANSFORMER PERCEPTION
PAIRWISE TRAINING -> UNSEEN >=3-FACTOR COMBINATIONS
=====
WolframVersion=15.0.0 for Microsoft Windows (64-bit) (May 26, 2026)
Date=Wed 19 Aug 2026 22:17:02
PublicActions={0, 1, 2, 3, 4, 5, 6, 7}
PublicProbes={0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13}
MaximumTrainingInteractionOrder=2
PolicyHash=8e0253972e57175c12eb1866635feace701742cdc36007567aaafad00c59c26c
HIDDEN WORLD DOES NOT EXIST YET.
=====
PHASE 1: TRAIN SHARED BINARY PERCEPTION
HIDDEN WORLD STILL DOES NOT EXIST
=====
Starting training.
Optimization Method: ADAM
    Beta1: 9.00*^-1
    Beta2: 9.99*^-1
    Epsilon: 1.00*^-5
    Gradient Clipping: --
    L2 Regularization: --
    Learning Rate: 3.00*^-4
    Learning Rate Schedule: --
    Weight Clipping: --

Device: CPU
Batch Size: 64
Batches Per Round: 38
Training Examples: 2400
  % round batch  examples      inputs    learning      time      time  \
>  current      round      test    current      round      test    current\
>
>      round      test
>      /20      /38 processed    /second      rate    elapsed      left    \
>  loss      loss      loss      acc.      acc.      acc.      error  \
>
>      error      error
>      7      2      18      3584      2097    7.02*^-5      2s      26s  \
>  1.25*^-1    1.83*^-1    1.36*^-3      95.84%    93.87%      100%    \
>
>  4.16%      6.13%      0%
>  15      4      2      7424      1620    9.93*^-5      4s      22s  \
>
>  1.53*^-3    1.36*^-3    4.57*^-4      100%      100%      100%    \
>
>  0%      0%      0%
>  24      5      30      11648      2182    1.22*^-4      6s      19s  \
>
>  9.37*^-4    1.04*^-3    3.34*^-4      100%      100%      100%    \
>
>  0%      0%      0%

```

32	7	15	15552	2041	1.39×10^{-4}	8s	17s	\
>	6.35×10^{-4}	6.50×10^{-4}	2.01×10^{-4}		100%	100%	100%	\
>	0%	0%	0%					
40	8	38	19456	2208	1.54×10^{-4}	10s	15s	\
>	4.67×10^{-4}	4.40×10^{-4}	1.36×10^{-4}		100%	100%	100%	\
>	0%	0%	0%					
48	10	26	23552	2118	1.66×10^{-4}	12s	13s	\
>	3.50×10^{-4}	3.69×10^{-4}	1.15×10^{-4}		100%	100%	100%	\
>	0%	0%	0%					
56	12	11	27456	1948	1.77×10^{-4}	14s	11s	\
>	2.83×10^{-4}	2.80×10^{-4}	8.62×10^{-5}		100%	100%	100%	\
>	0%	0%	0%					
65	13	38	31616	2223	1.87×10^{-4}	16s	9s	\
>	2.37×10^{-4}	2.26×10^{-4}	6.71×10^{-5}		100%	100%	100%	\
>	0%	0%	0%					
74	15	33	36160	2301	1.97×10^{-4}	18s	6s	\
>	1.96×10^{-4}	2.06×10^{-4}	5.99×10^{-5}		100%	100%	100%	\
>	0%	0%	0%					
83	17	20	40192	2143	2.05×10^{-4}	20s	4s	\
>	1.61×10^{-4}	1.64×10^{-4}	4.90×10^{-5}		100%	100%	100%	\
>	0%	0%	0%					
91	19	6	44160	1798	2.12×10^{-4}	22s	2s	\
>	1.42×10^{-4}	1.39×10^{-4}	4.10×10^{-5}		100%	100%	100%	\
>	0%	0%	0%					
99	20	34	48384	2211	2.19×10^{-4}	24s	0s	\
>	1.26×10^{-4}	1.29×10^{-4}	3.77×10^{-5}		100%	100%	100%	\

> 0% 0% 0%
PerceptionValidationAccuracy=1.
SHARED PERCEPTION FROZEN

=====

PHASE 2: GENERATE ORIGINAL S119B HIDDEN WORLD

=====

NEW HIDDEN SPARSELY COUPLED WORLD GENERATED

TrueJointStates=120

ProspectiveHighOrderStates=74

HIGH-ORDER OUTPUTS SEALED

=====

PHASE 3: ORIGINAL S119B FACTOR SUPPORT DISCOVERY

HIGH-ORDER OUTPUTS STILL SEALED

=====

InferredFactors=4

ProbeGroupSizes={5, 3, 4, 2}

ActionGroupSizes={2, 2, 2, 2}

LocalStateCounts={5, 3, 4, 2}

StoredLocalStates=14

ProductCapacity=120

=====

PHASE 4: ACTIVE PAIRWISE INTERACTION DISCOVERY

HIGH-ORDER OUTPUTS STILL SEALED

=====

LearnedParentsByAction=<|0 -> {}, 1 -> {}, 2 -> {}, 3 -> {2}, 4 -> {4},

> 5 -> {}, 6 -> {}, 7 -> {}|>

LearnedInteractionEdges={2 -> 1, 4 -> 3}

ConditionalTransitionCells=42

=====

S119B TCCT MODEL FROZEN

Factors=4

LocalStateCounts={5, 3, 4, 2}

LearnedInteractionEdges={2 -> 1, 4 -> 3}

ConditionalTransitionCells=42

MembershipQueries=1822

HighOrderHoldoutTouched=0/74

TCCTFreezeHash=aa81eba170f57e2278a8c551a664fe50bdfd44989fd36551ff348743b1f600\

> d4

=====

TrainingPerceptionAccuracy=1.

MaximumObservedTrainingInteractionOrder=2

=====

PHASE 5: TRAIN MATCHED NEURAL TRANSFORMER REASONER

USES EXACT SAME MEMBERSHIP DATA ACQUIRED BY S119B

HIGH-ORDER OUTPUTS STILL SEALED

=====

ReasonTrainingRows=1442

ReasonValidationRows=380

ReasonMaxSequenceLength=10

NetTrain::invindim: Data provided to port "Input" should be a non-empty list of nÃ3
3 matrices.

NetMeasurements::invnet: First argument to NetMeasurements should be a fully specifi
ed net.

ReasonBestRound=\$Failed[BestValidationRound]

ReasonValidationAccuracy=\$Failed

NetTrain::invindim: Data provided to port "Input" should be a non-empty list of nÃ3
3 matrices.

Export::invnet2: The second argument in Export[E:\TCCT_CODEX_HANDOFF_2026-08-13\S97A
_ReadoutBaseline_Development\S124_T5_Output\S124_T5_NEURAL_REASONER_FROZEN_BEFORE_HI
GHORDER.wlnet, \$Failed[TrainedNet]] is not a valid net.

```
=====
GLOBAL FREEZE COMPLETE
SharedPerceptionFrozen=True
TCCTFrozen=True
NeuralReasonerFrozen=True
HighOrderOutputsOpened=False
GlobalFreezeHash=bdf2d89fc71ce0357ebbb3635ae1b0a59c161c577117b4ea26e01b469776\
```

```
> 48a5
```

```
=====
PHASE 6: OPEN SEALED >=3-FACTOR HIGH-ORDER HOLDOUT
NO MODEL CHANGES AFTER THIS LINE
=====
```

```
=====
S124-T5 FINAL SUMMARY
=====
```

```
BenchmarkSource=S119B
SharedPerception=True
PerceptionValidationAccuracy=1.
TrainingMembershipPerceptionAccuracy=1.
MaximumTrainingInteractionOrder=2
HighOrderDefinition=>=3 nonzero factors
TrueJointStates=120
HighOrderHoldoutStates=74
HighOrderTouchedBeforeFreeze=0
InferredFactors=4
LocalStateCounts={5, 3, 4, 2}
LearnedInteractionEdges={2 -> 1, 4 -> 3}
ConditionalTransitionCells=42
MembershipQueriesBeforeFreeze=1822
ReasonValidationAccuracy=$Failed
ReasonBestRound=35
ZeroOnlyProbeBaseline=0.714286
TCCTHighOrderExact=74/74
TCCTHighOrderExactAccuracy=1.
NeuralHighOrderExact=0/74
NeuralHighOrderExactAccuracy=0.
TCCTProbeAccuracy=1.
NeuralProbeAccuracy=0.
NeuralPositiveRecall=0.
NeuralNegativeAccuracy=0.
NeuralBalancedAccuracy=0.
TCCTMinusNeuralExact=1.
TCCTMinusNeuralProbe=1.
WinnerMargin=0.02
STRICT PROTOCOL PASS=True
GENERALIZATION DIAGNOSIS=TCCT_HIGH_ORDER_COMPOSITIONAL_GENERALIZATION_ADVANTA\
```

```
> GE
```

```
=====
SummaryFile=E:\TCCT_CODEX_HANDOFF_2026-08-13\S97A_ReadoutBaseline_Development\
```

```
> \S124_T5_Output\S124_T5_summary.wl
=====
```

S124-T5 COMPLETE

=====

In [122...

```

Print["====="];
Print["S124-T5R NEURAL REASONER RECOVERY"];
Print["TCCT NOT RETRAINED"];
Print["PERCEPTION NOT RETRAINED"];
Print["RECOVERY DIAGNOSTIC ONLY"];
Print["====="];

T5RStop[msg_] := (Print["FATAL: ", msg]; Abort[]);

t5rReady = And[
  ValueQ[t5MembershipRows],
  ListQ[t5MembershipRows],
  Length[t5MembershipRows] > 0,
  ValueQ[prospectiveHighOrderHoldoutS119B],
  ListQ[prospectiveHighOrderHoldoutS119B],
  Length[prospectiveHighOrderHoldoutS119B] > 0,
  ValueQ[t5ReasonMaxSeqLen],
  ValueQ[t5ReasonInputDim],
  Length[DownValues[T5ReasonNet]] > 0,
  Length[DownValues[T5ReasonActionToken]] > 0,
  Length[DownValues[T5ReasonPadToken]] > 0,
  Length[DownValues[T5ReasonProbeToken]] > 0,
  Length[DownValues[S119BEnvRawOutput]] > 0,
  Length[DownValues[S119BFactorizedOutput]] > 0
];

If[!TrueQ[t5rReady],
  T5RStop["required T5 environment is not available in current kernel"]
];

t5rBatch = 64;
t5rRounds = 35;
t5rLR = 0.0003;
t5rSplitSeed = 9251001;
t5rSelectSeed = 9252001;
t5rFinalSeed = 9252002;
t5rTargetDevice = "CPU";

Clear[T5RReasonInput];

T5RReasonInput[seq_List, probe_Integer] :=
Module[{padded, tokens},
  padded = PadRight[seq, t5ReasonMaxSeqLen, -1];
  tokens =
  MapIndexed[
    If[
      #1 == -1,
      T5ReasonPadToken[First[#2]],
      T5ReasonActionToken[#1, First[#2]]
    ] &,
    padded
  ];
  N[

```

```

Append[
tokens,
T5ReasonProbeToken[probe,t5ReasonMaxSeqLen+1]
]
];

Clear[T5RInputs,T5RLabels,T5RRules];

T5RInputs[rows_List]:=
(T5RReasonInput[#["Seq"],#["Probe"]]&)/@rows;

T5RLabels[rows_List]:=
Lookup[rows,"Y"];

T5RRules[rows_List]:=
MapThread[
Rule,
{
T5RInputs[rows],
T5RLabels[rows]
}
];

t5rSampleRow=First[t5MembershipRows];
t5rSampleInput=
T5RReasonInput[
t5rSampleRow["Seq"],
t5rSampleRow["Probe"]
];

Print[""];
Print["====="];
Print["INPUT SHAPE AUDIT"];
Print["====="];
Print["SampleDimensions=",Dimensions[t5rSampleInput]];
Print["ExpectedDimensions=",{t5ReasonMaxSeqLen+1,t5ReasonInputDim}];

If[
Dimensions[t5rSampleInput]!={t5ReasonMaxSeqLen+1,t5ReasonInputDim},
T5RStop["corrected neural input shape still invalid"]
];

If[
!MatrixQ[t5rSampleInput,NumericQ],
T5RStop["corrected neural input is not a numeric matrix"]
];

Print["INPUT SHAPE AUDIT=PASS"];

Clear[T5RPredictBatched];

T5RPredictBatched[net_,inputs_List,batch_Integer]:=
Module[{out={},i,last,z},
Do[
last=Min[i+batch-1,Length[inputs]];

```

```

z=net[Take[inputs,{i,last}]];
If[!ListQ[z],z={z}];
out=Join[out,z];
Print["Prediction=",last,"/",Length[inputs]],
{i,1,Length[inputs],batch}
];
out
];

Clear[T5RClassAccuracy,T5RBalancedAccuracy];

T5RClassAccuracy[truth_List,pred_List,c_Integer]:=
Module[{idx},
idx=Flatten[Position[truth,c]];
If[
idx=={},
Indeterminate,
N[
Mean[
MapThread[
Boole[SameQ[#1,#2]]&,
{
pred[[idx]],
truth[[idx]]
}
]
]
]
];

T5RBalancedAccuracy[truth_List,pred_List]:=
Module[{a0,a1},
a0=T5RClassAccuracy[truth,pred,0];
a1=T5RClassAccuracy[truth,pred,1];
If[
NumericQ[a0]&&NumericQ[a1],
N[(a0+a1)/2.],
Indeterminate
]
];

Print[""];
Print["====="];
Print["REBUILD TRAIN/VALIDATION SPLIT"];
Print["====="];

t5rRowsBySequence=
GroupBy[
t5MembershipRows,
S119BSeqKey[#["Seq"]]&
];

t5rSequenceKeys=Keys[t5rRowsBySequence];

t5rShuffledKeys=

```



```

BlockRandom[
SeedRandom[t5rSplitSeed];
RandomSample[t5rSequenceKeys]
];

t5rValSequenceCount=
Max[
1,
Round[0.20*Length[t5rShuffledKeys]]
];

t5rValKeys=
Take[
t5rShuffledKeys,
t5rValSequenceCount
];

t5rTrainKeys=
Drop[
t5rShuffledKeys,
t5rValSequenceCount
];

t5rTrainRows=
Flatten[
Lookup[t5rRowsBySequence,t5rTrainKeys],
1
];

t5rValRows=
Flatten[
Lookup[t5rRowsBySequence,t5rValKeys],
1
];

t5rTrainInputs=T5RInputs[t5rTrainRows];
t5rTrainLabels=T5RLabels[t5rTrainRows];
t5rValInputs=T5RInputs[t5rValRows];
t5rValLabels=T5RLabels[t5rValRows];

t5rTrainRules=
MapThread[
Rule,
{
t5rTrainInputs,
t5rTrainLabels
}
];

t5rValRules=
MapThread[
Rule,
{
t5rValInputs,
t5rValLabels
}
];

```

```

];

Print["TrainingRows=",Length[t5rTrainRows]];
Print["ValidationRows=",Length[t5rValRows]];
Print["TrainingSequences=",Length[t5rTrainKeys]];
Print["ValidationSequences=",Length[t5rValKeys]];

t5rValZeroBaseline=
N[
Count[t5rValLabels,0]/
Length[t5rValLabels]
];

Print["ValidationZeroBaseline=",t5rValZeroBaseline];

Print[""];
Print["====="];
Print["TRAIN RECOVERY NEURAL REASONER"];
Print["====="];

SeedRandom[t5rSelectSeed];

t5rNet0=
NetInitialize[
T5ReasonNet[]
];

t5rSelection=
NetTrain[
t5rNet0,
t5rTrainRules,
All,
ValidationSet->t5rValRules,
MaxTrainingRounds->t5rRounds,
BatchSize->t5rBatch,
LearningRate->t5rLR,
Method->"ADAM",
TrainingProgressMeasurements->{
"Accuracy",
"ErrorRate"
},
TrainingStoppingCriterion-><|
"Criterion"->"Loss",
"Patience"->6
|>,
TrainingProgressReporting->"Print",
TargetDevice->t5rTargetDevice,
RandomSeeding->t5rSelectSeed
];

t5rSelectionNet=
t5rSelection["TrainedNet"];

Print[""];
Print["Running manual validation prediction..."];

```

```

t5rValPred=
T5RPredictBatched[
t5rSelectionNet,
t5rValInputs,
128
];

If[
Length[t5rValPred]!=Length[t5rValLabels],
T5RStop["validation prediction length mismatch"]
];

If[
!And@@(
MemberQ[{0,1},#]&/@t5rValPred
),
Print["Invalid validation predictions:"];
Print[DeleteDuplicates[t5rValPred]];
T5RStop["neural decoder did not return binary classes"]
];

t5rValidationAccuracy=
N[
Mean[
MapThread[
Boole[SameQ[#1,#2]]&,
{
t5rValPred,
t5rValLabels
}
]
]
];

t5rValidationZeroAccuracy=
T5RClassAccuracy[
t5rValLabels,
t5rValPred,
0
];

t5rValidationOneAccuracy=
T5RClassAccuracy[
t5rValLabels,
t5rValPred,
1
];

t5rValidationBalancedAccuracy=
T5RBalancedAccuracy[
t5rValLabels,
t5rValPred
];

Print[""];
Print["RECOVERY VALIDATION"];

```

```

Print["ValidationAccuracy=",t5rValidationAccuracy];
Print["ValidationZeroAccuracy=",t5rValidationZeroAccuracy];
Print["ValidationOneAccuracy=",t5rValidationOneAccuracy];
Print["ValidationBalancedAccuracy=",t5rValidationBalancedAccuracy];
Print["ZeroOnlyBaseline=",t5rValZeroBaseline];

If[
!NumericQ[t5rValidationAccuracy],
T5RStop["validation accuracy is not numeric"]
];

If[
!NumericQ[t5rValidationBalancedAccuracy],
T5RStop["validation balanced accuracy is not numeric"]
];

If[
t5rValidationBalancedAccuracy<0.60,
T5RStop["neural reasoner failed competence gate"]
];

Print["NEURAL VALIDATION PIPELINE=PASS"];

Print[""];
Print["====="];
Print["TRAIN FINAL REASONER ON ALL S119B MEMBERSHIP DATA"];
Print["FIXED ROUNDS=",t5rRounds];
Print["====="];

t5rAllInputs=T5RInputs[t5MembershipRows];
t5rAllLabels=T5RLabels[t5MembershipRows];

t5rAllRules=
MapThread[
Rule,
{
t5rAllInputs,
t5rAllLabels
}
];

SeedRandom[t5rFinalSeed];

t5rFinalNet0=
NetInitialize[
T5ReasonNet[]
];

t5rFinalTraining=
NetTrain[
t5rFinalNet0,
t5rAllRules,
All,
MaxTrainingRounds->t5rRounds,
BatchSize->t5rBatch,
LearningRate->t5rLR,

```

```

Method->"ADAM",
TrainingProgressMeasurements->{
  "Accuracy",
  "ErrorRate"
},
TrainingProgressReporting->"Print",
TargetDevice->t5rTargetDevice,
RandomSeeding->t5rFinalSeed
];

t5rFrozenReasoner=
t5rFinalTraining["TrainedNet"];

t5rModelFile=
FileNameJoin[
{
t5OutputDirectory,
"S124_T5R_NEURAL_REASONER_RECOVERY.wlnet"
}
];

Export[
t5rModelFile,
t5rFrozenReasoner
];

Print["RecoveryModel=", t5rModelFile];

Clear[T5RNeuralSignature];

T5RNeuralSignature[seq_List]:=
Module[{inputs, pred},
inputs=
Table[
T5RReasonInput[
seq,
probe
],
{probe, publicProbesS119B}
];
pred=
T5RPredictBatched[
t5rFrozenReasoner,
inputs,
14
];
If[
Length[pred]!=Length[publicProbesS119B],
T5RStop["high-order signature length mismatch"]
];
If[
!And@@(
MemberQ[{0,1},#]&/@pred
),
Print["Invalid high-order predictions:"];
Print[DeleteDuplicates[pred]];

```

```

T5RStop["high-order neural prediction is not binary"]
];
pred
];

Print[""];
Print["====="];
Print["RECOVERY HIGH-ORDER DIAGNOSTIC"];
Print["IMPORTANT: HOLDOUT WAS ALREADY OPENED IN ORIGINAL T5"];
Print["STRICT_PROSPECTIVE=False"];
Print["====="];

t5rHighOrderRows=
Table[
Module[
{seq,trueSignature,tcctSignature,neuralSignature},
seq=row["Sequence"];

trueSignature=
Table[
S119BEnvRawOutput[
seq,
probe
],
{probe,publicProbesS119B}
];

tcctSignature=
Table[
S119BFactorizedOutput[
seq,
probe
],
{probe,publicProbesS119B}
];

neuralSignature=
T5RNeuralSignature[
seq
];

<|
"TupleEvaluatorOnly"->row["TupleEvaluatorOnly"],
"TrueSignature"->trueSignature,
"TCCTSignature"->tcctSignature,
"NeuralSignature"->neuralSignature,
"TCCTExact"->SameQ[tcctSignature,trueSignature],
"NeuralExact"->SameQ[neuralSignature,trueSignature]
|>
],
{row,prospectiveHighOrderHoldoutS119B}
];

t5rTCCTExactCount=
Count[
Lookup[t5rHighOrderRows,"TCCTExact"],

```

```

True
];

t5rNeuralExactCount=
Count[
Lookup[t5rHighOrderRows,"NeuralExact"],
True
];

t5rTCCTExactAccuracy=
N[
t5rTCCTExactCount/
Length[t5rHighOrderRows]
];

t5rNeuralExactAccuracy=
N[
t5rNeuralExactCount/
Length[t5rHighOrderRows]
];

t5rTrueFlat=
Flatten[
Lookup[
t5rHighOrderRows,
"TrueSignature"
]
];

t5rTCCTFlat=
Flatten[
Lookup[
t5rHighOrderRows,
"TCCTSignature"
]
];

t5rNeuralFlat=
Flatten[
Lookup[
t5rHighOrderRows,
"NeuralSignature"
]
];

t5rTCCTProbeAccuracy=
N[
Mean[
MapThread[
Boole[SameQ[#1,#2]]&,
{
t5rTCCTFlat,
t5rTrueFlat
}
]
]
];

```

```

];

t5rNeuralProbeAccuracy=
N[
Mean[
MapThread[
Boole[SameQ[#1,#2]]&,
{
t5rNeuralFlat,
t5rTrueFlat
}
]
]
];

t5rNeuralZeroAccuracy=
T5RClassAccuracy[
t5rTrueFlat,
t5rNeuralFlat,
0
];

t5rNeuralOneAccuracy=
T5RClassAccuracy[
t5rTrueFlat,
t5rNeuralFlat,
1
];

t5rNeuralBalancedAccuracy=
T5RBalancedAccuracy[
t5rTrueFlat,
t5rNeuralFlat
];

t5rTCCTMinusNeuralExact=
N[
t5rTCCTExactAccuracy-
t5rNeuralExactAccuracy
];

t5rTCCTMinusNeuralProbe=
N[
t5rTCCTProbeAccuracy-
t5rNeuralProbeAccuracy
];

Print[""];
Print["====="];
Print["S124-T5R FINAL RECOVERY SUMMARY"];
Print["====="];
Print["RecoveryDiagnosticOnly=True"];
Print["StrictProspective=False"];
Print["ReasonInputShapeFixed=True"];
Print["ReasonValidationAccuracy=",t5rValidationAccuracy];
Print["ReasonValidationZeroAccuracy=",t5rValidationZeroAccuracy];

```



```

Print["ReasonValidationOneAccuracy=", t5rValidationOneAccuracy];
Print["ReasonValidationBalancedAccuracy=", t5rValidationBalancedAccuracy];
Print["ValidationZeroOnlyBaseline=", t5rValZeroBaseline];
Print["HighOrderStates=", Length[t5rHighOrderRows]];
Print["TCCTHighOrderExact=", t5rTCCTExactCount, "/", Length[t5rHighOrderRows]];
Print["TCCTHighOrderExactAccuracy=", t5rTCCTExactAccuracy];
Print["NeuralHighOrderExact=", t5rNeuralExactCount, "/", Length[t5rHighOrderRows]];
Print["NeuralHighOrderExactAccuracy=", t5rNeuralExactAccuracy];
Print["TCCTProbeAccuracy=", t5rTCCTProbeAccuracy];
Print["NeuralProbeAccuracy=", t5rNeuralProbeAccuracy];
Print["NeuralZeroAccuracy=", t5rNeuralZeroAccuracy];
Print["NeuralOneAccuracy=", t5rNeuralOneAccuracy];
Print["NeuralBalancedAccuracy=", t5rNeuralBalancedAccuracy];
Print["TCCTMinusNeuralExact=", t5rTCCTMinusNeuralExact];
Print["TCCTMinusNeuralProbe=", t5rTCCTMinusNeuralProbe];
Print["NEURAL PIPELINE VALID=True"];
Print["ATTRIBUTION STATUS=DIAGNOSTIC_ONLY_NOT_STRICTLY_SCORED"];
Print["====="];

t5rSummary=<|
"Stage"->"S124-T5R",
"RecoveryDiagnosticOnly"->True,
"StrictProspective"->False,
"ReasonInputShapeFixed"->True,
"ReasonValidationAccuracy"->t5rValidationAccuracy,
"ReasonValidationZeroAccuracy"->t5rValidationZeroAccuracy,
"ReasonValidationOneAccuracy"->t5rValidationOneAccuracy,
"ReasonValidationBalancedAccuracy"->t5rValidationBalancedAccuracy,
"ValidationZeroOnlyBaseline"->t5rValZeroBaseline,
"HighOrderStates"->Length[t5rHighOrderRows],
"TCCTHighOrderExactAccuracy"->t5rTCCTExactAccuracy,
"NeuralHighOrderExactAccuracy"->t5rNeuralExactAccuracy,
"TCCTProbeAccuracy"->t5rTCCTProbeAccuracy,
"NeuralProbeAccuracy"->t5rNeuralProbeAccuracy,
"NeuralZeroAccuracy"->t5rNeuralZeroAccuracy,
"NeuralOneAccuracy"->t5rNeuralOneAccuracy,
"NeuralBalancedAccuracy"->t5rNeuralBalancedAccuracy,
"TCCTMinusNeuralExact"->t5rTCCTMinusNeuralExact,
"TCCTMinusNeuralProbe"->t5rTCCTMinusNeuralProbe,
"NeuralPipelineValid"->True,
"AttributionStatus"->"DIAGNOSTIC_ONLY_NOT_STRICTLY_SCORED"
|>;

t5rSummaryFile=
FileNameJoin[
{
t5OutputDirectory,
"S124_T5R_recovery_summary.wl"
}
];

Put[
t5rSummary,
t5rSummaryFile
];

```

```
Print["SummaryFile=",t5rSummaryFile];  
Print["====="];  
Print["S124-T5R COMPLETE"];  
Print["====="];
```

```

=====
S124-T5R NEURAL REASONER RECOVERY
TCCT NOT RETRAINED
PERCEPTION NOT RETRAINED
RECOVERY DIAGNOSTIC ONLY
=====

=====
INPUT SHAPE AUDIT
=====
SampleDimensions={11, 33}
ExpectedDimensions={11, 33}
INPUT SHAPE AUDIT=PASS

=====
REBUILD TRAIN/VALIDATION SPLIT
=====
TrainingRows=1442
ValidationRows=380
TrainingSequences=242
ValidationSequences=61
ValidationZeroBaseline=0.721053

=====
TRAIN RECOVERY NEURAL REASONER
=====
Starting training.
Optimization Method: ADAM
    Beta1: 9.00*^-1
    Beta2: 9.99*^-1
    Epsilon: 1.00*^-5
    Gradient Clipping: --
    L2 Regularization: --
    Learning Rate: 3.00*^-4
    Learning Rate Schedule: --
    Weight Clipping: --

Device: CPU
Batch Size: 64
Batches Per Round: 23
Training Examples: 1442
  % round batch  examples    inputs    learning    time    time  \

>  current      round      test    current      round      test    current\

>      round      test
    /35    /23 processed    /second      rate    elapsed    left    \

>  loss      loss      loss      acc.      acc.      acc.      error  \

>  error      error
   5        2    21      2816      1571    6.29*^-5      2s      36s  \

>  6.42*^-1    6.60*^-1    6.30*^-1      69.1%      68.1%      72.1%      \

>  30.9%      31.9%      27.9%
   11        4    19      5632      1551    8.71*^-5      4s      33s  \

```

>	5.83 ⁻¹	5.94 ⁻¹	5.48 ⁻¹	72.3%	72.0%	72.1%	\
>	27.7%	28.0%	27.9%				
17	6	18	8512	1555	1.06 ⁻⁴	6s	31s \
>	5.36 ⁻¹	5.54 ⁻¹	5.06 ⁻¹	74.9%	73.5%	76.8%	\
>	25.1%	26.5%	23.2%				
22	8	18	11456	1547	1.21 ⁻⁴	8s	28s \
>	5.25 ⁻¹	5.23 ⁻¹	4.82 ⁻¹	76.0%	76.2%	79.2%	\
>	24.0%	23.8%	20.8%				
28	10	18	14400	1561	1.35 ⁻⁴	10s	26s \
>	5.15 ⁻¹	5.13 ⁻¹	4.76 ⁻¹	76.4%	76.5%	78.7%	\
>	23.6%	23.5%	21.3%				
34	12	18	17344	1553	1.46 ⁻⁴	12s	24s \
>	5.10 ⁻¹	5.21 ⁻¹	4.71 ⁻¹	76.6%	75.5%	78.4%	\
>	23.4%	24.5%	21.6%				
39	14	18	20288	1560	1.56 ⁻⁴	14s	22s \
>	5.09 ⁻¹	5.15 ⁻¹	4.68 ⁻¹	77.8%	77.2%	79.5%	\
>	22.2%	22.8%	20.5%				
45	16	18	23232	1554	1.66 ⁻⁴	16s	20s \
>	4.99 ⁻¹	4.96 ⁻¹	4.63 ⁻¹	77.3%	77.6%	79.5%	\
>	22.7%	22.4%	20.5%				
51	18	18	26176	1548	1.74 ⁻⁴	18s	18s \
>	4.98 ⁻¹	4.99 ⁻¹	4.56 ⁻¹	77.8%	77.6%	80.8%	\
>	22.2%	22.4%	19.2%				
57	20	18	29120	1537	1.81 ⁻⁴	20s	16s \
>	4.99 ⁻¹	4.94 ⁻¹	4.53 ⁻¹	77.5%	78.5%	80.5%	\
>	22.5%	21.5%	19.5%				
62	22	18	32064	1556	1.88 ⁻⁴	22s	13s \
>	4.80 ⁻¹	4.76 ⁻¹	4.42 ⁻¹	78.4%	78.5%	80.5%	\
>	21.6%	21.5%	19.5%				
68	24	18	35008	1560	1.95 ⁻⁴	24s	11s \
>	4.79 ⁻¹	4.83 ⁻¹	4.33 ⁻¹	78.5%	77.8%	81.1%	\
>	21.5%	22.2%	18.9%				
74	26	18	37952	1542	2.01 ⁻⁴	26s	9s \

>	4.84* ⁻¹	4.82* ⁻¹	4.27* ⁻¹	78.9%	79.5%	82.6%	\
>	21.1%	20.5%	17.4%				
	80	28	19	40960	1581	2.06* ⁻⁴	28s 7s \
>	4.64* ⁻¹	4.66* ⁻¹	4.19* ⁻¹	80.3%	80.2%	81.1%	\
>	19.7%	19.8%	18.9%				
	85	30	19	43904	1560	2.11* ⁻⁴	30s 5s \
>	4.61* ⁻¹	4.63* ⁻¹	4.12* ⁻¹	79.9%	79.8%	82.6%	\
>	20.1%	20.2%	17.4%				
	91	32	18	46784	1518	2.16* ⁻⁴	32s 3s \
>	4.42* ⁻¹	4.46* ⁻¹	4.00* ⁻¹	80.9%	80.6%	84.2%	\
>	19.1%	19.4%	15.8%				
	97	34	18	49728	1565	2.21* ⁻⁴	34s 1s \
>	4.37* ⁻¹	4.36* ⁻¹	3.98* ⁻¹	81.7%	81.7%	82.4%	\
>	18.3%	18.3%	17.6%				

Running manual validation prediction...

Prediction=128/380

Prediction=256/380

Prediction=380/380

RECOVERY VALIDATION

ValidationAccuracy=0.823684

ValidationZeroAccuracy=0.923358

ValidationOneAccuracy=0.566038

ValidationBalancedAccuracy=0.744698

ZeroOnlyBaseline=0.721053

NEURAL VALIDATION PIPELINE=PASS

=====

TRAIN FINAL REASONER ON ALL S119B MEMBERSHIP DATA

FIXED ROUNDS=35

=====

Starting training.

Optimization Method: ADAM

Beta1: 9.00*⁻¹

Beta2: 9.99*⁻¹

Epsilon: 1.00*⁻⁵

Gradient Clipping: --

L2 Regularization: --

Learning Rate: 3.00*⁻⁴

Learning Rate Schedule: --

Weight Clipping: --

Device: CPU

Batch Size: 64

Batches Per Round: 29

Training Examples: 1822

%	round	batch	examples	inputs	learning	time	time	\
---	-------	-------	----------	--------	----------	------	------	---

>	current /35	round /29	current processed	round /second	current rate	round elapsed	left	\
>	loss 4	loss 2	acc. 16	acc. 2880	error 1659	error 6.35* ⁻⁵	2s	45s \
>	6.39* ⁻¹ 9	6.59* ⁻¹ 4	69.4% 8	67.9% 6080	30.6% 1642	32.1% 9.03* ⁻⁵	4s	39s \
>	5.71* ⁻¹ 14	5.74* ⁻¹ 6	72.8% 1	72.5% 9344	27.2% 1631	27.5% 1.11* ⁻⁴	6s	36s \
>	5.31* ⁻¹ 19	5.24* ⁻¹ 7	75.9% 23	76.3% 12608	24.1% 1626	23.7% 1.27* ⁻⁴	8s	33s \
>	5.20* ⁻¹ 24	5.24* ⁻¹ 9	76.0% 15	75.2% 15808	24.0% 1606	24.8% 1.40* ⁻⁴	10s	31s \
>	5.04* ⁻¹ 29	5.06* ⁻¹ 11	77.4% 7	76.8% 19008	22.6% 1620	23.2% 1.52* ⁻⁴	12s	29s \
>	4.94* ⁻¹ 34	4.91* ⁻¹ 12	77.7% 29	78.1% 22272	22.3% 1620	21.9% 1.63* ⁻⁴	14s	27s \
>	4.91* ⁻¹ 39	4.90* ⁻¹ 14	77.6% 21	77.0% 25472	22.4% 1594	23.0% 1.72* ⁻⁴	16s	25s \
>	4.84* ⁻¹ 44	4.84* ⁻¹ 16	78.0% 13	77.9% 28672	22.0% 1599	22.1% 1.80* ⁻⁴	18s	23s \
>	4.74* ⁻¹ 49	4.79* ⁻¹ 18	78.9% 5	79.0% 31872	21.1% 1612	21.0% 1.88* ⁻⁴	20s	21s \
>	4.80* ⁻¹ 54	4.70* ⁻¹ 19	78.2% 26	78.7% 35072	21.8% 1606	21.3% 1.95* ⁻⁴	22s	19s \
>	4.66* ⁻¹ 59	4.71* ⁻¹ 21	79.2% 18	78.6% 38272	20.8% 1616	21.4% 2.01* ⁻⁴	24s	17s \
>	4.65* ⁻¹ 64	4.63* ⁻¹ 23	79.2% 10	79.4% 41472	20.8% 1612	20.6% 2.07* ⁻⁴	26s	15s \
>	4.57* ⁻¹ 69	4.50* ⁻¹ 25	79.8% 2	80.5% 44672	20.2% 1617	19.5% 2.13* ⁻⁴	28s	13s \
>	4.41* ⁻¹ 74	4.43* ⁻¹ 26	80.9% 23	80.5% 47872	19.1% 1592	19.5% 2.18* ⁻⁴	30s	11s \
>	4.40* ⁻¹ 79	4.43* ⁻¹ 28	81.0% 16	81.2% 51136	19.0% 1622	18.8% 2.23* ⁻⁴	32s	9s \
>	4.30* ⁻¹ 84	4.31* ⁻¹ 30	81.3% 8	81.5% 54336	18.7% 1604	18.5% 2.27* ⁻⁴	34s	7s \
>	4.21* ⁻¹	4.16* ⁻¹	82.2%	82.3%	17.8%	17.7%		


```
NeuralOneAccuracy=0.22973
NeuralBalancedAccuracy=0.5
TCCTMinusNeuralExact=1.
TCCTMinusNeuralProbe=0.38417
NEURAL PIPELINE VALID=True
ATtribution STATUS=DIAGNOSTIC_ONLY_NOT_STRICTLY_SCORED
=====
SummaryFile=E:\TCCT_CODEX_HANDOFF_2026-08-13\S97A_ReadoutBaseline_Development\

> \S124_T5_Output\S124_T5R_recovery_summary.wl
=====
S124-T5R COMPLETE
=====
```

In [138...

InputForm[t5rSummary]

```

Out[138... <|"Stage" -> "S124-T5R", "RecoveryDiagnosticOnly" -> True, "StrictPr
ospective" -> False,\

>
  "ReasonInputShapeFixed" -> True, "ReasonValidationAccuracy" ->\

>    0.8236842105263158,
  "ReasonValidationZeroAccuracy" -> 0.9233576642335767,

>    "ReasonValidationOneAccuracy" -> 0.5660377358490566,

>    "ReasonValidationBalancedAccuracy" -> 0.7446977000413166,

>    "ValidationZeroOnlyBaseline" -> 0.7210526315789474, "HighOrderS
tates" -> 74,

>    "TCCTHighOrderExactAccuracy" -> 1., "NeuralHighOrderExactAccura
cy" -> 0.,

>    "TCCTProbeAccuracy" -> 1., "NeuralProbeAccuracy" -> 0.615830115
8301158,

>    "NeuralZeroAccuracy" -> 0.7702702702702703, "NeuralOneAccuracy"
->\

>    0.22972972972972974,
  "NeuralBalancedAccuracy" -> 0.5, "TCCTMinusNeuralExact" ->\

>    1.,
  "TCCTMinusNeuralProbe" -> 0.3841698841698842, "NeuralPipelineValid"
-> True,

>    "AttributionStatus" -> "DIAGNOSTIC_ONLY_NOT_STRICTLY_SCORED"|>

```

In []: